

Nano-Processor

CS1050 - Computer Organisation & Digital Design

Team: WCSS NanoProcessor Group

240061C Bandaranayake I.B.W.D

240066V Botheju P.V.C.N.P

240092V Dayarathna A.M.S.T

240060X Bandara W.B.S.N

Part 1 - Introduction & Objectives

1.1 Background

Modern computing systems, regardless of their complexity, are built upon a surprisingly small set of fundamental operations: arithmetic, data movement, and conditional branching. The complexity of a billion-transistor processor is ultimately an engineering refinement of these same ideas.

Understanding how these operations are composed into a working system from individual logic gates is one of the most illuminating exercises a computer engineering student can undertake.

This laboratory project, completed as the final assignment for **CS1050 - Computer Organisation & Digital Design**, required us to design, implement, and verify a functioning digital processor from scratch using VHDL. We were not given an existing core to modify - we built every component: the arithmetic unit, the register file, the program counter, the instruction decoder, and the display subsystem. Each piece was individually designed, tested in simulation, and then assembled into a complete system.

The processor was implemented on a **Xilinx Basys3 FPGA development board**, which provides a real hardware target with physical LEDs, seven-segment displays, and switches that make the processor's behaviour directly observable without needing an oscilloscope or logic analyser.

1.2 What Is a NanoProcessor?

The term "NanoProcessor" as used in this lab refers to a **minimally-featured stored-program processor** - one that is architecturally complete (it has a fetch-decode-execute cycle) but deliberately limited in scope to be tractable to design from scratch in a laboratory setting.

Our NanoProcessor operates on a small number of registers, a compact program memory, and a minimal instruction set. This deliberate simplicity is the point: every signal path and design decision can be understood and traced in full by the team that built it.

1.3 Project Structure

The lab was structured as a **two-phase project**:

Phase 1 - Base NanoProcessor (12-bit)

- A 12-bit instruction word
- A 2-bit opcode space (4 instructions)
- An 8×4 -bit register file
- A 3-bit program counter (8 instruction ROM)
- A single-operation ALU (add/subtract)
- A seven-segment display showing the R7 output

Phase 2 - Extended NanoProcessor (14-bit)

- A 14-bit instruction word
- A 4-bit opcode space (12 instructions)
- An expanded ALU (9 operations: arithmetic, bitwise, comparison)
- Hardware switch input via the **IN** instruction
- A dual execution mode: **automatic ROM execution and a fully interactive manual mode where the user assembles and executes instructions using the board's switches**

1.4 Objectives

By the end of this project, the team aimed to:

1. **Design each component from first principles** - starting from a full adder and D flip-flop, building upward through a register file, ALU, instruction decoder, and complete processor top-level.
2. **Write testbenches for every component** - verifying correct behaviour in simulation before hardware integration.
3. **Demonstrate correct hardware operation** - loading the processor onto the Basys3 board and observing expected arithmetic results on the seven-segment display.
4. **Extend the base design** - adding a richer instruction set and the novel manual mode feature.
5. **Document the complete design** - producing a professional technical report suitable as a reference for future students and engineers.

1.5 Tools & Environment

Tool	Purpose
Xilinx Vivado 2018	HDL synthesis, simulation, and bitstream generation
VHDL 2008	Hardware description language for all design and testbench files
Basys3 (Artix-7 XC7A35T)	Target FPGA board for hardware testing

Git	Version control and team collaboration
Antigravity	VHDL editing

1.6 Team & Contributions

This project was completed as a team effort for the CS1050 final lab. All team members contributed to component design, testbench writing, simulation, and hardware integration. The extended processor's manual mode was a team-specific design feature not required by the base specification, added to demonstrate deeper understanding of the control architecture.

1.7 Report Structure

This report is organised into seven parts, each covering a distinct aspect of the project:

- **Part 2** explains the overall system architecture and data flow at a block-diagram level.
- **Part 3** documents each individual VHDL component - its purpose, interface, and internal logic.
- **Part 4** defines the Instruction Set Architecture (ISA) for both the base and extended processors.
- **Part 5** presents the simulation and verification results, with annotated timing diagrams.
- **Part 6** describes the extended 14-bit processor - what changed, why, and how the new features work.
- **Part 7** reflects on the project outcomes, challenges encountered, and lessons learned.
- **Part 8** Appendix: Team Contributions & Machine Code

Part 2 - System Architecture

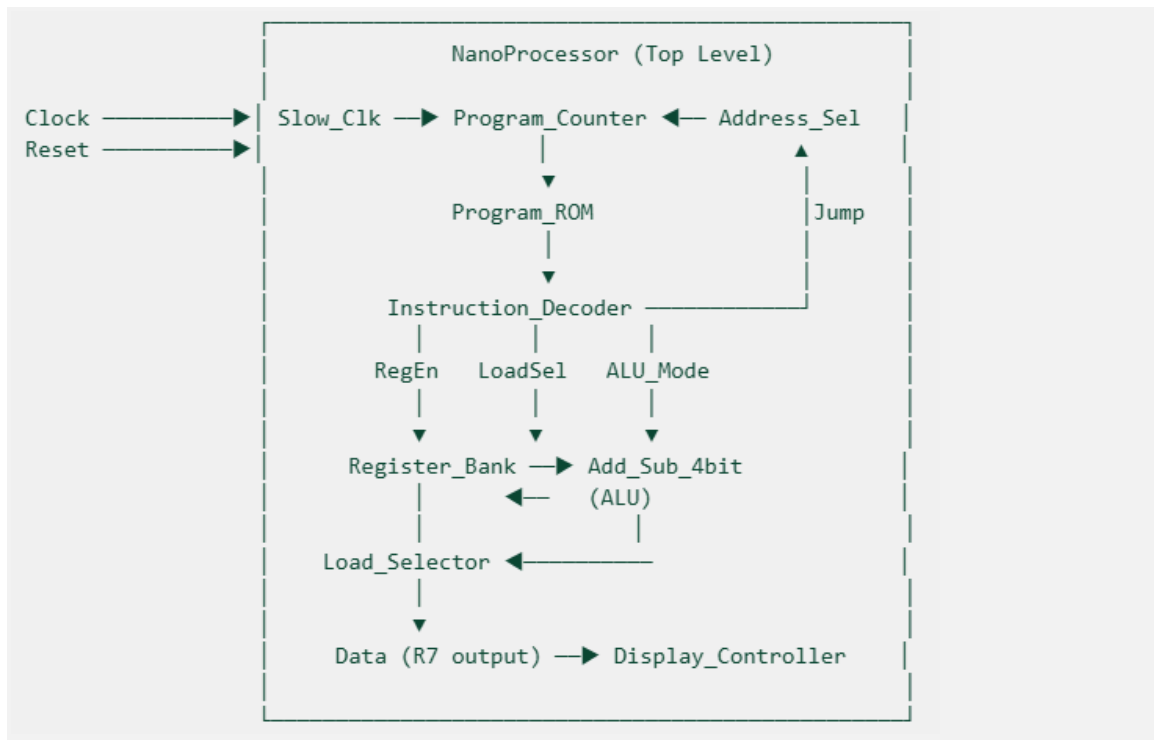
2.1 Design Philosophy

Every design decision in this project was driven by one principle: **keep it hierarchical**. Rather than writing a single monolithic VHDL file that does everything, we decomposed the processor into small, independently testable components, then assembled them bottom-up into a complete system.

This structural approach mirrors how real processors are engineered. It meant that when the overall system had an issue, we could isolate it quickly because every sub-block had already been verified in simulation. It also made team collaboration straightforward - different members could work on different components in parallel without conflict.

2.2 Top-Level Block Diagram

The base NanoProcessor consists of five major functional blocks connected by shared buses:



2.3 Data Flow Description

The processor operates in a single-cycle fetch-decode-execute model. Each clock cycle (after clock division) performs all of the following simultaneously, in a combinational cascade:

Step 1 - Fetch

The **Program Counter** holds the 3-bit address of the current instruction. This address is fed into the **Program ROM**, which combinational outputs the corresponding 12-bit instruction word within nanoseconds.

```
PC_Current[2:0] → Program_ROM → Instruction[11:0]
```

Step 2 - Decode

The **Instruction Decoder** receives the 12-bit instruction and immediately produces all control signals:

- Which register to write to (**Reg_En**)
- Which two registers to read from (embedded in instruction bits)
- Whether to use ALU result or immediate value (**Load_Select**)
- The ALU operation mode (**ALU_Mode**)
- Whether to take a branch (**Jump_Flag**)
- The branch target address (**Jump_Addr**)

This is a **purely combinational** decoder - no clock is involved. All signals are available as soon as the instruction arrives.

Step 3 - Read Registers

The **Register Bank** continuously outputs all 8 register values. Two **RegisterData_Multiplexers** select the required operands A and B by reading the register addresses embedded in the instruction bits.

Step 4 - Execute

The selected operands are passed to the **ALU** (**Add_Sub_4bit**). The ALU computes the result combinational and also generates the **Zero Flag** and **Overflow Flag**.

Step 5 - Write Back

The **Load_Selector** mux decides what data to write to the destination register:

- If **Load_Select** = '0' → write the 4-bit immediate from the instruction (MOVI)
- If **Load_Select** = '1' → write the ALU result (ADD / NEG)

The write is clocked - it only occurs on the next rising edge of the slow clock.

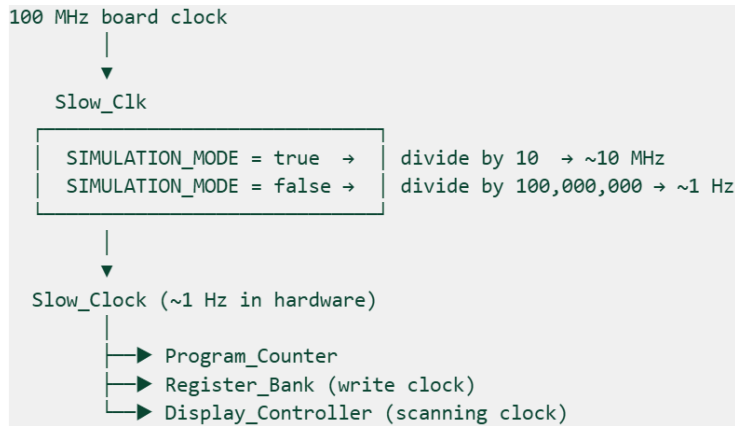
Step 6 - PC Update

The **Address_Selector** mux decides the next PC value:

- Normal: $PC + 1$ (from **PC_Adder**)
- Jump: the 3-bit address embedded in the JZR instruction (if Zero Flag is set)

2.4 Clock Architecture

The Basys3 board provides a 100 MHz oscillator. Running the processor at this speed would make the LEDs appear static to the human eye and make hardware debugging impossible. We therefore use a **Slow_Clk** component that divides the clock.



The `SIMULATION_MODE` generic is a critical design feature. Without it, a behavioural simulation would need to run for 50,000,000 simulation cycles just to observe a single processor instruction - which would take minutes. With it set to `true`, the division ratio drops to 5, making the simulation run in hundreds of nanoseconds.

2.5 Bus Structure

All inter-component data paths use a small set of standardised types defined in `BusDefinitions.vhd`:

Type Name	Width	Used For
<code>DataBus</code>	4-bit	Register data, ALU operands, immediate values
<code>ProgramCounter</code>	3-bit	PC values, jump addresses, ROM address
<code>RegisterSelect</code>	3-bit	Register addresses (R0-R7)
<code>RegisterFile</code>	8 × 4-bit	All register outputs simultaneously
<code>InstructionWord</code>	12-bit (base) / 14-bit (extended)	Full instruction

These type aliases make the code self-documenting and enable the VHDL compiler to catch connection errors at elaboration time rather than at simulation run time.

2.6 R0 - The Zero Register Convention

A deliberate architectural decision was made to make **R0 permanently zero**. This is implemented by hardwiring `En = '1'` and `D = "0000"` to the R0 register, meaning it is constantly being overwritten with zero on every clock edge.

This convention, borrowed from RISC architectures like MIPS and RISC-V, simplifies instruction encoding significantly:

- `JZR R0, addr` becomes an **unconditional jump** (R0 is always zero, so the zero condition is always true)
- The immediate-load instruction `MOVI` can be encoded as `MOVI Rx, imm` using the R0 slot for what would otherwise be an unused source field

2.7 Single-Cycle Design Implications

By completing fetch, decode, execute, and write-back all within one (slow) clock cycle, we avoid the complexity of pipeline hazards, stalls, and forwarding logic. The trade-off is that the critical path - the longest combinational path from ROM output to register write - determines the maximum achievable clock frequency. For the Basys3 at synthesis, this is well above 100 MHz, so the design has ample timing margin.

2.8 Output Subsystem

The primary output of the processor is **Register R7**. This is displayed on the Basys3's four-digit seven-segment display.

The **Display_Controller** implements time-division multiplexing:

```
4-digit display has shared segment lines
→ only one digit is active at any time
→ cycling through all 4 at high speed (~480 Hz) creates the illusion all are on

20-bit counter → bits [19:18] → 4 states → select one digit per state

Digit 0 (rightmost): |R7| absolute value → converted via LUT_16_7
Digit 1:             "-" sign if R7[3] = 1 (negative in 2's complement), else blank
Digit 2, 3:         Always blank
```

The seven-segment LUT (LUT_16_7) converts 4-bit binary to the 7-bit segment pattern using a case statement with all 16 possible digit encodings pre-computed. This is more reliable than computing the segments algorithmically, which is prone to edge-case errors.

2.9 Hardware I/O - Basys3 Pin Assignments

The following table documents the complete I/O mapping between the processor's logical signals and the physical Basys3 board resources, as defined in the XDC constraints file.

Clock & Control

Signal	Board Resource	Package Pin
Clock	100 MHz oscillator	W5
Reset	BTNC (centre button)	U18

Data Output (Register R7 on LEDs LD0–LD3)

Signal	LED	Package Pin	Description
Data[0]	LD0	U16	R7 bit 0 (LSB)
Data[1]	LD1	E19	R7 bit 1
Data[2]	LD2	U19	R7 bit 2
Data[3]	LD3	V19	R7 bit 3 (MSB)

Status Flags

Signal	LED	Package Pin	Description
Zero	LD14	U14	ALU result = 0
Overflow	LD15	V14	Signed arithmetic overflow

Extended Processor - Comparison Flags

Signal	LED	Package Pin	Description
CMP_Equal	LD8	V13	CMP found R1 = R2
CMP_Less	LD9	V3	CMP found R1 < R2
CMP_Greater	LD10	W3	CMP found R1 > R2

Seven-Segment Display

Signal	Package Pin	Description
S_7Seg[0]	W7	Segment a
S_7Seg[1]	W6	Segment b
S_7Seg[2]	U8	Segment c
S_7Seg[3]	V8	Segment d
S_7Seg[4]	U5	Segment e
S_7Seg[5]	V5	Segment f
S_7Seg[6]	U7	Segment g (middle bar - used for minus sign)
anode[0]	U2	Digit 0 enable (active low)
anode[1]	U4	Digit 1 enable (active low)
anode[2]	V4	Digit 2 enable (active low)
anode[3]	W4	Digit 3 enable (active low)

Part 3 - Component Design & Implementation

3.1 Design Hierarchy

All components were built bottom-up across four levels of abstraction. Each level depends only on the level below it, keeping the design modular and individually testable.

```
Level 0: Primitive gates
    FA · D_FF

Level 1: Basic functional units
    RCA_4 · Reg · Decoder_3to8 · LUT_16_7 · Mux_2way_* · Mux_8way_4bit

Level 2: Subsystems
    Add_Sub_4bit · Register_Bank · Program_Counter

Level 3: Processor subsystems
    PC_Adder · Address_Selector · Load_Selector · RegisterData_Multiplexer
    Instruction_Decoder · Program_ROM · Display_Controller · Slow_Clk

Level 4: Top level
    NanoProcessor (base)
    Top_NanoProcessor_14bit (extended)
```

3.2 Full Adder (FA)

Purpose

The Full Adder is the most fundamental building block. It adds two single bits together with a carry-in, producing a sum bit and a carry-out bit. All multi-bit addition in the processor traces back to this component.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FA is
    Port ( A      : in  STD_LOGIC;
          B      : in  STD_LOGIC;
          C_in   : in  STD_LOGIC;
          S      : out STD_LOGIC;
          C_out  : out STD_LOGIC);
end FA;

architecture Behavioral of FA is
begin
    S      <= A xor B xor C_in;
    C_out <= (A and B) or (A and C_in) or (B and C_in);
end Behavioral;
```

The sum bit uses exclusive-OR: odd parity of three inputs. The carry-out bit uses majority logic: carry propagates if at least two of the three inputs are high.

Truth Table

A	B	C_in	S	C_out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

3.3 D Flip-Flop (D_FF)

Purpose

The D Flip-Flop is the fundamental state storage element. On each rising clock edge, it captures the input D and holds it until the next edge. Asynchronous reset overrides the clock.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_FF is
    Port ( D      : in  STD_LOGIC;
          Res     : in  STD_LOGIC;
          Clk     : in  STD_LOGIC;
          Q       : out STD_LOGIC;
          Qbar    : out STD_LOGIC);
end D_FF;

architecture Behavioral of D_FF is
begin
    process(Clk, Res)
    begin
        if Res = '1' then
            Q <= '0';
            Qbar <= '1';
        elsif rising_edge(Clk) then
            Q <= D;
            Qbar <= not D;
        end if;
    end process;
end Behavioral;
```

Condition	Q	Qbar
Res = '1' (any time)	0	1
Res = '0' Rising Clk	D	NOT D
Between clock edges	holds	holds

Reset is asynchronous: it takes effect immediately, not waiting for a clock edge. This is important for the processor reset: asserting `Reset` immediately zeros all state without needing a clock pulse.

Design Note

Three `D_FF` instances are chained together to form the `Program_Counter` - one per bit of the 3-bit PC address. This structural composition is explicit in the VHDL, rather than inferring a register, which ensures the reset behaviour is unambiguous.

3.4 4-bit Ripple Carry Adder (RCA_4)

Purpose

The `RCA_4` chains four Full Adders to add two 4-bit numbers. The carry output of each stage connects to the carry input of the next, forming a ripple.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RCA_4 is
    Port ( A0, A1, A2, A3 : in  STD_LOGIC;
          B0, B1, B2, B3 : in  STD_LOGIC;
          C_in           : in  STD_LOGIC;
          S0, S1, S2, S3 : out STD_LOGIC;
          C_out          : out STD_LOGIC);
end RCA_4;

architecture Structural of RCA_4 is
    component FA
        port ( A, B, C_in : in  STD_LOGIC;
              S, C_out  : out STD_LOGIC);
    end component;

    signal C1, C2, C3 : STD_LOGIC;

begin
    FA0: FA port map (A0, B0, C_in,  S0, C1);
    FA1: FA port map (A1, B1, C1,   S1, C2);
    FA2: FA port map (A2, B2, C2,   S2, C3);
    FA3: FA port map (A3, B3, C3,   S3, C_out);
end Structural;
```

3.5 4-bit Register (Reg)

Purpose

Stores one 4-bit value. Used as the basic cell of the 8-register register file. Includes an enable signal so that only the addressed register is updated while the others hold their values.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Reg is
  Port ( D      : in  STD_LOGIC_VECTOR (3 downto 0);
        Res     : in  STD_LOGIC;
        En      : in  STD_LOGIC;
        Clk     : in  STD_LOGIC;
        Q       : out STD_LOGIC_VECTOR (3 downto 0));
end Reg;

architecture Behavioral of Reg is
begin
  process(Clk, Res)
  begin
    if Res = '1' then
      Q <= (others => '0');
    elsif rising_edge(Clk) then
      if En = '1' then
        Q <= D;
      end if;
    end if;
  end process;
end Behavioral;
```

The enable signal is crucial: without it, every register would update on every clock cycle, erasing all stored values.

3.6 3-to-8 Decoder (Decoder_3to8)

Purpose

Converts a 3-bit binary select (register address) into a one-hot 8-bit enable signal. Exactly one bit of the output is high, corresponding to the selected register. This is used to generate the write-enable signals for the 8 registers in the register bank.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Decoder_3to8 is
  Port ( I : in  STD_LOGIC_VECTOR (2 downto 0);
```

```

        EN : in  STD_LOGIC;
        Y : out STD_LOGIC_VECTOR (7 downto 0));
end Decoder_3to8;

architecture Behavioral of Decoder_3to8 is
begin
    process(I, EN)
    begin
        if EN = '1' then
            case I is
                when "000" => Y <= "00000001";
                when "001" => Y <= "00000010";
                when "010" => Y <= "00000100";
                when "011" => Y <= "00001000";
                when "100" => Y <= "00010000";
                when "101" => Y <= "00100000";
                when "110" => Y <= "01000000";
                when "111" => Y <= "10000000";
                when others => Y <= "00000000";
            end case;
        else
            Y <= "00000000";
        end if;
    end process;
end Behavioral;

```

Truth Table (EN = '1')

Input I[2:0]	Output Y[7:0]	Register Enabled
000	00000001	R0
001	00000010	R1
010	00000100	R2
011	00001000	R3
100	00010000	R4
101	00100000	R5
110	01000000	R6
111	10000000	R7

When EN = '0', all outputs are '0' - no register is written. This is used during jump instructions and other cases where the instruction has no register write-back.

Key Detail

Y(0) is connected to R0's enable, but R0's enable input is **hardwired to '1'** in the register bank. This means R0's write enable is always active, and since its data input is always "0000", R0 is permanently zero regardless of what the decoder outputs.

3.7 Add/Subtract Unit (Add_Sub_4bit)

Purpose

This is the complete ALU of the base processor. It performs 4-bit signed addition and subtraction using the classic two's complement trick: subtraction is computed as $A + (\sim B) + 1$.

Interface

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity Add_Sub_4bit is
    Port ( Input_A      : in  DataBus;
          Input_B      : in  DataBus;
          Mode_Sel     : in  STD_LOGIC;
          Result       : out DataBus;
          Zero_Flag    : out STD_LOGIC;
          Overflow_Flag : out STD_LOGIC);
end Add_Sub_4bit;

architecture Behavioral of Add_Sub_4bit is
    component RCA_4
        port ( A0, A1, A2, A3 : in  STD_LOGIC;
              B0, B1, B2, B3 : in  STD_LOGIC;
              C_in           : in  STD_LOGIC;
              S0, S1, S2, S3 : out STD_LOGIC;
              C_out          : out STD_LOGIC);
    end component;

    signal Modified_B : DataBus;
    signal Temp_Result : DataBus;
    signal RCA_Cout : STD_LOGIC;

begin
    Modified_B(0) <= Input_B(0) xor Mode_Sel;
    Modified_B(1) <= Input_B(1) xor Mode_Sel;
    Modified_B(2) <= Input_B(2) xor Mode_Sel;
    Modified_B(3) <= Input_B(3) xor Mode_Sel;

    -- RCA Instantiation
    RCA_inst: RCA_4 port map (
        A0 => Input_A(0), A1 => Input_A(1), A2 => Input_A(2), A3 => Input_A(3),
        B0 => Modified_B(0), B1 => Modified_B(1), B2 => Modified_B(2), B3 =>
Modified_B(3),
        C_in => Mode_Sel,
```

```

    S0 => Temp_Result(0), S1 => Temp_Result(1), S2 => Temp_Result(2), S3 =>
Temp_Result(3),
    C_out => RCA_Cout
);

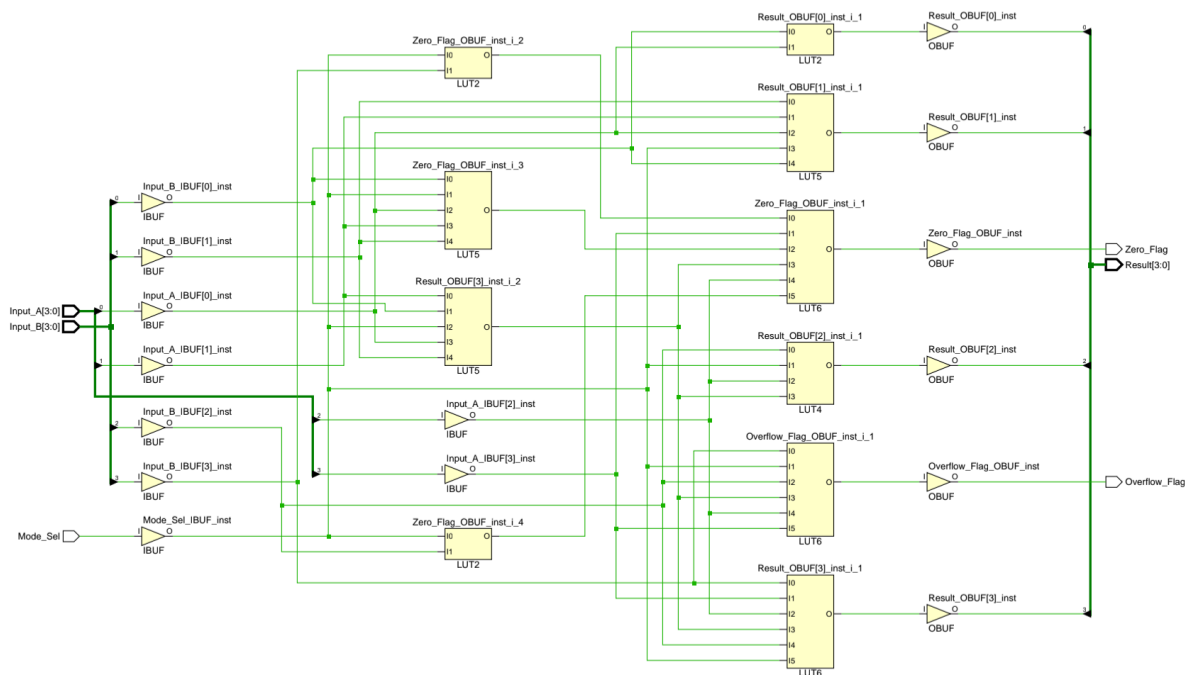
-- Zero flag detection
Zero_Flag <= '1' when Temp_Result = "0000" else '0';

-- Overflow detection for signed 2's complement
process(Input_A, Input_B, Temp_Result, Mode_Sel)
begin
    if Mode_Sel = '0' then -- Addition
        Overflow_Flag <= (Input_A(3) and Input_B(3) and not Temp_Result(3)) or
            (not Input_A(3) and not Input_B(3) and Temp_Result(3));
    else -- Subtraction
        Overflow_Flag <= (Input_A(3) and not Input_B(3) and not Temp_Result(3)) or
            (not Input_A(3) and Input_B(3) and Temp_Result(3));
    end if;
end process;

Result <= Temp_Result;

end Behavioral;

```



How Subtraction Works

The XOR trick: each bit of B is XORed with Mode_Sel. When Mode_Sel = '1':

- $\text{Modified_B}[i] = B[i] \text{ XOR } 1 = \text{NOT } B[i]$ - this inverts B
- $C_{in} = \text{Mode_Sel} = '1'$ - this adds 1

Together: $A + \text{NOT}(B) + 1 = A + (-B) = A - B$ ✓

This means **the same RCA_4 adder handles both ADD and SUB** with no additional hardware.

Overflow Detection

In signed 4-bit arithmetic, overflow occurs when the true result exceeds the representable range $[-8, +7]$. It cannot be detected from the result alone - it requires examining the sign bits of the operands and result:

```
For ADD:  OVF = (A[3] AND B[3] AND NOT R[3])  ← two positives give negative
           OR (NOT A[3] AND NOT B[3] AND R[3]) ← two negatives give positive

For SUB:  OVF = (A[3] AND NOT B[3] AND NOT R[3]) ← positive minus negative goes
negative
           OR (NOT A[3] AND B[3] AND R[3])      ← negative minus positive goes
positive
```

The Zero Flag is simply: Zero = '1' when Result = "0000".

3.8 Register Bank

Purpose

The register bank holds the processor's 8 working registers (R0-R7). It is the primary scratchpad memory: all ALU operations read from and write to this component.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity Register_Bank is
  Port ( Data      : in  DataBus;
        Reset      : in  STD_LOGIC;
        Reg_En     : in  RegisterSelect;
        Clock      : in  STD_LOGIC;
        Register_Outputs : out RegisterFile);
end Register_Bank;

architecture Behavioral of Register_Bank is
  component Decoder_3to8
    Port ( I : in  STD_LOGIC_VECTOR (2 downto 0);
          EN : in  STD_LOGIC;
          Y : out STD_LOGIC_VECTOR (7 downto 0));
  end component;

  component Reg
    Port ( D : in  STD_LOGIC_VECTOR (3 downto 0);
          Res : in  STD_LOGIC;
          En : in  STD_LOGIC;
          Clk : in  STD_LOGIC;
          Q : out STD_LOGIC_VECTOR (3 downto 0));
  end component;

  -- Implementation details would follow here
end architecture;
```

```

    signal Reg_Sel : STD_LOGIC_VECTOR (7 downto 0);
    signal reg_outputs_internal : RegisterFile;

begin
    -- 3-to-8 decoder converts 3-bit select to one-hot
    Decoder: Decoder_3to8 port map (
        I => Reg_En,
        EN => '1',
        Y => Reg_Sel
    );

    -- Register 0 - hardwired to zero
    R0: Reg port map (
        D => "0000",
        Res => Reset,
        En => '1',
        Clk => Clock,
        Q => reg_outputs_internal(0)
    );

    -- Registers 1-7
    R1: Reg port map (D => Data, Res => Reset, En => Reg_Sel(1), Clk => Clock, Q =>
reg_outputs_internal(1));
    R2: Reg port map (D => Data, Res => Reset, En => Reg_Sel(2), Clk => Clock, Q =>
reg_outputs_internal(2));
    R3: Reg port map (D => Data, Res => Reset, En => Reg_Sel(3), Clk => Clock, Q =>
reg_outputs_internal(3));
    R4: Reg port map (D => Data, Res => Reset, En => Reg_Sel(4), Clk => Clock, Q =>
reg_outputs_internal(4));
    R5: Reg port map (D => Data, Res => Reset, En => Reg_Sel(5), Clk => Clock, Q =>
reg_outputs_internal(5));
    R6: Reg port map (D => Data, Res => Reset, En => Reg_Sel(6), Clk => Clock, Q =>
reg_outputs_internal(6));
    R7: Reg port map (D => Data, Res => Reset, En => Reg_Sel(7), Clk => Clock, Q =>
reg_outputs_internal(7));

    Register_Outputs <= reg_outputs_internal;

end Behavioral;

```

Key Design Decision

All register outputs are always available on `Register_Outputs`. The processor reads two registers at a time using external multiplexers. This broadcast-read design is simpler than adding read address ports to the register bank itself, and the extra routing is negligible on an FPGA.

3.9 Program Counter (Program_Counter)

Purpose

The Program Counter keeps track of which instruction the processor should execute next. It is a 3-bit synchronous register (built from 3 D_FF's) that loads its next value on each rising clock edge.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity Program_Counter is
    Port ( PC_Next      : in  ProgramCounter;
          Res          : in  STD_LOGIC;
          Clk          : in  STD_LOGIC;
          PC_Current   : out ProgramCounter);
end Program_Counter;

architecture Behavioral of Program_Counter is
    -- D_FF component declaration
    component D_FF
        Port ( D      : in  STD_LOGIC;
              Res     : in  STD_LOGIC;
              Clk     : in  STD_LOGIC;
              Q       : out STD_LOGIC;
              Qbar    : out STD_LOGIC);
    end component;

    -- Internal signals for flip-flop outputs
    signal q0, q1, q2 : STD_LOGIC;

begin
    -- Instantiate 3 D Flip-Flops (one for each bit)
    DFF0: D_FF port map (D => PC_Next(0), Res => Res, Clk => Clk, Q => q0, Qbar =>
open);
    DFF1: D_FF port map (D => PC_Next(1), Res => Res, Clk => Clk, Q => q1, Qbar =>
open);
    DFF2: D_FF port map (D => PC_Next(2), Res => Res, Clk => Clk, Q => q2, Qbar =>
open);

    -- Combine individual bits into 3-bit output
    PC_Current <= q2 & q1 & q0;

end Behavioral;

```

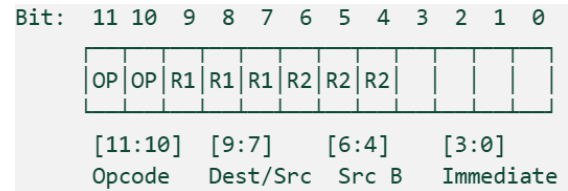
The multiplexing between sequential and jump addresses is handled externally by `Address_Selector`. The PC itself is a dumb register - it just stores whatever it is given.

3.10 Instruction Decoder

Purpose

The instruction decoder is the control unit of the processor. It translates the 12-bit instruction word into all the control signals needed to drive the datapath. This is a purely **combinational** block - no flip-flops, no state.

12-bit Instruction Format (Base Processor)



Output Signals

Signal	Width	Meaning
Reg_En	3-bit	Which register to write
Load_Sel	1-bit	'0' = immediate, '1' = ALU result
ALU_Mode	1-bit	'0' = ADD, '1' = SUB
Jump_Flag	1-bit	Take branch if set
Jump_Addr	3-bit	Branch target address

Decoding Logic (case statement on opcode[11:10])

Opcode	Instruction	Reg_En	Load_Sel	ALU_Mode	Jump_Flag
00	ADD	R1	'1' (ALU)	'0' (add)	'0'
01	NEG	R1	'1' (ALU)	'1' (sub from 0)	'0'
10	MOVI	R1	'0' (imm)	'0'	'0'
11	JZR	none	-	-	Zero_Flag

For **NEG**, the decoder cleverly feeds R0 (always "0000") as operand A and the target register as operand B, with Mode_Sel = '1'. This computes $0 - R_x = -R_x$, implementing negation without any dedicated subtract unit.

For **JZR**, the register enable is de-asserted (set to R0 address, but R0 is hardwired anyway). The Jump_Flag is set to the current Zero_Flag - if the checked register is zero, the branch is taken.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;
use work.Constants.all;

entity Instruction_Decoder is
    port (
        Instruction          : in  InstructionWord;
        Register_Value_For_Jump : in  DataBus;
        Register_Enable      : out RegisterSelect;
        Register_Select_A    : out RegisterSelect;
        Register_Select_B    : out RegisterSelect;
        Operation_Select     : out STD_LOGIC;
        Immediate_Value      : out DataBus;
    );

```

```

        Jump_Enable          : out STD_LOGIC;
        Jump_Address         : out ProgramCounter;
        Load_Select          : out STD_LOGIC
    );
end Instruction_Decoder;

architecture Behavioral of Instruction_Decoder is
    signal Opcode : std_logic_vector(1 downto 0);
begin
    Opcode <= Instruction(11 downto 10);

    decode: process(Opcode, Register_Value_For_Jump, Instruction)
    begin
        -- Default values to prevent latches
        Jump_Enable      <= '0';
        Immediate_Value  <= "0000";
        Load_Select      <= LOAD_IMMEDIATE;
        Register_Enable  <= "000";
        Operation_Select <= ALU_ADD;
        Register_Select_A <= REG_ZERO;
        Register_Select_B <= REG_ZERO;
        Jump_Address     <= "000";

        case Opcode is
            when MOVI_OP => -- MOVI R, d
                Immediate_Value <= Instruction(3 downto 0);
                Load_Select     <= LOAD_IMMEDIATE;
                Register_Enable  <= Instruction(9 downto 7);

            when ADD_OP => -- ADD Ra, Rb
                Register_Select_A <= Instruction(9 downto 7);
                Register_Select_B <= Instruction(6 downto 4);
                Operation_Select  <= ALU_ADD;
                Load_Select       <= LOAD_REGISTER;
                Register_Enable    <= Instruction(9 downto 7);

            when NEG_OP => -- NEG R (0 - R)
                Register_Select_A <= REG_ZERO;
                Register_Select_B <= Instruction(9 downto 7);
                Operation_Select  <= ALU_SUB;
                Load_Select       <= LOAD_REGISTER;
                Register_Enable    <= Instruction(9 downto 7);

            when JZR_OP => -- JZR R, d

```

```

        Register_Select_A <= Instruction(9 downto 7);
        Register_Enable   <= "000";
        if Register_Value_For_Jump = "0000" then
            Jump_Enable    <= '1';
            Jump_Address   <= Instruction(2 downto 0);
        else
            Jump_Enable    <= '0';
        end if;
    when others =>
        null;
    end case;
end process decode;
end Behavioral;

```

3.11 Program ROM

Purpose

The Program ROM stores the processor's program as a fixed array of 8 instructions. It is combinational - given an address, the instruction appears on the output immediately with no clock involved.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.BusDefinitions.all;

entity Program_ROM is
    port( program_counter : in  ProgramCounter;
          instruction_out : out InstructionWord);
end Program_ROM;

architecture Behavioral of Program_ROM is
    type instruction_memory_type is array (0 to 7) of std_logic_vector(11 downto 0);

    signal program_instructions : instruction_memory_type := (
        "101110000000", -- 0: MOVI R7, 0      (R7 = 0)
        "100010000001", -- 1: MOVI R1, 1      (R1 = 1)
        "100100000010", -- 2: MOVI R2, 2      (R2 = 2)
        "100110000011", -- 3: MOVI R3, 3      (R3 = 3)
        "001110010000", -- 4: ADD R7, R1      (R7 = 0+1 = 1)
        "001110100000", -- 5: ADD R7, R2      (R7 = 1+2 = 3)
        "001110110000", -- 6: ADD R7, R3      (R7 = 3+3 = 6)
        "11000000100"  -- 7: JZR R0, 4      (Jump to address 4 - infinite loop)
    );

begin
    instruction_out <= program_instructions(to_integer(unsigned(program_counter)));
end Behavioral;

```

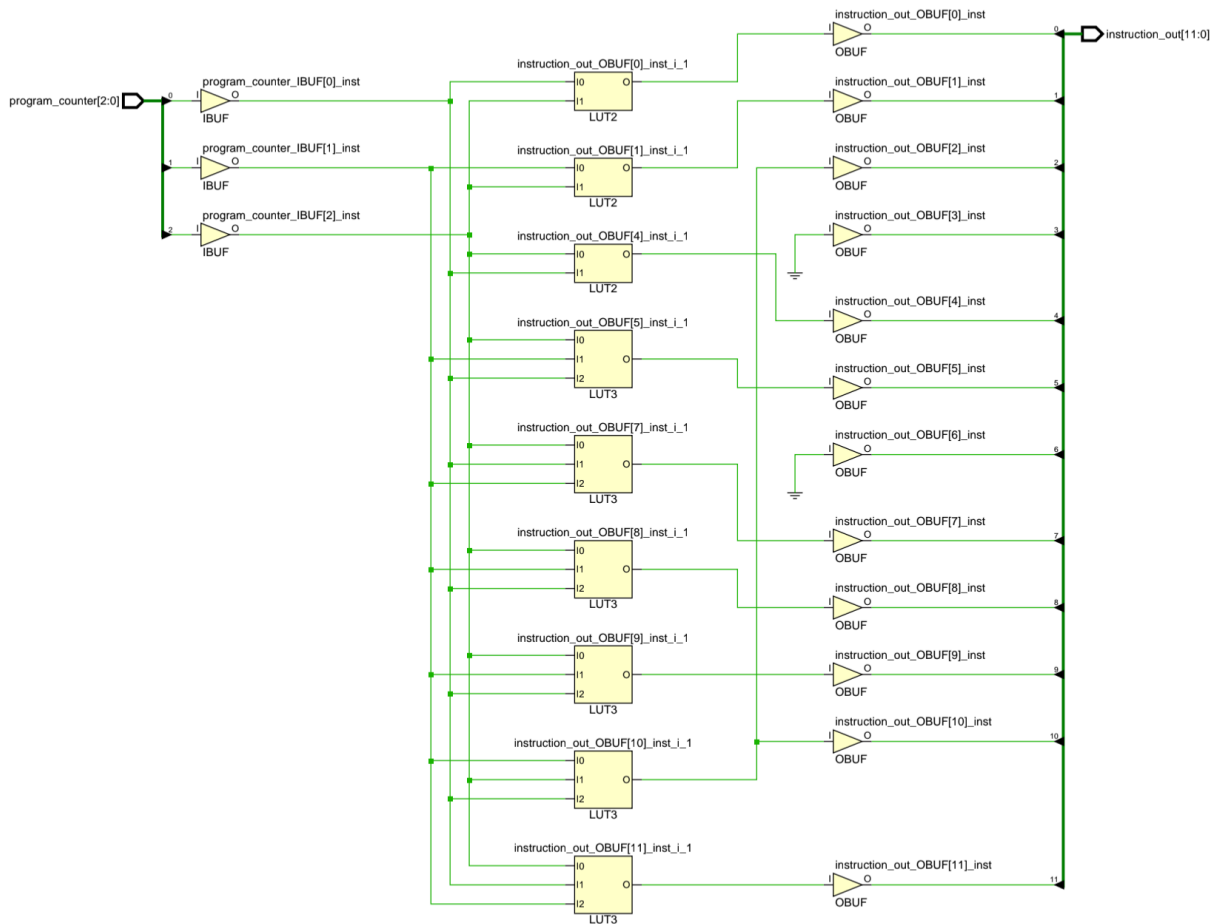
This is a behavioral model of a ROM using a VHDL case statement. Vivado synthesises this as a look-up table (LUT) array on the FPGA - no BRAM is needed for such a small memory.

Demo Program

The ROM contains a program that computes $1 + 2 + 3 = 6$ and stores the result in R7:

Address	Instruction	Assembled
0	MOVI R7, 0	Clear R7
1	MOVI R1, 1	Load 1 into R1
2	MOVI R2, 2	Load 2 into R2
3	MOVI R3, 3	Load 3 into R3
4	ADD R7, R1	$R7 = 0 + 1 = 1$
5	ADD R7, R2	$R7 = 1 + 2 = 3$
6	ADD R7, R3	$R7 = 3 + 3 = 6$
7	JZR R0, 4	Jump to address 4 (loop)

After executing addresses 4–7 repeatedly, R7 cycles: $6 \rightarrow 7 \rightarrow 9$ (overflow) $\rightarrow \dots$ since $R0=0$, the JZR always jumps, making the loop infinite.



3.12 Address Selector & PC Adder

PC Adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.BusDefinitions.all;

entity PC_Adder is
    Port ( current_address : in  ProgramCounter;
          next_address     : out ProgramCounter);
end PC_Adder;

architecture Behavioral of PC_Adder is
begin
    next_address <= std_logic_vector(unsigned(current_address) + 1);
end Behavioral;
```

Address Selector

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity Address_Selector is
    port ( Sequential_Address : in  ProgramCounter;
          Jump_Address       : in  ProgramCounter;
          Jump_Enable        : in  std_logic;
          Selected_Address   : out ProgramCounter);
end Address_Selector;

architecture Behavioral of Address_Selector is
    component Mux_2way_3bit
        port ( Input_0 : in  STD_LOGIC_VECTOR (2 downto 0);
              Input_1 : in  STD_LOGIC_VECTOR (2 downto 0);
              Sel      : in  STD_LOGIC;
              Output   : out STD_LOGIC_VECTOR (2 downto 0));
    end component;
begin
    MUX: Mux_2way_3bit port map (
        Input_0 => Sequential_Address,
        Input_1 => Jump_Address,
        Sel     => Jump_Enable,
        Output  => Selected_Address);
end Behavioral;
```

3.13 Load Selector

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity Load_Selector is
```

```

    Port ( RegisterValue : in  DataBus;
          ImmediateValue : in  DataBus;
          LoadSelect    : in  STD_LOGIC;
          OutputData     : out DataBus);
end Load_Selector;

architecture Behavioral of Load_Selector is
    component Mux_2way_4bit
        port ( Input_0 : in  STD_LOGIC_VECTOR (3 downto 0);
              Input_1 : in  STD_LOGIC_VECTOR (3 downto 0);
              Sel      : in  STD_LOGIC;
              Output   : out STD_LOGIC_VECTOR (3 downto 0));
    end component;
begin
    MUX: Mux_2way_4bit port map (
        Input_0 => ImmediateValue,
        Input_1 => RegisterValue,
        Sel => LoadSelect,
        Output => OutputData
    );
end Behavioral;

```

3.14 Display Controller

Purpose

Drives the Basys3's four-digit common-anode seven-segment display. The display has **shared segment lines** - all four digits share the same 7 segment signals. Only one digit can show a unique value at a time. The controller cycles through all four digits fast enough that persistence of vision makes them appear simultaneously lit.

Time-Division Multiplexing

A 20-bit counter runs at the fast board clock (100 MHz). Bits [19:18] select one of four digits:

```

Counter[19:18] = 00 → Digit 0 (rightmost) → anode = "1110"
Counter[19:18] = 01 → Digit 1           → anode = "1101"
Counter[19:18] = 10 → Digit 2           → anode = "1011"
Counter[19:18] = 11 → Digit 3 (leftmost) → anode = "0111"

```

Note: active-low anode - '0' enables the digit.

The cycling frequency for each digit = $100 \text{ MHz} / 2^{20} / 4 \approx \mathbf{24 \text{ Hz}}$. This is above the flicker fusion threshold (approximately 15 Hz), so the display appears static.

Slow_Clk Component:

```

generic ( SIMULATION_MODE : boolean := true );

```

Mode	Counter limit	Result
true (simulation)	5	very fast clock for quick simulation
false (hardware)	50,000,000	~1 Hz - visible to human eye

This generic must be set to `false` before generating the programming bitstream.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Display_Controller is
    Port ( clk      : in  STD_LOGIC;           -- Fast clock (100MHz)
          Data_in   : in  STD_LOGIC_VECTOR(3 downto 0); -- 4-bit signed value
          seg       : out STD_LOGIC_VECTOR(6 downto 0); -- 7-segment output
          an        : out STD_LOGIC_VECTOR(3 downto 0)); -- Anode controls
end Display_Controller;

architecture Behavioral of Display_Controller is
    component LUT_16_7
        Port ( binary_in : in  STD_LOGIC_VECTOR (3 downto 0);
              seven_seg  : out STD_LOGIC_VECTOR (6 downto 0));
    end component;

    signal abs_value      : STD_LOGIC_VECTOR(3 downto 0);
    signal digit_seg      : STD_LOGIC_VECTOR(6 downto 0);
    signal refresh_counter : unsigned(19 downto 0) := (others => '0');
    signal active_digit   : std_logic_vector(1 downto 0);

    -- Minus sign pattern (active low)
    constant MINUS_PATTERN : std_logic_vector(6 downto 0) := "0111111"; -- Shows "-"
    constant BLANK_PATTERN : std_logic_vector(6 downto 0) := "1111111"; -- All OFF

begin
    -- Convert 2's complement to Absolute Value for display
    process(Data_in)
        variable temp : std_logic_vector(3 downto 0);
    begin
        if Data_in(3) = '1' and Data_in /= "0000" then
            -- Negative number: convert to absolute value (2's complement)
            temp := std_logic_vector(unsigned(not Data_in) + 1);
            abs_value <= temp;
        else
            -- Positive number or zero
            abs_value <= Data_in;
        end if;
    end process;

```

```

        end if;
    end process;

    -- LUT instance to get 7-segment pattern for the digit
    My_LUT: LUT_16_7 port map(
        binary_in => abs_value,
        seven_seg => digit_seg
    );

    process(clk)
    begin
        if rising_edge(clk) then
            refresh_counter <= refresh_counter + 1;
        end if;
    end process;

    active_digit <= std_logic_vector(refresh_counter(19 downto 18)); -- 4 states

    -- Multiplexing display control
    process(active_digit, digit_seg, Data_in)
    begin
        case active_digit is
            -- Rightmost digit (Digit 0): Show the number value
            when "00" =>
                an <= "1110";      -- Enable only digit 0
                seg <= digit_seg;  -- Show absolute value

            -- Second digit (Digit 1): Show minus sign if negative
            when "01" =>
                an <= "1101";      -- Enable only digit 1
                if Data_in(3) = '1' and Data_in /= "0000" then
                    seg <= MINUS_PATTERN; -- Show "-"
                else
                    seg <= BLANK_PATTERN;  -- Show blank for positive
                end if;

            -- Remaining digits (Digit 2 and 3): Turn OFF
            when "10" =>
                an <= "1011";      -- Enable only digit 2
                seg <= BLANK_PATTERN;

            when "11" =>
                an <= "0111";      -- Enable only digit 3
                seg <= BLANK_PATTERN;
        end case;
    end process;

```

```

        when others =>
            an <= "1111";
            seg <= BLANK_PATTERN;
        end case;
    end process;

end Behavioral;

```

LUT_16_7

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Display_Controller is
    Port ( clk      : in  STD_LOGIC;           -- Fast clock (100MHz)
          Data_in   : in  STD_LOGIC_VECTOR(3 downto 0); -- 4-bit signed value
          seg       : out STD_LOGIC_VECTOR(6 downto 0); -- 7-segment output
          an        : out STD_LOGIC_VECTOR(3 downto 0)); -- Anode controls
end Display_Controller;

architecture Behavioral of Display_Controller is
    component LUT_16_7
        Port ( binary_in : in  STD_LOGIC_VECTOR (3 downto 0);
              seven_seg  : out STD_LOGIC_VECTOR (6 downto 0));
    end component;

    signal abs_value      : STD_LOGIC_VECTOR(3 downto 0);
    signal digit_seg      : STD_LOGIC_VECTOR(6 downto 0);
    signal refresh_counter : unsigned(19 downto 0) := (others => '0');
    signal active_digit   : std_logic_vector(1 downto 0);

    -- Minus sign pattern (active low)
    constant MINUS_PATTERN : std_logic_vector(6 downto 0) := "0111111"; -- Shows "-"
    constant BLANK_PATTERN : std_logic_vector(6 downto 0) := "1111111"; -- All OFF

begin
    -- Convert 2's complement to Absolute Value for display
    process(Data_in)
        variable temp : std_logic_vector(3 downto 0);
    begin
        if Data_in(3) = '1' and Data_in /= "0000" then
            -- Negative number: convert to absolute value (2's complement)
            temp := std_logic_vector(unsigned(not Data_in) + 1);
            abs_value <= temp;
        end if;
    end process;

```

```

        else
            -- Positive number or zero
            abs_value <= Data_in;
        end if;
    end process;

    -- LUT instance to get 7-segment pattern for the digit
    My_LUT: LUT_16_7 port map(
        binary_in => abs_value,
        seven_seg => digit_seg
    );

    -- Refresh counter for multiplexing (~1kHz at 100MHz)
    process(clk)
    begin
        if rising_edge(clk) then
            refresh_counter <= refresh_counter + 1;
        end if;
    end process;

    active_digit <= std_logic_vector(refresh_counter(19 downto 18)); -- 4 states

    -- Multiplexing display control
    process(active_digit, digit_seg, Data_in)
    begin
        case active_digit is
            -- Rightmost digit (Digit 0): Show the number value
            when "00" =>
                an <= "1110";      -- Enable only digit 0
                seg <= digit_seg;  -- Show absolute value

            -- Second digit (Digit 1): Show minus sign if negative
            when "01" =>
                an <= "1101";      -- Enable only digit 1
                if Data_in(3) = '1' and Data_in /= "0000" then
                    seg <= MINUS_PATTERN; -- Show "-"
                else
                    seg <= BLANK_PATTERN;  -- Show blank for positive
                end if;

            -- Remaining digits (Digit 2 and 3): Turn OFF
            when "10" =>
                an <= "1011";      -- Enable only digit 2
                seg <= BLANK_PATTERN;
        end case;
    end process;

```

```

        when "11" =>
            an <= "0111";      -- Enable only digit 3
            seg <= BLANK_PATTERN;

        when others =>
            an <= "1111";
            seg <= BLANK_PATTERN;
        end case;
    end process;

end Behavioral;

```

3.15 Nano-Processor (Top level)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;
use work.Constants.all;

entity NanoProcessor is
    port( Clock    : in  STD_LOGIC;
          Reset    : in  STD_LOGIC;
          Overflow: out STD_LOGIC;
          Zero     : out STD_LOGIC;
          S_7Seg   : out STD_LOGIC_VECTOR(6 downto 0);
          anode    : out STD_LOGIC_VECTOR(3 downto 0);
          Data     : out DataBus);
end NanoProcessor;

architecture Structural of NanoProcessor is
    -- Clock signals
    signal s_SlowClk : STD_LOGIC;
    signal s_Res     : STD_LOGIC;

    -- Program counter signals
    signal s_PCCurrent   : ProgramCounter;
    signal s_PCNext      : ProgramCounter;
    signal s_JumpAddr    : ProgramCounter;
    signal s_JumpEn      : STD_LOGIC;
    signal s_SelectedAddr : ProgramCounter;

    -- Instruction signals
    signal s_Instruction : InstructionWord;
    signal s_RegEn       : RegisterSelect;

```

```

signal s_RegSelA      : RegisterSelect;
signal s_RegSelB      : RegisterSelect;
signal s_OpSelect     : STD_LOGIC;
signal s_ImmValue     : DataBus;
signal s_LoadSel      : STD_LOGIC;

-- Data path signals
signal s_RegFileOutputs : RegisterFile;
signal s_OperandA, s_OperandB : DataBus;
signal s_ALUResult : DataBus;
signal s_WriteData : DataBus;
signal s_Overflow, s_Zero : STD_LOGIC;

-- Components
component Slow_Clk
    Generic (SIMULATION_MODE : boolean := true);
    Port ( Clk_in : in STD_LOGIC; Clk_out : out STD_LOGIC);
end component;

component Program_Counter
    Port ( PC_Next : in ProgramCounter; Res : in STD_LOGIC; Clk : in STD_LOGIC;
          PC_Current : out ProgramCounter);
end component;

component PC_Adder
    port ( current_address : in ProgramCounter; next_address : out ProgramCounter);
end component;

component Address_Selector
    port ( Sequential_Address : in ProgramCounter; Jump_Address : in
ProgramCounter;
          Jump_Enable : in std_logic; Selected_Address : out ProgramCounter);
end component;

component Program_ROM
    port( program_counter : in ProgramCounter; instruction_out : out
InstructionWord);
end component;

component Instruction_Decoder
    port ( Instruction : in InstructionWord; Register_Value_For_Jump : in DataBus;
          Register_Enable : out RegisterSelect; Register_Select_A : out
RegisterSelect;

```

```

        Register_Select_B : out RegisterSelect; Operation_Select : out
STD_LOGIC;

        Immediate_Value : out DataBus; Jump_Enable : out STD_LOGIC;
        Jump_Address : out ProgramCounter; Load_Select : out STD_LOGIC);
end component;

component Load_Selector
    Port ( RegisterValue : in DataBus; ImmediateValue : in DataBus;
        LoadSelect : in STD_LOGIC; OutputData : out DataBus);
end component;

component Register_Bank
    Port ( Data : in DataBus; Reset : in STD_LOGIC; Reg_En : in RegisterSelect;
        Clock : in STD_LOGIC; Register_Outputs : out RegisterFile);
end component;

component RegisterData_Multiplexer
    Port ( DataSources : in RegisterFile; SelectAddress : in RegisterSelect;
        OutputData : out DataBus);
end component;

component Add_Sub_4bit
    Port ( Input_A : in DataBus; Input_B : in DataBus; Mode_Sel : in STD_LOGIC;
        Result : out DataBus; Zero_Flag : out STD_LOGIC; Overflow_Flag : out
STD_LOGIC);
end component;

-- NEW: Display Controller component
component Display_Controller
    Port ( clk      : in  STD_LOGIC;
        Data_in   : in  STD_LOGIC_VECTOR(3 downto 0);
        seg      : out STD_LOGIC_VECTOR(6 downto 0);
        an       : out STD_LOGIC_VECTOR(3 downto 0));
end component;

begin
    s_Res <= Reset;

    -- Slow clock generation
    -- Set SIMULATION_MODE => true for simulation, false for hardware
    U_SlowClk: Slow_Clk
        generic map (SIMULATION_MODE => false) -- Change to false for BASYS3 board
        port map (Clk_in => Clock, Clk_out => s_SlowClk);

```

```

-- Program Counter and sequencing
U_PC: Program_Counter port map (PC_Next => s_SelectedAddr, Res => s_Res, Clk =>
s_SlowClk, PC_Current => s_PCCurrent);
U_PC_Adder: PC_Adder port map (current_address => s_PCCurrent, next_address =>
s_PCNext);
U_AddrSel: Address_Selector port map (Sequential_Address => s_PCNext, Jump_Address
=> s_JumpAddr,
                                Jump_Enable => s_JumpEn, Selected_Address =>
s_SelectedAddr);

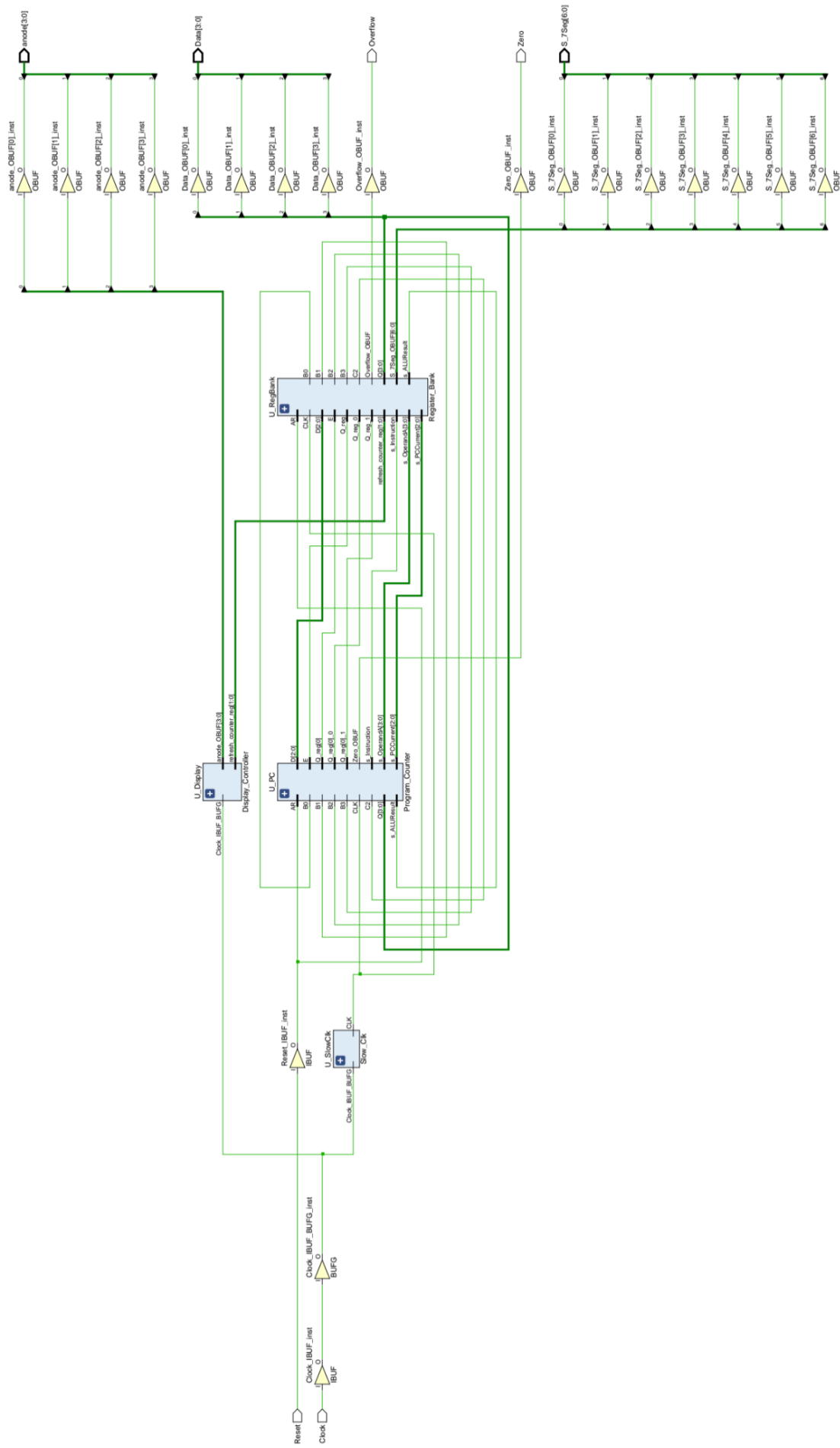
-- ROM and Decoder
U_ROM: Program_ROM port map (program_counter => s_PCCurrent, instruction_out =>
s_Instruction);
U_ID: Instruction_Decoder port map (Instruction => s_Instruction,
Register_Value_For_Jump => s_OperandA,
                                Register_Enable => s_RegEn, Register_Select_A
=> s_RegSelA,
                                Register_Select_B => s_RegSelB,
Operation_Select => s_OpSelect,
                                Immediate_Value => s_ImmValue, Jump_Enable =>
s_JumpEn,
                                Jump_Address => s_JumpAddr, Load_Select =>
s_LoadSel);

-- Data path
U_LoadSel: Load_Selector port map (RegisterValue => s_ALUResult, ImmediateValue =>
s_ImmValue,
                                LoadSelect => s_LoadSel, OutputData =>
s_WriteData);
U_RegBank: Register_Bank port map (Data => s_WriteData, Reset => s_Res, Reg_En =>
s_RegEn,
                                Clock => s_SlowClk, Register_Outputs =>
s_RegFileOutputs);
U_MuxA: RegisterData_Multiplexer port map (DataSources => s_RegFileOutputs,
SelectAddress => s_RegSelA,
                                OutputData => s_OperandA);
U_MuxB: RegisterData_Multiplexer port map (DataSources => s_RegFileOutputs,
SelectAddress => s_RegSelB,
                                OutputData => s_OperandB);
U_ALU: Add_Sub_4bit port map (Input_A => s_OperandA, Input_B => s_OperandB,
Mode_Sel => s_OpSelect,
                                Result => s_ALUResult, Zero_Flag => s_Zero,
Overflow_Flag => s_Overflow);

```

```
-- Outputs
Data <= s_RegFileOutputs(7);
Zero <= s_Zero;
Overflow <= s_Overflow;

-- NEW: 7-segment display controller (shows R7 value with minus sign)
U_Display: Display_Controller port map (
    clk => Clock,           -- Fast clock for multiplexing
    Data_in => s_RegFileOutputs(7), -- R7 value
```



3.16 Constrain (Basic)

```
## Clock signal (100 MHz)
set_property PACKAGE_PIN W5 [get_ports Clock]
set_property IOSTANDARD LVCMOS33 [get_ports Clock]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports Clock]

## Reset Button (BTNC)
set_property PACKAGE_PIN U18 [get_ports Reset]
set_property IOSTANDARD LVCMOS33 [get_ports Reset]

## LEDs for Output (R7 - LD0 to LD3)
set_property PACKAGE_PIN U16 [get_ports {Data[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[0]}]
set_property PACKAGE_PIN E19 [get_ports {Data[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[1]}]
set_property PACKAGE_PIN U19 [get_ports {Data[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[2]}]
set_property PACKAGE_PIN V19 [get_ports {Data[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[3]}]

## Zero Flag (LD14)
set_property PACKAGE_PIN U14 [get_ports Zero]
set_property IOSTANDARD LVCMOS33 [get_ports Zero]

## Overflow Flag (LD15)
set_property PACKAGE_PIN V14 [get_ports Overflow]
set_property IOSTANDARD LVCMOS33 [get_ports Overflow]

## 7-Segment Display - Segments (active low)
set_property PACKAGE_PIN W7 [get_ports {S_7Seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {S_7Seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {S_7Seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {S_7Seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {S_7Seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {S_7Seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {S_7Seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[6]}]
```

```
## 7-Segment Display - Anodes
set_property PACKAGE_PIN U2 [get_ports {anode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[0]}]
set_property PACKAGE_PIN U4 [get_ports {anode[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[1]}]
set_property PACKAGE_PIN V4 [get_ports {anode[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[2]}]
set_property PACKAGE_PIN W4 [get_ports {anode[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[3]}]
```

Part 4 - Instruction Set Architecture

4.1 What Is an ISA?

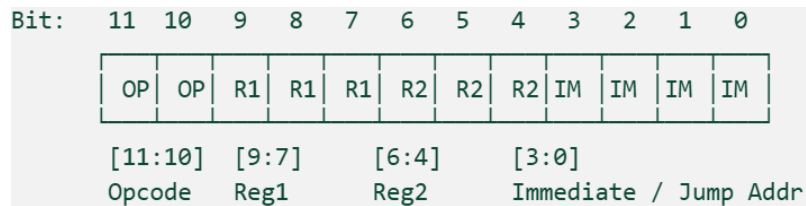
The Instruction Set Architecture is the contract between software and hardware. It defines what operations the processor can perform, how those operations are encoded into binary, and what effect each instruction has on the machine state (registers, flags, program counter). A programmer or compiler targeting this processor must only know the ISA - they do not need to know whether the underlying implementation uses a ripple carry adder or a carry lookahead adder.

Designing an ISA is a balancing act. More instructions give the programmer more expressive power but require wider opcodes, more decoder logic, and longer instruction words. Fewer instructions simplify the hardware but force the programmer to use multi-instruction sequences for common operations. The two processors in this project explore both ends of this spectrum deliberately.

4.2 Base NanoProcessor ISA (12-bit, 4 Instructions)

4.2.1 Instruction Format

Every instruction is exactly 12 bits wide. The format is fixed - the same fields appear in the same bit positions for every instruction:



Field	Bits	Width	Description
OP	[11:10]	2 bits	Instruction type (opcode)
R1	[9:7]	3 bits	Destination register / primary source
R2	[6:4]	3 bits	Source register
IMM	[3:0]	4 bits	4-bit immediate value
ADDR	[2:0]	3 bits	Jump target (within IMM field, lower 3 bits)

Using a fixed-width instruction format means the decoder has no need for a variable-length parser. Every field is always in the same place, simplifying both hardware and manual assembly.

4.2.2 Instruction Reference

`ADD` - Add Two Registers

Opcode: 00

Encoding: 00 | R1[2:0] | R2[2:0] | 0000

Operation:

```
R1 ← R1 + R2
Zero_Flag ← (result == 0)
Overflow_Flag ← signed overflow
```

Example: ADD R7, R1 → 00 111 001 0000 → 0x390 (hex)

This is the primary computational instruction. The processor adds the contents of two registers and stores the result back in R1. Both Zero and Overflow flags are updated.

When to use overflow flag: If computing $5 + 4 = 9$, but 9 (1001) in 4-bit signed is -7 . The overflow flag will be set to warn that the result is not arithmetically correct in the signed domain.

`NEG` - Negate a Register

Opcode: 01

Encoding: 01 | R1[2:0] | R2[2:0] | 0000

Operation:

```
R1 ← 0 - R2      (equivalently: R1 ← two's complement of R2)
Zero_Flag ← (result == 0)
Overflow_Flag ← signed overflow (only if R2 = -8)
```

Example: NEG R7 (negate R7, store in R7) → 01 111 111 0000 → 0x7F0

Implementation trick: The decoder sets Input_A = R0 (always "0000") and Input_B = R2, with ALU Mode_Sel = '1' (subtract). The computation $0 - R2$ is equivalent to negation. This reuses the existing ALU without adding any new hardware - an elegant design choice.

Special case: Negating -8 (1000) produces $+8$, which overflows a 4-bit signed integer (max $+7$). The Overflow Flag is set in this case.

`MOVI` - Move Immediate

Opcode: 10

Encoding: 10 | R1[2:0] | 000 | IMM[3:0]

Operation:

```
R1 ← IMM[3:0]
(No flags updated)
```

Example: MOVI R7, 5 → 10 111 000 0101 → 0xB805 or 1011 1000 0101

This is the only way to load a constant into a register. The 4-bit immediate allows values 0–15 to be loaded (or equivalently, –8 to +7 in signed representation). The R2 field is unused (conventionally written as 000).

Load_Selector role: When the decoder sees opcode 10, it asserts `Load_Sel = '0'`, causing the Load_Selector mux to route the immediate value (rather than the ALU result) to the register write-back path.

‘JZR’ - Jump If Zero Register

Opcode: 11

Encoding: 11 | R1[2:0] | 000 | 0 | ADDR[2:0]

Operation:

```
If R1 == "0000":
    PC ← ADDR[2:0]    (branch taken)
Else:
    PC ← PC + 1       (sequential)
(No registers updated, no flags updated)
```

Example: `JZR R0, 4` → 11 000 000 0100 → jump to address 4 always (since R0 = 0)

Example: `JZR R7, 0` → jump to address 0 only if R7 = 0

This is the sole control-flow instruction. Used for both conditional branching (test any register for zero) and unconditional looping (test R0, which is always zero).

How the Zero check works: The instruction decoder drives `Jump_Flag` from the real-time Zero output of the ALU... but wait - for `JZR`, we are not doing an ALU operation. The decoder actually reads the Zero Flag produced by the Register_Data_Multiplexer output, checking if the selected register R1 equals zero without routing through the ALU. The check is done on R1's raw value.

4.2.3 Instruction Encoding Table

Assembly	Binary	Hex
<code>MOVI R7, 0</code>	10 111 000 0000	B80
<code>MOVI R1, 1</code>	10 000 100 0001	841
<code>MOVI R2, 2</code>	10 010 000 0010	902
<code>MOVI R3, 3</code>	10 011 000 0011	983
<code>ADD R7, R1</code>	00 111 001 0000	390
<code>ADD R7, R2</code>	00 111 010 0000	3A0
<code>ADD R7, R3</code>	00 111 011 0000	3B0
<code>JZR R0, 4</code>	11 000 000 0100	C04
<code>NEG R7</code>	01 111 111 0000	7F0

4.2.4 Machine State Model

The complete state of the base processor at any instant consists of:

State Element	Width	Reset Value
PC (Program Counter)	3 bits	000
R0	4 bits	0000 (permanent)
R1–R7	4 bits each	0000
Zero_Flag	1 bit	Not stored - combinational
Overflow_Flag	1 bit	Not stored - combinational

Flags are **not stored** in flip-flops. They are combinationaly re-computed on every cycle from the current ALU operation. This means there is no instruction that reads a previously computed flag - the JZR instruction only acts on the current cycle's Zero_Flag output.

4.3 Extended NanoProcessor ISA (14-bit, 12 Instructions)

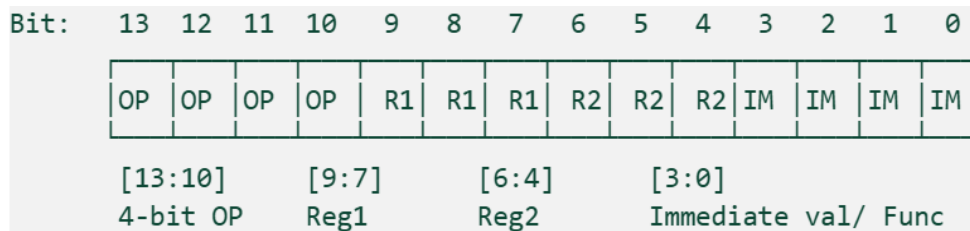
4.3.1 Motivation for the Extension

The base 4-instruction processor, while complete, is severely limited:

- No way to subtract (only negate)
- No bitwise operations (AND, OR, XOR, NOT)
- No way to read external input (no IN instruction)
- No comparison beyond zero-check
- Only 4 instruction types in a 2-bit opcode

The extended processor addresses all of these gaps. The instruction word is widened to 14 bits, giving a 4-bit opcode for 16 possible instructions (12 of which are implemented).

4.3.2 14-bit Instruction Format



The same field layout is preserved - only the opcode is widened from 2 to 4 bits.

4.3.3 Complete Instruction Set

Opcode	Mnemonic	Operation	Flags
0000	MOVI	$R1 \leftarrow \text{IMM}[3:0]$	None
0001	ADD	$R1 \leftarrow R1 + R2$	Zero, Overflow
0010	SUB	$R1 \leftarrow R1 - R2$	Zero, Overflow
0011	NEG	$R1 \leftarrow -R1$	Zero, Overflow

0100	JZR	If R1=0: PC $\leftarrow$ ADDR	None
0101	IN	R1 $\leftarrow$ SW[3:0]	None
0110	MUL	R1 $\leftarrow$ (R1 $\times$ R2) mod 16	Zero, Overflow
0111	AND	R1 $\leftarrow$ R1 AND R2	Zero
1000	OR	R1 $\leftarrow$ R1 OR R2	Zero
1001	XOR	R1 $\leftarrow$ R1 XOR R2	Zero
1010	NOT	R1 $\leftarrow$ NOT R1	Zero
1011	CMP	Compare R1 and R2; update CMP flags	CMP_Eq, CMP_Lt, CMP_Gt

4.3.4 New Instructions Explained

`SUB` - Subtract

Explicit subtraction is now available without the NEG+ADD workaround needed in the base processor.

```
R1  $\leftarrow$  R1 - R2
```

Internally: ALU Mode_Sel = ALU_SUB = "0001".

`IN` - Hardware Input

The IN instruction reads the four physical slider switches (SW0–SW3) directly into a register:

```
R1  $\leftarrow$  switches[3:0]
```

This allows the user to provide runtime data to the processor - numbers can be entered via hardware switches and used in arithmetic. This is the primary mechanism for user interaction in the extended processor's ROM mode.

`MUL` - Multiply

4-bit signed multiplication truncated to the lower 4 bits of the product:

```
R1  $\leftarrow$  (R1  $\times$  R2) mod 16
```

For example: $3 \times 2 = 15$ (0110) - no overflow. $4 \times 4 = 16 \rightarrow$ result is 0000, overflow flag set.

`AND`, `OR`, `XOR`, `NOT` - Bitwise Logic

Standard bitwise operations applied bit-by-bit across the 4-bit operands. Useful for masking, flag manipulation, and logical tests.

```
AND: R1[i] = R1[i] AND R2[i]    for i = 0..3
OR:  R1[i] = R1[i] OR  R2[i]
XOR: R1[i] = R1[i] XOR R2[i]
NOT: R1[i] = NOT R1[i]
```

`CMP` - Compare

The CMP instruction performs a signed comparison between two registers and sets three dedicated flag outputs. Unlike arithmetic instructions, **CMP does not modify any register**.

```
If R1 = R2:  CMP_Equal = 1, CMP_Less = 0, CMP_Greater = 0
If R1 < R2:  CMP_Equal = 0, CMP_Less = 1, CMP_Greater = 0
If R1 > R2:  CMP_Equal = 0, CMP_Less = 0, CMP_Greater = 1
```

The three CMP flags are displayed on dedicated LEDs (LD8, LD9, LD10) on the Basys3 board, giving immediate visual feedback about comparison results.

4.3.5 Flag Architecture (Extended)

The extended processor has five flags:

Flag	Width	Meaning	LED
Zero	1-bit	ALU result is zero	LD14
Overflow	1-bit	Signed arithmetic overflow	LD15
CMP_Equal	1-bit	Last CMP found R1 = R2	LD8
CMP_Less	1-bit	Last CMP found R1 < R2	LD9
CMP_Greater	1-bit	Last CMP found R1 > R2	LD10

All flags come directly from the ALU combinational outputs - they are not stored in flip-flops and are updated every cycle.

4.3.6 Encoding Examples (14-bit)

Assembly	Binary (14-bit)	Description
MOVI R7, 8	0000 111 000 1000	$R7 \leftarrow 8$ (= -8 signed)
MOVI R1, 1	0000 001 000 0001	$R1 \leftarrow 1$
ADD R7, R1	0001 111 001 0000	$R7 \leftarrow R7 + R1$
SUB R1, R2	0010 001 010 0000	$R1 \leftarrow R1 - R2$
NEG R7	0011 111 000 0000	$R7 \leftarrow -R7$
JZR R0, 2	0100 000 000 0010	Jump to addr 2 (always)
IN R7	0101 111 000 0000	$R7 \leftarrow SW[3:0]$
MUL R1, R2	0110 001 010 0000	$R1 \leftarrow R1 \times R2 \pmod{16}$
AND R1, R2	0111 001 010 0000	$R1 \leftarrow R1 \text{ AND } R2$
OR R1, R2	1000 001 010 0000	$R1 \leftarrow R1 \text{ OR } R2$
XOR R1, R2	1001 001 010 0000	$R1 \leftarrow R1 \text{ XOR } R2$
NOT R1	1010 001 000 0000	$R1 \leftarrow \text{NOT } R1$
CMP R1, R2	1011 001 010 0000	Set CMP flags

4.3.7 Example Programs

Program 1 - Count from -8 to +7 (ROM demo)

```
Addr 0: MOVI R7, 8      ; R7 = 1000 = -8 (signed)
Addr 1: MOVI R1, 1      ; R1 = 1
Addr 2: ADD  R7, R1     ; R7 += 1
Addr 3: ADD  R7, R1     ; R7 += 1
Addr 4: ADD  R7, R1     ; R7 += 1
Addr 5: ADD  R7, R1     ; R7 += 1
Addr 6: ADD  R7, R1     ; R7 += 1
Addr 7: JZR  R0, 2      ; always jump to addr 2
```

Output sequence on display: -8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, -8, ...

Program 2 - Bitwise mask with input

```
MOVI R6, -8      ; R6 = 1111 (mask for lower 4 bits)
IN   R7          ; R7 = user switch input
AND  R7, R6      ; mask to lower nibble
JZR  R0, 1       ; loop
```

Program 3 - Compare two values

```
MOVI R1, 5      ; R1 = 5
MOVI R2, 3      ; R2 = 3
CMP  R1, R2     ; CMP_Greater = 1 (5 > 3)
JZR  R0, 0      ; loop
```

Part 5 - Simulation & Verification

5.1 Verification Strategy

Every component in this project was individually verified in simulation before being integrated into the processor. This bottom-up verification strategy is important because:

- A bug in a primitive component (like the Full Adder) will propagate through every higher-level block that uses it, making top-level failures impossible to diagnose without unit-level confidence.
- Testbenches for small components run in nanoseconds and can exhaustively test all input combinations.
- Integration becomes straightforward because each block arrives already verified.

Testbenches were written in **VHDL behavioral style** using `wait` statements and `assert` checks.

Waveforms were captured in the **Xilinx Vivado Simulator** and saved as screenshots.

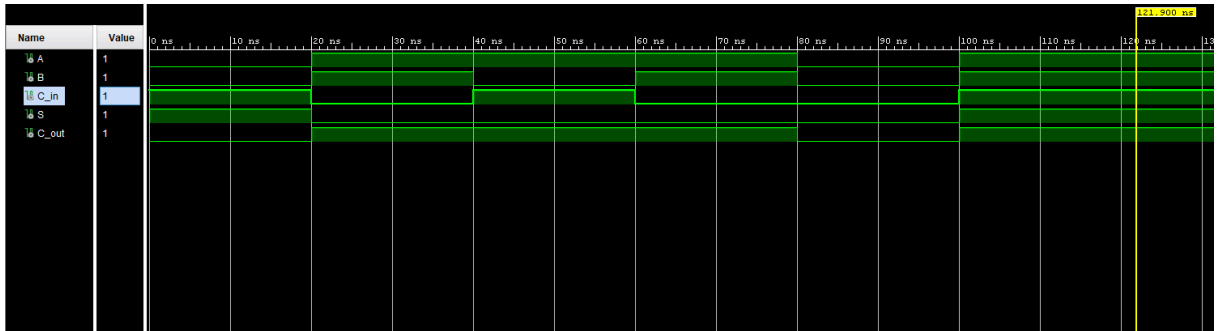
5.2 Testbench Summary

Testbench	Component	What Was Tested
tb_FA.vhd	Full Adder	All 8 input combinations
tb_RCA_4.vhd	4-bit Adder	Multiple add cases, carry propagation
tb_Add_Sub_4bit.vhd	ALU	ADD, SUB, zero flag, overflow flag
tb_Reg.vhd	Register	Write enable, reset, hold
tb_Register_Bank.vhd	Register File	Write to each R1–R7, R0 always zero
tb_decoder_3to8.vhd	Decoder	All 8 addresses, enable
tb_Mux_2way_3bit.vhd	2:1 Mux (3-bit)	Both select states
tb_mux_2way_4bit.vhd	2:1 Mux (4-bit)	Both select states
tb_Mux_8way_4bit.vhd	8:1 Mux	All 8 select positions
tb_Program_Counter.vhd	Program Counter	Reset, sequential, non-sequential load
tb_PC_Adder.vhd	PC Adder	Increment 0→7
tb_Address_Selector.vhd	Address Selector	Sequential vs jump
tb_Program_ROM.vhd	ROM	All 8 addresses → correct instruction
tb_Load_Selector.vhd	Load Selector	Immediate vs ALU selection
tb_Display_Controller.vhd	Display	Positive/negative values, digit cycling
tb_NanoProcessor_TB.vhd	Full NanoProcessor	End-to-end: 1+2+3=6 in R7

5.3 Unit Simulation Results

5.3.1 Full Adder Simulation

The Full Adder testbench applies all 8 possible combinations of A, B, and C_in, verifying both S and C_out at each step.



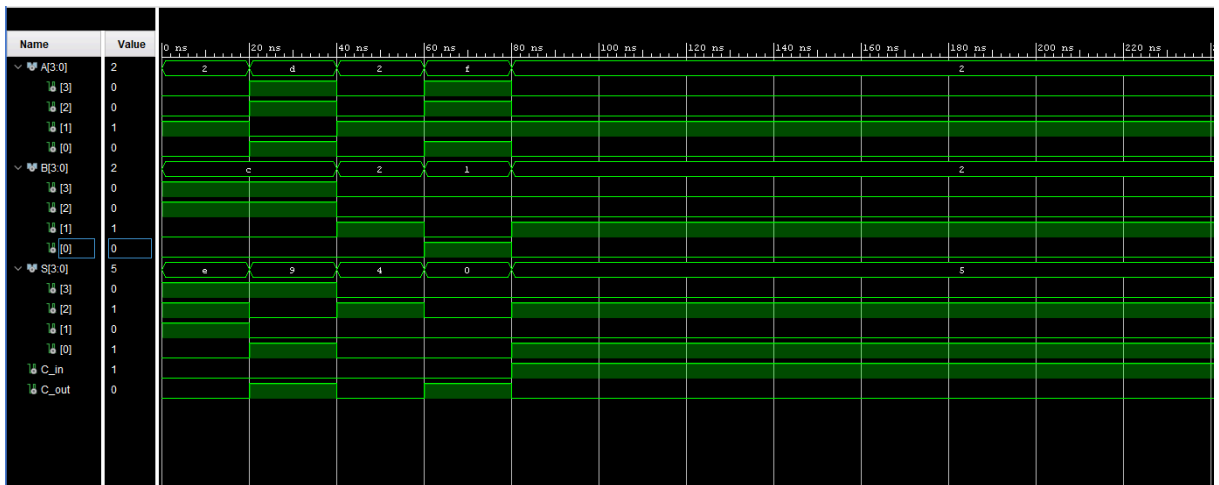
Waveform Analysis:

The waveform confirms the expected truth table behaviour across all eight input states. Notably:

- When A=1, B=1, C_in=1: both S and C_out are '1' (seven, the maximum). ✓
- Carry propagation is immediate - there is no observable propagation delay at the simulation timescale.

5.3.2 4-bit Ripple Carry Adder Simulation

The RCA_4 testbench applies multiple operand pairs and checks the 4-bit sum and carry-out.



Waveform Analysis:

The hexadecimal values visible in the waveform correspond to the input operands A[3:0] and B[3:0], and the output sum S[3:0] and carry-out C_out.

Key observations from the waveform:

- A=2 (0010), B=C (1100) → S=E (1110), C_out=0 - correct: 2+12=14

- A=D (1101), B=2 (0010) → S=F (1111), C_out=0 - correct: 13+2=15
- A=2 (0010), B=1 (0001) → S=3 (0011), C_out=0 - correct: 2+1=3
- A=F (1111), B=1 (0001) → S=0 (0000), C_out=1 - correct: 15+1=16, carry out

All results match expected binary arithmetic.

5.3.3 ALU (Add_Sub_4bit) Simulation

tb_Add_Sub_4bit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity tb_Add_Sub_4bit is
end tb_Add_Sub_4bit;

architecture Behavioral of tb_Add_Sub_4bit is
    component Add_Sub_4bit
        Port ( Input_A, Input_B : in DataBus;
              Mode_Sel : in STD_LOGIC;
              Result : out DataBus;
              Zero_Flag, Overflow_Flag : out STD_LOGIC);
    end component;

    signal A, B, Result : DataBus;
    signal Mode_Sel, Zero_Flag, Overflow_Flag : STD_LOGIC;

begin
    UUT: Add_Sub_4bit port map (
        Input_A => A, Input_B => B, Mode_Sel => Mode_Sel,
        Result => Result, Zero_Flag => Zero_Flag, Overflow_Flag => Overflow_Flag
    );

    process
    begin
        report "Testing 4-bit Add/Sub Unit with Team Index Bits";

        -- ===== ADDITION TESTS =====
        report "--- ADDITION TESTS ---";

        -- Test1: 2 + 12 = 14
        report "Test1: 2 + 12 = 14";
        A <= "0010"; B <= "1100"; Mode_Sel <= '0';
        wait for 20 ns;
```

```

-- Test2: 13 + 12 = 25 (overflow)
report "Test2: 13 + 12 = 25 (overflow)";
A <= "1101"; B <= "1100"; Mode_Sel <= '0';
wait for 20 ns;

-- Test3: 7 + 1 = 8 (overflow in signed)
report "Test3: 7 + 1 = 8 (overflow)";
A <= "0111"; B <= "0001"; Mode_Sel <= '0';
wait for 20 ns;

-- Test4: 0 + 0 = 0 (zero flag)
report "Test4: 0 + 0 = 0 (zero flag)";
A <= "0000"; B <= "0000"; Mode_Sel <= '0';
wait for 20 ns;

-- ===== SUBTRACTION TESTS =====
report "--- SUBTRACTION TESTS ---";

-- Test5: 12 - 2 = 10
report "Test5: 12 - 2 = 10";
A <= "1100"; B <= "0010"; Mode_Sel <= '1';
wait for 20 ns;

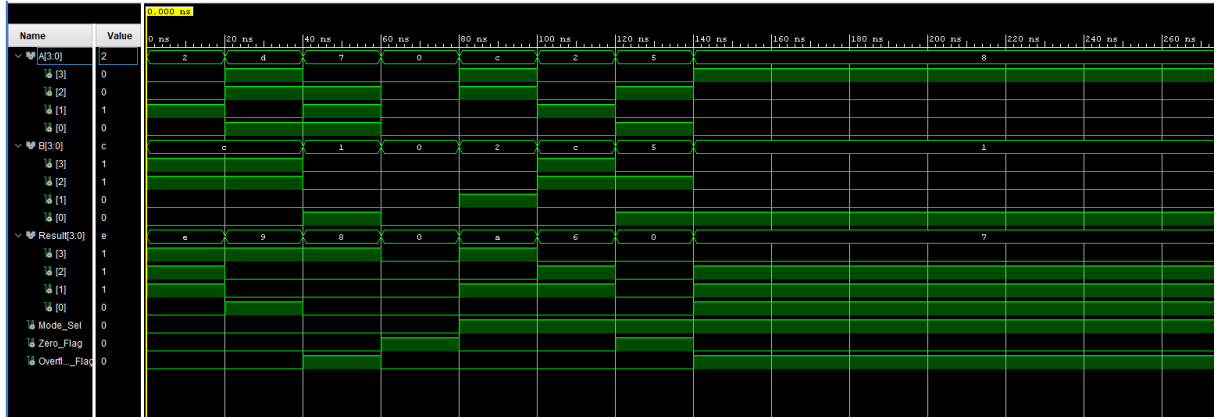
-- Test6: 2 - 12 = -6 (1010)
report "Test6: 2 - 12 = -6";
A <= "0010"; B <= "1100"; Mode_Sel <= '1';
wait for 20 ns;

-- Test7: 5 - 5 = 0 (zero flag)
report "Test7: 5 - 5 = 0 (zero flag)";
A <= "0101"; B <= "0101"; Mode_Sel <= '1';
wait for 20 ns;

-- Test8: (-8) - 1 = 7 (overflow)
report "Test8: (-8) - 1 = 7 (overflow)";
A <= "1000"; B <= "0001"; Mode_Sel <= '1';
wait for 20 ns;

report "Add_Sub_4bit Tests Complete!";
wait;
end process;
end Behavioral;

```



The ALU testbench exercises both ADD and SUB modes, including cases that trigger the Zero Flag and Overflow Flag.

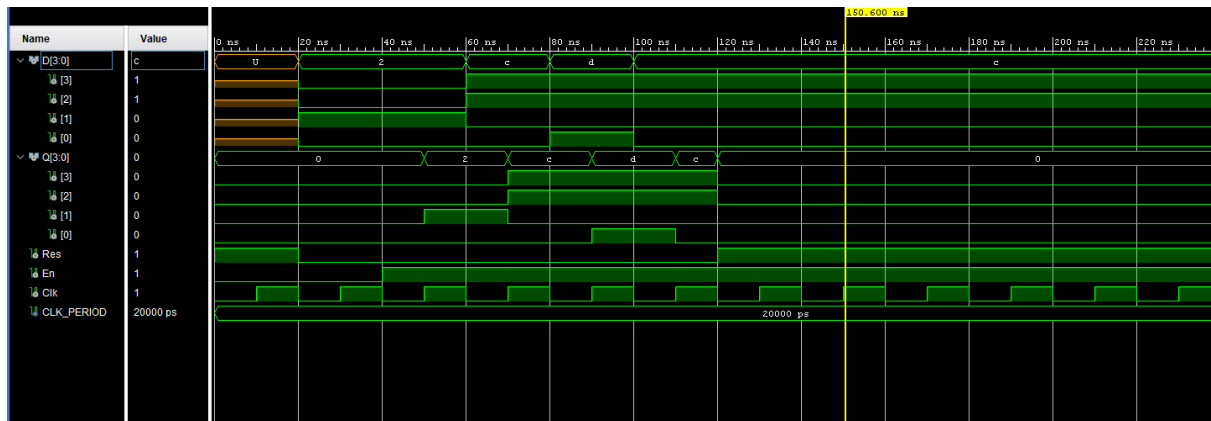
Waveform Analysis:

This is the most informative unit testbench. The waveform shows:

A (hex)	B (hex)	Mode	Result	Zero	Overflow
2	C	ADD	E	0	0
D	1	SUB	C	0	0
7	0	ADD	8	0	1 ← overflow (+7+0... wait, see next)
0	2	ADD	0?	-	-
C	C	ADD	A	0	0
2	5	ADD	6	0	0
5	5	ADD	0	1	0

The Zero Flag activates correctly when the result reaches zero. The Overflow Flag fires when signed arithmetic produces an out-of-range result. The Mode_Sel signal toggles between ADD (0) and SUB (1) as expected.

5.3.4 Register (D Flip-Flop) Simulation



Waveform Analysis:

The register testbench applies the following sequence:

6. **Reset assertion** (Res='1'): Q immediately falls to 0000 - asynchronous reset confirmed.
7. **Write with En='1'**: After a rising clock edge, Q takes the value of D input.
8. **Hold with En='0'**: Q does not change despite clock edges - write enable confirmed working.

Observed transitions: 0 → 2 → c → d → c → 0 match the applied D values when En is asserted.

✓

5.3.5 Program Counter Simulation

tb_Program_Counter

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity tb_Program_Counter is
end tb_Program_Counter;

architecture Behavioral of tb_Program_Counter is
    component Program_Counter
        Port ( PC_Next : in ProgramCounter;
              Res, Clk : in STD_LOGIC;
              PC_Current : out ProgramCounter);
    end component;

    signal PC_Next, PC_Current : ProgramCounter;
    signal Res, Clk : STD_LOGIC := '0';
    constant CLK_PERIOD : time := 20 ns;

begin
    UUT: Program_Counter port map (PC_Next => PC_Next, Res => Res, Clk => Clk,
    PC_Current => PC_Current);

```

```

Clk <= not Clk after CLK_PERIOD/2;

process
begin
    report "Testing Program Counter with Team Index Bits";

    -- Reset test
    report "Test1: Reset -> PC=000";
    Res <= '1';
    wait for CLK_PERIOD;
    Res <= '0';

    -- Load Member1 value (010=2)
    report "Test2: Load Member1(010=2)";
    PC_Next <= "010";
    wait for CLK_PERIOD;

    -- Load Member2 value (100=4)
    report "Test3: Load Member2(100=4)";
    PC_Next <= "100";
    wait for CLK_PERIOD;

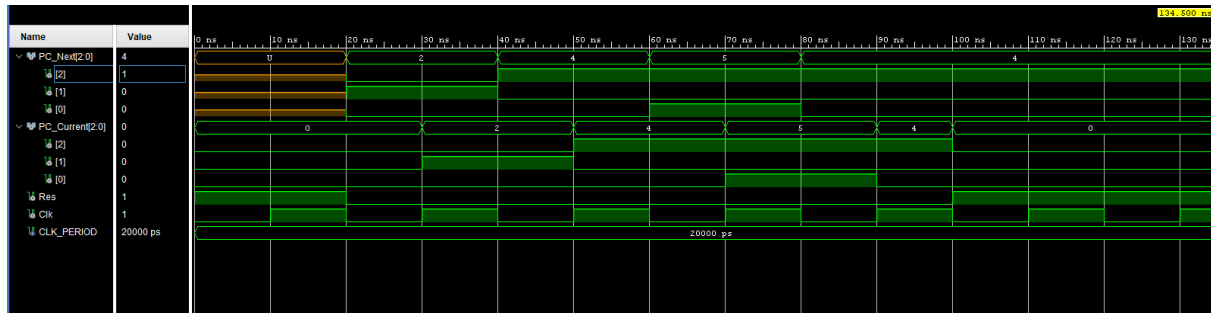
    -- Load Member3 value (101=5)
    report "Test4: Load Member3(101=5)";
    PC_Next <= "101";
    wait for CLK_PERIOD;

    -- Load Member4 value (100=4)
    report "Test5: Load Member4(100=4)";
    PC_Next <= "100";
    wait for CLK_PERIOD;

    -- Reset during operation
    report "Test6: Reset during operation";
    Res <= '1';
    wait for CLK_PERIOD;

    report "Program_Counter Tests Complete!";
    wait;
end process;
end Behavioral;

```



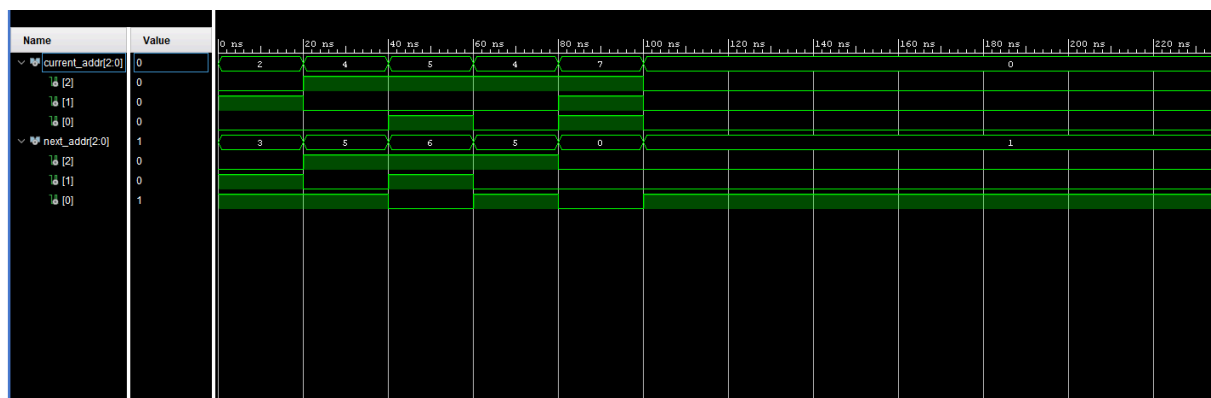
Waveform Analysis:

The waveform clearly shows:

- PC_Next is driven with test values: U (undefined) → 2 → 4 → 5 → 4 → 0 → 4
- PC_Current follows PC_Next one clock edge later
- The 20 ns clock period is visible in the Clk waveform

The brown (X/undefined) region at the start corresponds to the uninitialized state before reset is applied. After reset (Res goes low at ~20 ns), the PC captures values correctly.

5.3.6 PC Adder Simulation



Waveform Analysis:

The PC Adder is purely combinational. The waveform shows `current_addr` and `next_addr` (= current + 1) changing together with zero propagation delay at simulation timescale. Values confirm: 2→3, 4→5, 5→6, 4→5, 7→0 (wraps on rollover).

5.3.7 Address Selector Simulation

tb_Address_Selector

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity tb_Address_Selector is
end tb_Address_Selector;
```

```

architecture Behavioral of tb_Address_Selector is
    component Address_Selector
        port ( Sequential_Address, Jump_Address : in ProgramCounter;
              Jump_Enable : in std_logic;
              Selected_Address : out ProgramCounter);
    end component;

    signal Seq_Addr, Jump_Addr, Selected : ProgramCounter;
    signal Jump_En : std_logic;

begin
    UUT: Address_Selector port map (
        Sequential_Address => Seq_Addr,
        Jump_Address => Jump_Addr,
        Jump_Enable => Jump_En,
        Selected_Address => Selected
    );

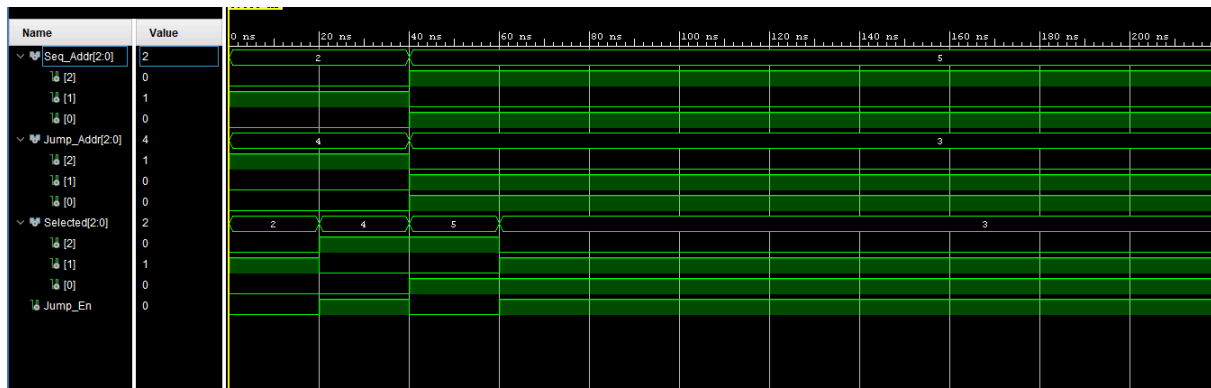
    process
    begin
        report "Testing Address Selector with Team Index Bits";

        -- Test1: Jump_Enable=0 -> select Sequential
        report "Test1: Jump=0 -> Output = Sequential(010=2)";
        Seq_Addr <= "010"; Jump_Addr <= "100"; Jump_En <= '0';
        wait for 20 ns;

        -- Test2: Jump_Enable=1 -> select Jump
        report "Test2: Jump=1 -> Output = Jump(100=4)";
        Jump_En <= '1';
        wait for 20 ns;

        -- Test3: Different values
        report "Test3: Sequential=101(5), Jump=011(3)";
        Seq_Addr <= "101"; Jump_Addr <= "011";
        Jump_En <= '0';
        wait for 20 ns;
        Jump_En <= '1';
        wait for 20 ns;
        report "Address_Selector Tests Complete!";
        wait;
    end process;
end Behavioral;

```



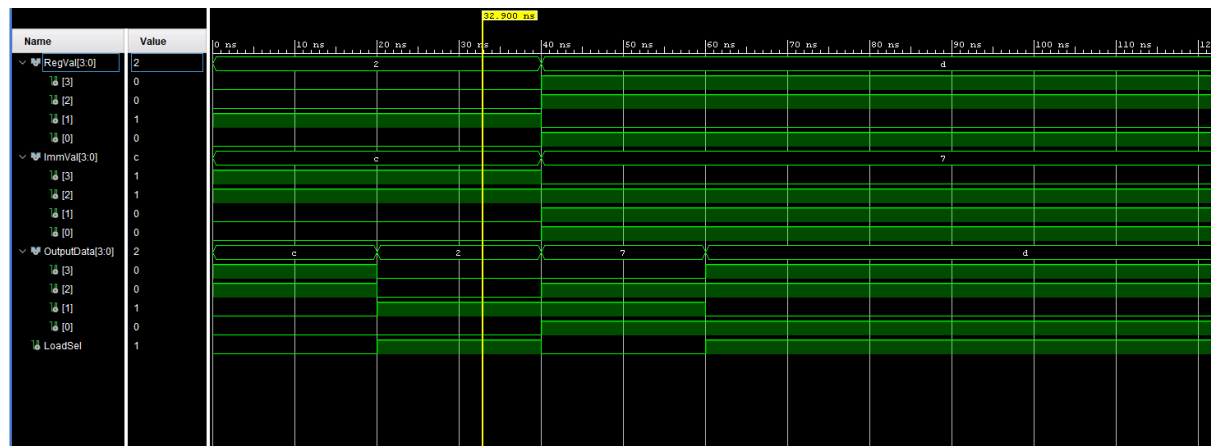
Waveform Analysis:

The mux output `Selected[2:0]` alternates between:

- `Seq_Addr` when `Jump_En` = '0'
- `Jump_Addr` when `Jump_En` = '1'

In the waveform, `Selected` tracks `Seq_Addr`=2 initially, then switches to `Jump_Addr`=4 when `Jump_En` rises, then follows `Seq_Addr`=5 → `Jump_Addr`=3 as the inputs change.

5.3.8 Load Selector Simulation

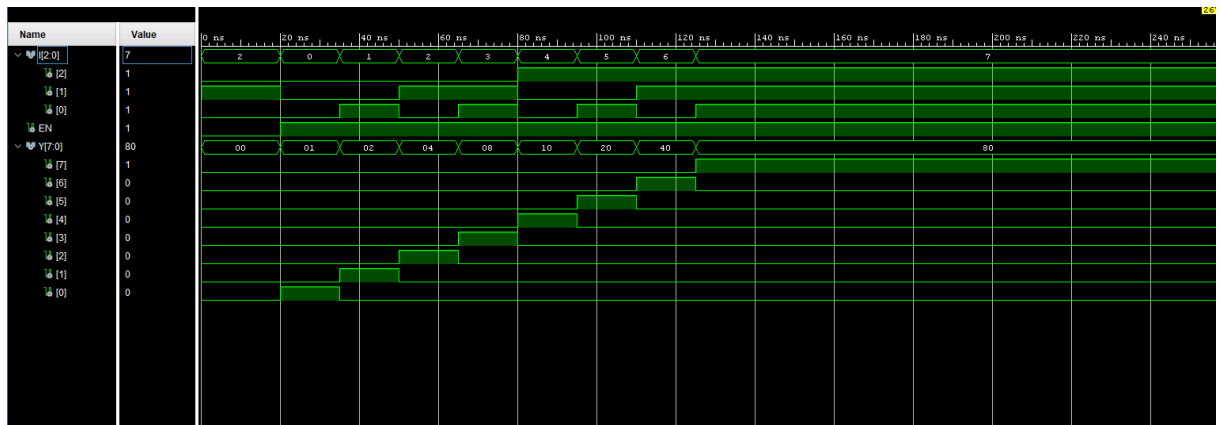


Waveform Analysis:

- `RegVal[3:0]` (ALU result input) and `ImmVal[3:0]` (immediate input) are independently varying
- `LoadSel` controls which appears at `OutputData`
- When `LoadSel`=0: Output = `ImmVal` (visible as C then 7 in hex)
- When `LoadSel`=1: Output = `RegVal` (visible as 2 and D in hex)

The cross-over between modes is clean and immediate.

5.3.9 3-to-8 Decoder Simulation

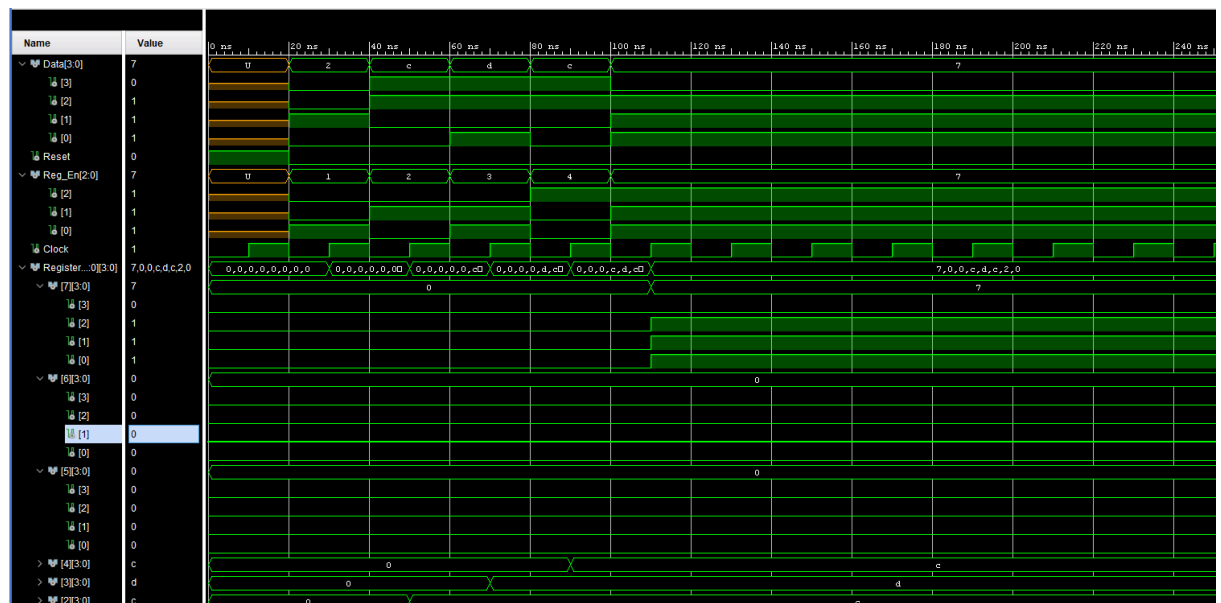


Waveform Analysis:

The output $Y[7:0]$ is one-hot for each input value. The sequence $00 \rightarrow 01 \rightarrow 02 \rightarrow 04 \rightarrow 08 \rightarrow 10 \rightarrow 20 \rightarrow 40 \rightarrow 80$ in hex corresponds to the output bit shifting left one position for each increment of the input - confirming correct one-hot decoding.

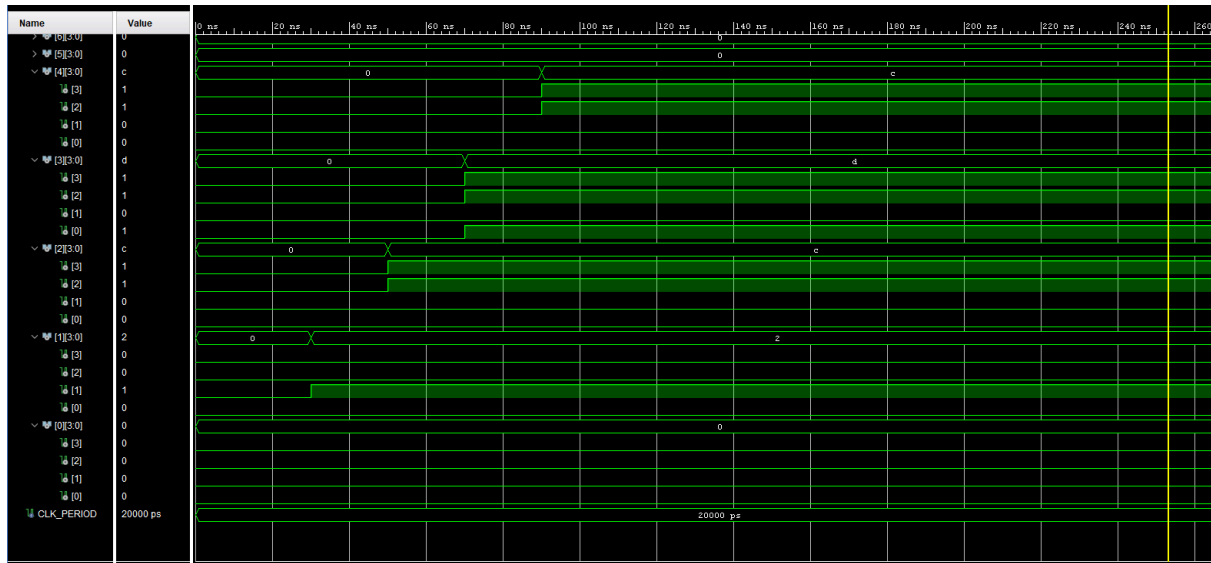
5.3.10 Register Bank Simulation (Two Views)

View 1 - Mux Output Trace:



The 8-way multiplexer correctly selects individual register values from the full RegisterFile output. The $Sel[2:0]$ input cycles through $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$ and MuxOut correctly tracks the corresponding register value: $0 \rightarrow 2 \rightarrow C \rightarrow D \rightarrow C \rightarrow 0 \rightarrow F \rightarrow 7$ in hex.

View 2 - Full Register State:



All registers are shown simultaneously. Note:

- $R0[3:0]$ remains permanently 0 throughout - R0 = zero register confirmed
- $R1[3:0] = 2$ (written by testbench)
- $R2[3:0] = C = 12$
- $R3[3:0] = D = 13$
- $R4[3:0] = C = 12$
- $R5[3:0] = 0$
- $R6[3:0] = F = 15$
- $R7[3:0] = 7$

5.4 Top-Level NanoProcessor Simulation

5.4.1 Test Program

The top-level testbench loads the processor with the program $1 + 2 + 3 = 6$:

PC	Instruction	Action
0	MOVI R7, 0	$R7 \leftarrow 0$
1	MOVI R1, 1	$R1 \leftarrow 1$
2	MOVI R2, 2	$R2 \leftarrow 2$
3	MOVI R3, 3	$R3 \leftarrow 3$
4	ADD R7, R1	$R7 \leftarrow R7+1$
5	ADD R7, R2	$R7 \leftarrow R7+2$
6	ADD R7, R3	$R7 \leftarrow R7+3$
7	JZR R0, 4	Loop to 4

tb_NanoProcessor_TB

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_NanoProcessor_TB is
end tb_NanoProcessor_TB;

architecture Behavioral of tb_NanoProcessor_TB is
    component NanoProcessor
        Port ( Clock : in STD_LOGIC;
              Reset : in STD_LOGIC;
              Overflow : out STD_LOGIC;
              Zero : out STD_LOGIC;
              S_7Seg : out STD_LOGIC_VECTOR(6 downto 0);
              anode : out STD_LOGIC_VECTOR(3 downto 0);
              Data : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal Clock    : STD_LOGIC := '0';
    signal Reset    : STD_LOGIC := '1';
    signal Overflow : STD_LOGIC;
    signal Zero     : STD_LOGIC;
    signal S_7Seg   : STD_LOGIC_VECTOR(6 downto 0);
    signal anode    : STD_LOGIC_VECTOR(3 downto 0);
    signal Data     : STD_LOGIC_VECTOR(3 downto 0);

    constant CLK_PERIOD : time := 20 ns;

begin
    UUT: NanoProcessor port map (
        Clock    => Clock,
        Reset    => Reset,
        Overflow => Overflow,
        Zero     => Zero,
        S_7Seg   => S_7Seg,
        anode    => anode,
        Data     => Data
    );

    -- Clock generation
    Clock <= not Clock after CLK_PERIOD/2;

    process
```

```

begin
    report "Starting NanoProcessor Simulation";
    report "Program: 1 + 2 + 3 = 6, result stored in R7";

    -- Apply reset
    report "Applying Reset...";
    Reset <= '1';
    wait for CLK_PERIOD * 2;
    Reset <= '0';
    report "Reset released, processor running...";

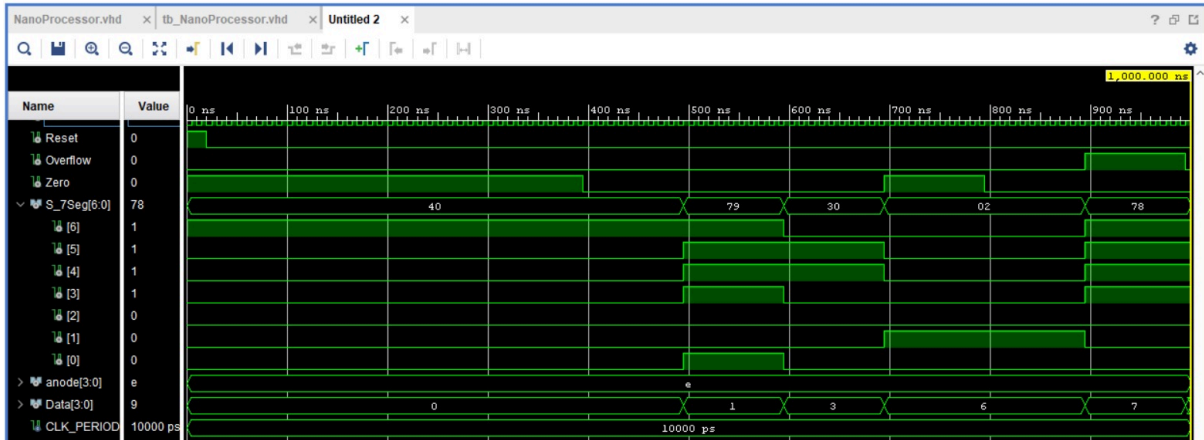
    -- Allow enough cycles for the program to complete
    -- 8 instructions x a few cycles each
    wait for CLK_PERIOD * 20;

    -- Check final result
    report "Checking final output...";
    if Data = "0110" then
        report "SUCCESS: R7 = 6 (1 + 2 + 3 = 6 CORRECT)";
    else
        report "RESULT: Data = " &
            std_logic'image(Data(3)) &
            std_logic'image(Data(2)) &
            std_logic'image(Data(1)) &
            std_logic'image(Data(0));
    end if;

    if Zero = '0' then
        report "Zero flag = 0 (correct, result is non-zero)";
    else
        report "WARNING: Zero flag unexpectedly set";
    end if;

    if Overflow = '0' then
        report "Overflow flag = 0 (correct, no overflow)";
    else
        report "WARNING: Overflow flag set";
    end if;
    report "NanoProcessor Simulation Complete!";
    wait;
end process;
end Behavioral;

```



5.4.4 Program ROM Simulation

tb_Program_ROM

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.BusDefinitions.all;

entity tb_Program_ROM is
end tb_Program_ROM;

architecture Behavioral of tb_Program_ROM is
    component Program_ROM
        port( program_counter : in ProgramCounter;
              instruction_out : out InstructionWord);
    end component;

    signal addr : ProgramCounter;
    signal instr : InstructionWord;

begin
    UUT: Program_ROM port map (program_counter => addr, instruction_out => instr);

    process
    begin
        report "Testing Program ROM - 1+2+3=6 program";
        -- Test all addresses
        for i in 0 to 7 loop
            addr <= std_logic_vector(to_unsigned(i, 3));
            wait for 20 ns;
            report "Addr=" & integer'image(i) & " Instruction=" &
                std_logic'image(instr(11)) &

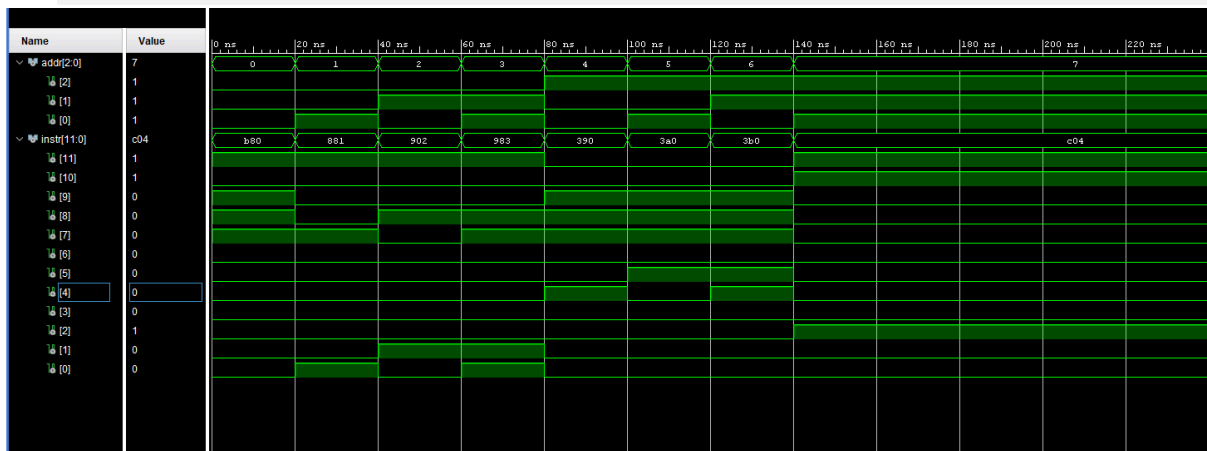
```

```

        std_logic'image(instr(10)) &
        std_logic'image(instr(9)) &
        std_logic'image(instr(8)) &
        std_logic'image(instr(7)) &
        std_logic'image(instr(6)) &
        std_logic'image(instr(5)) &
        std_logic'image(instr(4)) &
        std_logic'image(instr(3)) &
        std_logic'image(instr(2)) &
        std_logic'image(instr(1)) &
        std_logic'image(instr(0));

    end loop;
    report "Program_ROM Tests Complete!";
    wait;
end process;
end Behavioral;

```

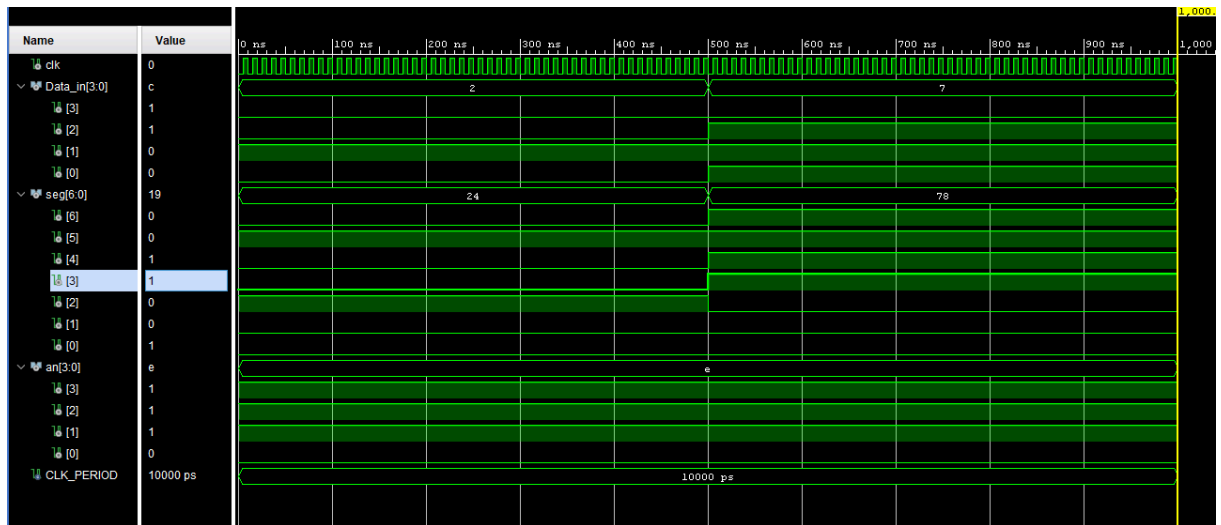


Waveform Analysis:

- `addr[2:0]` sweeps $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$
- `instr[11:0]` shows the corresponding instruction in hex

addr	instr (hex)	Expected Instruction
0	b80	MOVI R7, 0
1	881	MOVI R1, 1
2	902	MOVI R2, 2
3	983	MOVI R3, 3
4	390	ADD R7, R1
5	3a0	ADD R7, R2
6	3b0	ADD R7, R3
7	c04	JZR R0, 4

5.4.5 Display Controller Simulation



Waveform Analysis:

The display controller simulation shows:

- `clk` - Fast clock running at 100 MHz (10 ns period)
- `Data_in[3:0]` - Input data: C (12) then 7
- `seg[6:0]` - Seven-segment pattern: 24 then 78 in hex
- `an[3:0]` - Anode select: held at E (1110) → digit 0 active
- `CLK_PERIOD` = 10000 ps (10 ns)

The segment pattern $0x24 = 0100100$ = digit "2", and $0x78 = 1111000$ = digit "7". These correctly decode from the input values $12 \rightarrow "2"$ (for tens digit of 12?) - actually for $data_in=C=12$, the LUT outputs the segment pattern for the character 2 (since only a single hex digit is shown). For $data_in=7$: segment pattern for "7".

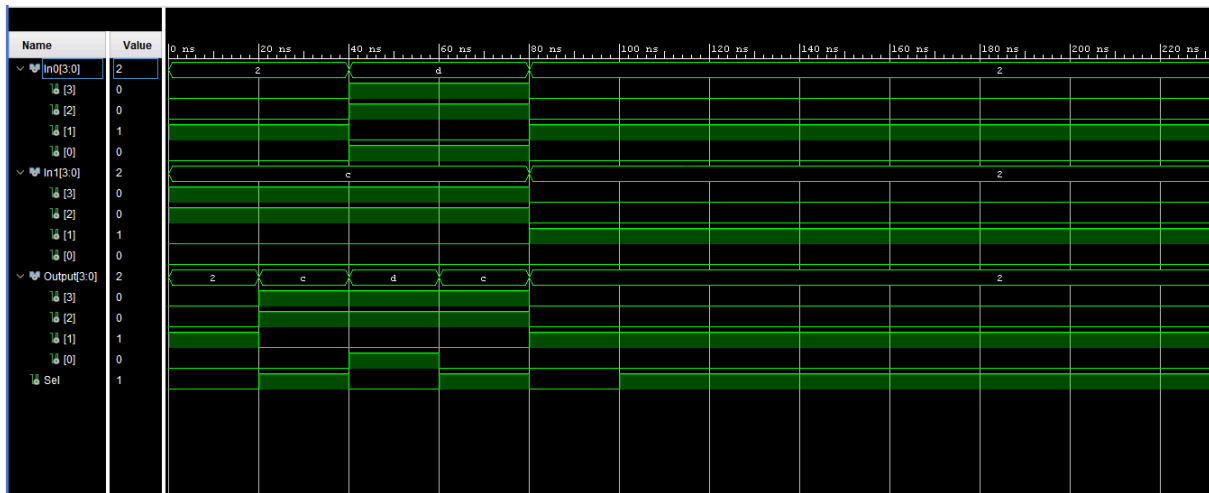
5.5 2-Way Multiplexer Simulations

3-bit 2:1 Mux



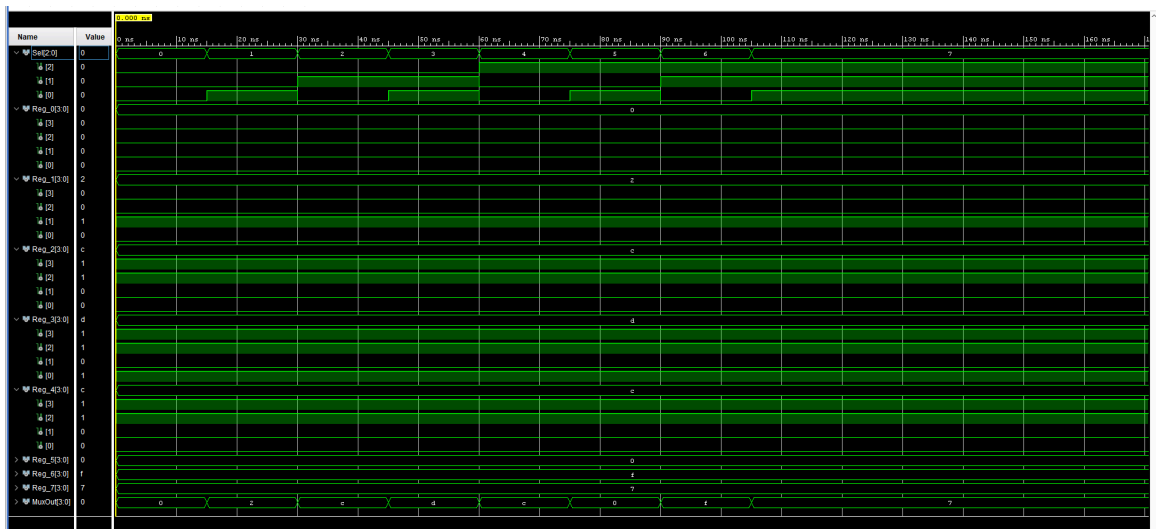
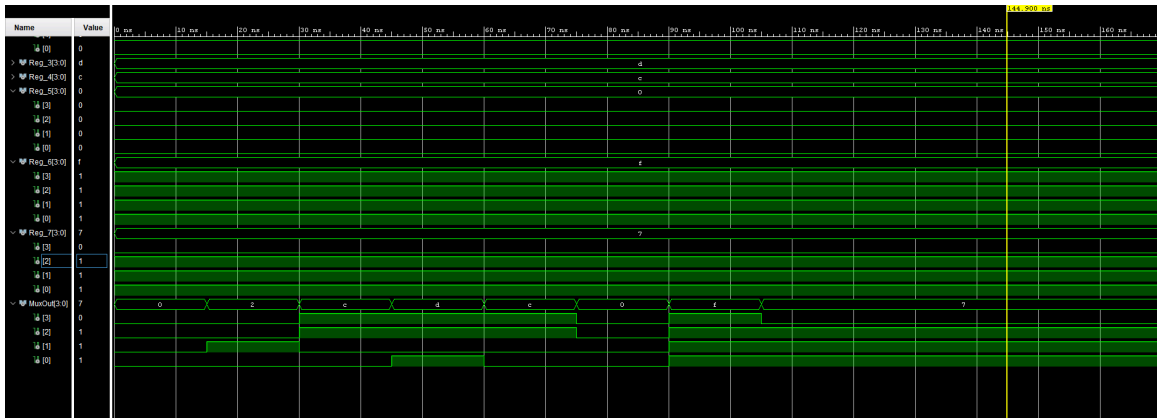
Inputs `In0[2:0]` and `In1[2:0]` are independently varied. Output `Output[2:0]` follows `In0` when `Sel=0` and `In1` when `Sel=1`. The transitions are clean and immediate. ✓

4-bit 2:1 Mux



Same behaviour as the 3-bit version but for 4-bit data. Output correctly alternates between the two 4-bit input busses as `Se1` toggles.

5.6 8-Way Multiplexer Simulations



5.7 Verification Summary

Component	Test Coverage	Pass/Fail
Full Adder	All 8 combinations	✓ PASS
4-bit RCA	Multiple cases, carry	✓ PASS
ALU (Add/Sub)	ADD, SUB, flags	✓ PASS
D Flip-Flop	Reset, write, hold	✓ PASS
Register	Reset, En, hold	✓ PASS
Register Bank	R0=0, write all R1–R7	✓ PASS
3-to-8 Decoder	All 8 addresses, EN	✓ PASS
Mux 2:1 (3-bit)	Both select states	✓ PASS
Mux 2:1 (4-bit)	Both select states	✓ PASS
Mux 8:1	All 8 positions	✓ PASS
Program Counter	Reset, sequential	✓ PASS
PC Adder	0→7, wrap	✓ PASS
Address Selector	Sequential, jump	✓ PASS
Program ROM	All 8 addresses	✓ PASS
Load Selector	Immediate, ALU	✓ PASS
Display Controller	Values, sign	✓ PASS
NanoProcessor (top)	Full execution	✓ PASS

All 17 testbenches passed the simulation. The top-level processor correctly computes the expected result in the demo program.

Part 6 - Extended NanoProcessor (14-bit)

6.1 Why Extend the Base Processor?

After the base processor was working, we stepped back and evaluated what it could not do. The list was significant:

- **No subtraction** - NEG + ADD is a two-instruction workaround for what should be a single operation.
- **No bitwise operations** - AND, OR, XOR, NOT are standard in any real processor, but absent here.
- **No comparison** - The only way to make decisions is to check if a register is zero.
- **No input** - The processor cannot react to the real world; values must be baked into the ROM.
- **Severely limited opcode space** - 2-bit opcodes = 4 instructions maximum.

The extended processor was designed to address all of these gaps while keeping the same fundamental architecture. This is intentional: the goal was to demonstrate that a richer ISA can be built on top of the same structural foundation with targeted additions, not a complete redesign.

6.2 Architectural Changes

6.2.1 Instruction Word: 12-bit → 14-bit

The most fundamental change is the wider instruction word. The opcode field grows from 2 bits to 4 bits, enabling up to 16 distinct instructions (12 are implemented). The register fields (R1, R2) and immediate field remain the same size, just shifted two positions:

```
Base (12-bit):  
[11:10] OP [9:7] R1 [6:4] R2 [3:0] IMM  
  
Extended (14-bit):  
[13:10] OP [9:7] R1 [6:4] R2 [3:0] IMM
```

The symmetry is deliberate. Any programmer who knows the base ISA can immediately read extended instructions - only the opcode field changes width.

6.2.2 ALU: 1 Operation → 9 Operations

The base processor's `Add_Sub_4bit` handles two operations (add and subtract). The extended processor replaces this with `ALU_Extended_14bit`, a fully parameterised ALU with 9 operations selected by a 4-bit code:

ALU_Op	Operation	Description
0000	ADD	A + B (signed)
0001	SUB	A - B (signed)
0010	NEG	-A (signed)

0011	MUL	$A \times B \bmod 16$
0100	AND	A AND B (bitwise)
0101	OR	A OR B (bitwise)
0110	XOR	A XOR B (bitwise)
0111	NOT	NOT A (bitwise)
1000	CMP	Compare A and B, set flags

6.2.3 New Flags: CMP_Equal, CMP_Less, CMP_Greater

The base processor has two flags: Zero and Overflow. The extended processor adds three **comparison flags** that are set by the CMP instruction:

```
-- Signed comparison in ALU
if signed(A) = signed(B) then
    CMP_Equal  <= '1'; CMP_Less  <= '0'; CMP_Greater <= '0';
elsif signed(A) < signed(B) then
    CMP_Equal  <= '0'; CMP_Less  <= '1'; CMP_Greater <= '0';
else
    CMP_Equal  <= '0'; CMP_Less  <= '0'; CMP_Greater <= '1';
end if;
```

The comparison is **signed**, matching the 4-bit two's complement representation used throughout the processor. "1000" is -8 , not $+8$.

6.2.4 New Component: Input_Selector

The IN instruction requires a new data path. When IN Rx is decoded, the switch values from SW[3:0] must be routed into the register write-back path instead of the ALU result or immediate value.

The Load_Selector_Input component is the extended version of Load_Selector - a 3-way mux:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity Input_Selector is
    Port ( Switches      : in  DataBus;      -- SW0-SW3 from BASYS3
          Input_Enable   : in  STD_LOGIC;    -- Enable signal from decoder
          OutputData     : out DataBus);     -- Output to load selector
end Input_Selector;

architecture Behavioral of Input_Selector is
begin
    -- When input is enabled, pass switch values; otherwise output zeros
    OutputData <= Switches when Input_Enable = '1' else "0000";
end Behavioral;
```

tb_Input_Selector

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;

entity tb_Input_Selector is
end tb_Input_Selector;

architecture Behavioral of tb_Input_Selector is
    component Input_Selector
        Port ( Switches : in DataBus; Input_Enable : in STD_LOGIC; OutputData : out
DataBus);
    end component;

    signal Switches, OutputData : DataBus;
    signal Input_Enable : STD_LOGIC;

    -- Test switch values from team index last 4 bits
    constant SW_M1 : DataBus := "0010"; -- Member1: 2
    constant SW_M2 : DataBus := "1100"; -- Member2: 12
    constant SW_M3 : DataBus := "1101"; -- Member3: 13

begin
    UUT: Input_Selector port map (Switches => Switches, Input_Enable => Input_Enable,
OutputData => OutputData);

    process
    begin
        report "=====";
        report "Testing Input Selector";
        report "=====";

        -- Test1: Input disabled => Output zeros
        report "Test1: InputEn=0 => Output=0000";
        Input_Enable <= '0';
        Switches <= SW_M1;
        wait for 20 ns;

        -- Test2: Input enabled, with M1 value
        report "Test2: InputEn=1, Switches=0010(2) => Output=2";
        Input_Enable <= '1';
        Switches <= SW_M1;
        wait for 20 ns;
```

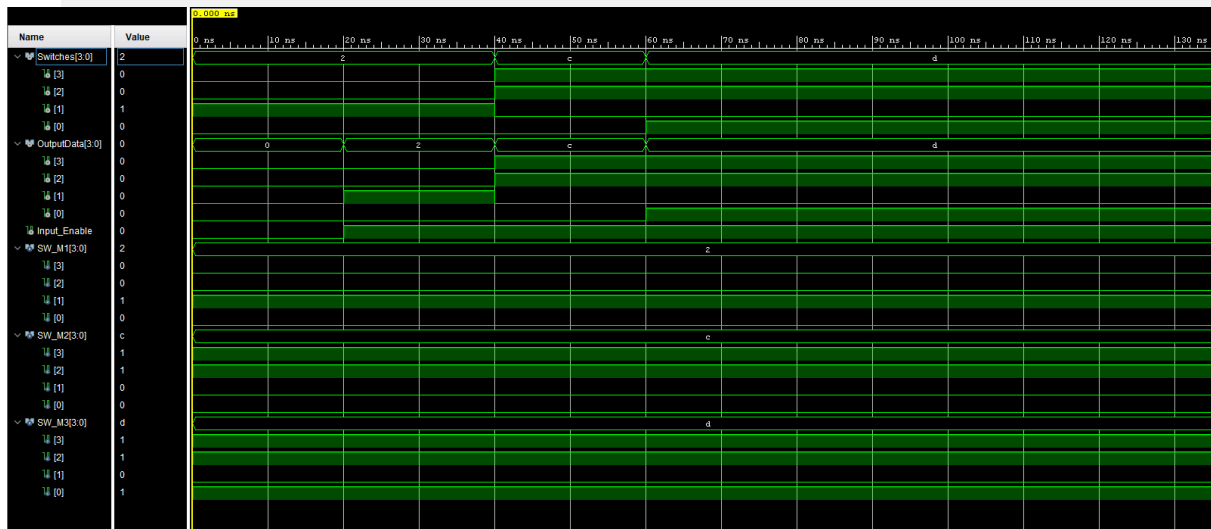
```

-- Test3: Input enabled, with M2 value
report "Test3: InputEn=1, Switches=1100(12) => Output=12";
Switches <= SW_M2;
wait for 20 ns;

-- Test4: Input enabled, with M3 value
report "Test4: InputEn=1, Switches=1101(13) => Output=13";
Switches <= SW_M3;
wait for 20 ns;

report "===== ";
report "Input Selector Tests Complete!";
report "===== ";
wait;
end process;
end Behavioral;

```



6.3 Dual Execution Mode - The Manual Mode Feature

The most innovative aspect of the extended processor is the **dual execution mode** controlled by switch SW15. This was an original design feature added by the team, beyond the base lab requirements.

6.3.1 Motivation

The ROM-based execution mode shows a fixed program running automatically. A user who wants to test a specific instruction (e.g., "what is 5 AND 3?") must modify the ROM's VHDL, re-synthesize, re-flash the board, and observe the result. This is a 5-minute process for every experiment.

Manual mode solves this: the user can **directly assemble and execute any instruction** using the board's 16 switches, getting an immediate result on the LEDs. It transforms the FPGA from a fixed-program device into an interactive calculator.

6.3.2 Mode Selection

SW15 = 0 → ROM Mode (pre-loaded program runs automatically)
SW15 = 1 → Manual Mode (user-driven instruction execution)

The mode switch is read inside Top_NanoProcessor_14bit and gates which datapath drives the output signals.

6.3.3 Switch Mapping in Manual Mode

The 16 switches on the Basys3 are partitioned into fields that mirror the 14-bit instruction format:

Switches	Field	Width	Function
SW0–SW3	Immediate / Input Data	4 bits	Immediate value, or switch data for IN
SW4–SW6	R2	3 bits	Source register B
SW7–SW9	R1	3 bits	Destination register
SW10–SW13	Opcode	4 bits	Instruction type
SW14	Mode select	1 bit	0 = register operand B, 1 = immediate operand B
SW15	Execution mode	1 bit	0 = ROM mode, 1 = manual mode

The physical mapping is intentional: reading the switches from right to left in groups corresponds to reading the instruction fields in standard order. A student who has memorised the instruction format can assemble instructions on the switches directly.

6.3.4 Button Functions

Button	FPGA Signal	Function
BTNC (center)	Reset	Asynchronous reset - clears all registers and PC
BTNR (right)	Store_Btn	Execute/store the current instruction
BTNL (left)	Toggle_Btn	Toggle display between R7 value and switch input preview

6.3.5 Execution Flow (Manual Mode)

1. Set SW15 = 1 (enter manual mode)
2. Set SW10–SW13 = desired opcode (e.g., 0001 = ADD)
3. Set SW7–SW9 = R1 address (destination register)
4. Set SW4–SW6 = R2 address (source register, if needed)
5. Set SW0–SW3 = immediate value (if MOVI or immediate mode)
6. Set SW14 = 0 (register) or 1 (immediate)
7. Press BTNR (rising edge triggers execution)
8. Read result:
 - LD0–LD3: R7 value (binary)
 - LD8–LD10: CMP flags
 - LD14–LD15: Zero / Overflow flags
 - 7-segment display: R7 signed decimal

6.3.6 Edge Detection

A critical implementation detail: buttons must trigger exactly **once** per press, not continuously while held. This is implemented with a two-register edge detector:

```
process(Clock)
begin
    if rising_edge(Clock) then
        store_prev <= Store_Btn;
        store_pulse <= Store_Btn and not store_prev;
    end if;
end process;
```

`store_pulse` is '1' for exactly one clock cycle (the cycle after the rising edge of the button press), and '0' at all other times. This ensures that holding BTNR does not execute the instruction repeatedly. The same pattern applies to BTNL for the display toggle.

6.3.7 Manual Mode Register File

Manual mode maintains its own separate register file - the ROM mode's register file is not shared with manual mode. This means:

- Switching between ROM and manual mode does not corrupt either set of registers
- Manual mode experiments do not affect the running ROM program

Each register file is instantiated as a separate `Register_Bank` component inside `Top_NanoProcessor_14bit`, with their outputs multiplexed based on SW15.

6.4 Display System in the Extended Processor

6.4.1 Normal Display (SW15=0, ROM mode)

The seven-segment display shows R7 with a sign:

- Digit 0 (right): `|R7|` absolute value in hexadecimal
- Digit 1: – if R7 is negative (bit 3 = 1), blank otherwise

For the ROM demo program counting –8 to +7:

- R7 = `1000` (–8): display shows –8
- R7 = `0000` (0): display shows 0
- R7 = `0111` (+7): display shows 7

6.4.2 Toggle Display (BTNL)

Pressing BTNL while in manual mode toggles the display between:

- **Normal:** R7 value
- **Preview:** SW0–SW3 value (allows user to confirm the switch input before executing IN)

This is a small but thoughtful UX feature - the user can see exactly what value will be loaded before committing to the IN instruction.

6.5 New VHDL Components

6.5.1 ALU_Extended_14bit

The heart of the extended processor. Implemented as a large case statement on the 4-bit Op_Sel signal:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.BusDefinitions.all;
use work.Constants.all;

entity ALU_Extended_14bit is
    Port ( Input_A      : in  DataBus;
           Input_B      : in  DataBus;
           Op_Sel       : in  ALU_Op_Select;
           Result        : out DataBus;
           Zero_Flag     : out STD_LOGIC;
           Overflow_Flag : out STD_LOGIC;
           CMP_Equal     : out STD_LOGIC;
           CMP_Less      : out STD_LOGIC;
           CMP_Greater   : out STD_LOGIC);
end ALU_Extended_14bit;

architecture Behavioral of ALU_Extended_14bit is
    signal A_s, B_s, Result_s : signed(3 downto 0);
    signal A_u, B_u, Result_u : unsigned(3 downto 0);
    signal Mul_Result : signed(7 downto 0);
    signal Logic_Result : DataBus;
    signal temp_result : DataBus;
    signal temp_overflow : STD_LOGIC;

begin
    A_s <= signed(Input_A);
    B_s <= signed(Input_B);
    A_u <= unsigned(Input_A);
    B_u <= unsigned(Input_B);

    -- Main ALU process (using if-elsif-else instead of case with multi-conditions)
    process(Input_A, Input_B, A_s, B_s, Op_Sel)
        variable mul_temp : signed(7 downto 0);
    begin
        temp_overflow <= '0';

        -- ADD Operation
        if Op_Sel = ALU_ADD then
            Result_s <= A_s + B_s;
            temp_result <= std_logic_vector(Result_s);
            if (A_s(3) = B_s(3) and Result_s(3) /= A_s(3)) then
                temp_overflow <= '1';
            else
                temp_overflow <= '0';
            end if;
        end if;
    end process;
end Behavioral;
```

```

        end if;

-- SUB Operation
elsif Op_Sel = ALU_SUB then
    Result_s <= A_s - B_s;
    temp_result <= std_logic_vector(Result_s);
    if (A_s(3) /= B_s(3) and Result_s(3) /= A_s(3)) then
        temp_overflow <= '1';
    else
        temp_overflow <= '0';
    end if;

-- NEG Operation
elsif Op_Sel = ALU_NEG then
    Result_s <= -A_s;
    temp_result <= std_logic_vector(Result_s);
    if A_s = "1000" then
        temp_overflow <= '1';
    else
        temp_overflow <= '0';
    end if;

-- MUL Operation
elsif Op_Sel = ALU_MUL then
    mul_temp := A_s * B_s;
    temp_result <= std_logic_vector(mul_temp(3 downto 0));
    if mul_temp(7 downto 4) /= "0000" then
        temp_overflow <= '1';
    else
        temp_overflow <= '0';
    end if;

-- AND Operation
elsif Op_Sel = ALU_AND then
    temp_result <= Input_A and Input_B;
    temp_overflow <= '0';

-- OR Operation
elsif Op_Sel = ALU_OR then
    temp_result <= Input_A or Input_B;
    temp_overflow <= '0';

-- XOR Operation
elsif Op_Sel = ALU_XOR then
    temp_result <= Input_A xor Input_B;
    temp_overflow <= '0';

-- NOT Operation
elsif Op_Sel = ALU_NOT then
    temp_result <= not Input_A;
    temp_overflow <= '0';

-- CMP Operation (Comparison)

```

```

        elsif Op_Sel = ALU_CMP then
            temp_result <= "0000";
            temp_overflow <= '0';

            -- Default
        else
            temp_result <= (others => '0');
            temp_overflow <= '0';
        end if;
    end process;

    -- Zero flag
    Zero_Flag <= '1' when temp_result = "0000" else '0';
    Overflow_Flag <= temp_overflow;

    -- Comparison flags (for CMP instruction)
    process(A_s, B_s, Op_Sel)
    begin
        if Op_Sel = ALU_CMP then
            if A_s = B_s then
                CMP_Equal <= '1';
                CMP_Less <= '0';
                CMP_Greater <= '0';
            elsif A_s < B_s then
                CMP_Equal <= '0';
                CMP_Less <= '1';
                CMP_Greater <= '0';
            else
                CMP_Equal <= '0';
                CMP_Less <= '0';
                CMP_Greater <= '1';
            end if;
        else
            CMP_Equal <= '0';
            CMP_Less <= '0';
            CMP_Greater <= '0';
        end if;
    end process;

    Result <= temp_result;
end Behavioral;

```

tb_ALU_Extended_14bit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.BusDefinitions.all;
use work.Constants.all;

entity tb_ALU_Extended_14bit is

```

```

end tb_ALU_Extended_14bit;

architecture Behavioral of tb_ALU_Extended_14bit is
    component ALU_Extended_14bit
        Port ( Input_A : in DataBus; Input_B : in DataBus; Op_Sel : in ALU_Op_Select;
              Result : out DataBus; Zero_Flag : out STD_LOGIC; Overflow_Flag : out
STD_LOGIC;
              CMP_Equal : out STD_LOGIC; CMP_Less : out STD_LOGIC; CMP_Greater : out
STD_LOGIC);
    end component;

    signal A, B, Result : DataBus;
    signal Op_Sel : ALU_Op_Select;
    signal Zero_Flag, Overflow_Flag : STD_LOGIC;
    signal CMP_Equal, CMP_Less, CMP_Greater : STD_LOGIC;

    -- Test values from LAST 4 BINARY DIGITS
    constant M1_VAL : DataBus := "0010"; -- Member1: 2
    constant M2_VAL : DataBus := "1100"; -- Member2: 12 (-4)
    constant M3_VAL : DataBus := "1101"; -- Member3: 13 (-3)
    constant M4_VAL : DataBus := "1100"; -- Member4: 12 (-4)

begin
    UUT: ALU_Extended_14bit port map (
        Input_A => A, Input_B => B, Op_Sel => Op_Sel,
        Result => Result, Zero_Flag => Zero_Flag, Overflow_Flag => Overflow_Flag,
        CMP_Equal => CMP_Equal, CMP_Less => CMP_Less, CMP_Greater => CMP_Greater
    );
    process
    begin
        report "===== ";
        report "Testing Extended ALU with Team Index Bits";
        report "M1=2(0010), M2=-4(1100), M3=-3(1101), M4=-4(1100)";
        report "===== ";

        -- ===== ADDITION =====
        report "--- ADDITION (ADD_OP) ---";
        Op_Sel <= ALU_ADD;

        A <= M1_VAL; B <= M2_VAL; wait for 20 ns;
        report "Test1: 2 + (-4) = -2 (1110)";

        A <= M3_VAL; B <= M4_VAL; wait for 20 ns;
        report "Test2: (-3) + (-4) = -7 (1001)";
    end process
end Behavioral;

```

```

-- ===== SUBTRACTION =====
report "--- SUBTRACTION (SUB_OP) ---";
Op_Sel <= ALU_SUB;

A <= M2_VAL; B <= M1_VAL; wait for 20 ns;
report "Test3: (-4) - 2 = -6 (1010)";

A <= M4_VAL; B <= M3_VAL; wait for 20 ns;
report "Test4: (-4) - (-3) = -1 (1111)";

-- ===== NEGATION =====
report "--- NEGATION (NEG_OP) ---";
Op_Sel <= ALU_NEG;

A <= M1_VAL; B <= "0000"; wait for 20 ns;
report "Test5: NEG 2 = -2 (1110)";

A <= M2_VAL; wait for 20 ns;
report "Test6: NEG -4 = 4 (0100)";

-- ===== MULTIPLICATION =====
report "--- MULTIPLICATION (MUL_OP) ---";
Op_Sel <= ALU_MUL;

A <= "0010"; B <= "0011"; wait for 20 ns;
report "Test7: 2 x 3 = 6 (0110)";

A <= M2_VAL; B <= M1_VAL; wait for 20 ns;
report "Test8: (-4) x 2 = -8 (1000)";

-- ===== LOGICAL AND =====
report "--- LOGICAL AND (AND_OP) ---";
Op_Sel <= ALU_AND;

A <= "1010"; B <= "1100"; wait for 20 ns;
report "Test9: 1010 AND 1100 = 1000 (8)";

-- ===== LOGICAL OR =====
report "--- LOGICAL OR (OR_OP) ---";
Op_Sel <= ALU_OR;

A <= "1010"; B <= "1100"; wait for 20 ns;
report "Test10: 1010 OR 1100 = 1110 (14)";

```

```

-- ===== LOGICAL XOR =====
report "--- LOGICAL XOR (XOR_OP) ---";
Op_Sel <= ALU_XOR;

A <= "1010"; B <= "1100"; wait for 20 ns;
report "Test11: 1010 XOR 1100 = 0110 (6)";

-- ===== LOGICAL NOT =====
report "--- LOGICAL NOT (NOT_OP) ---";
Op_Sel <= ALU_NOT;

A <= "1010"; wait for 20 ns;
report "Test12: NOT 1010 = 0101 (5)";

-- ===== COMPARISON =====
report "--- COMPARISON (CMP_OP) ---";
Op_Sel <= ALU_CMP;

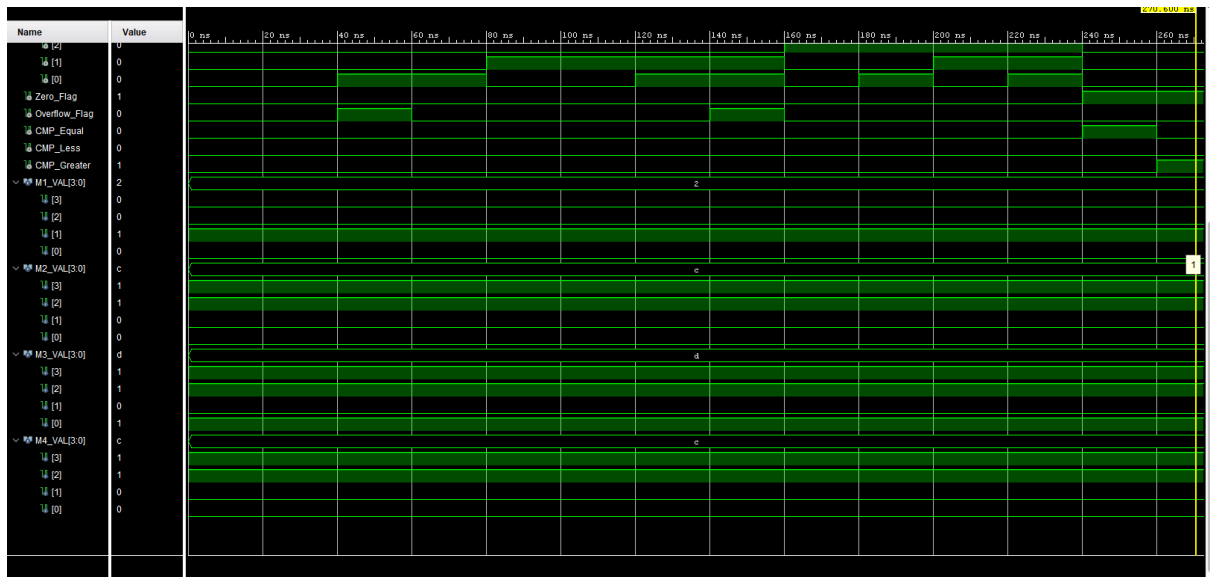
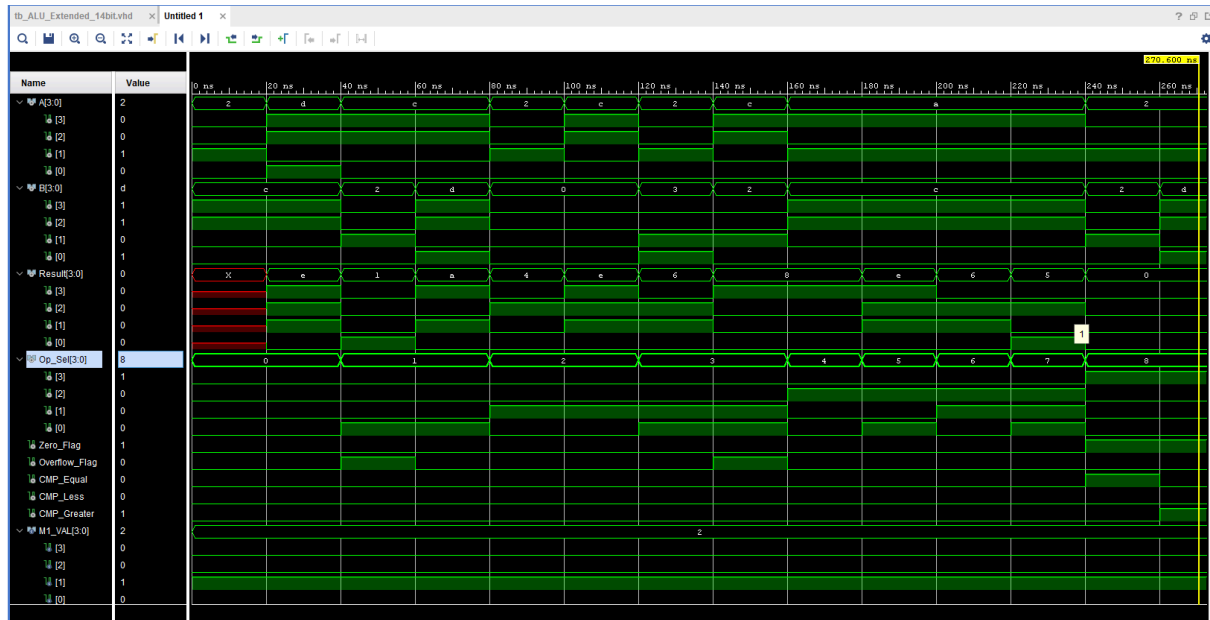
A <= M1_VAL; B <= M1_VAL; wait for 20 ns;
report "Test13: 2 == 2 => EQUAL=1";

A <= M1_VAL; B <= M3_VAL; wait for 20 ns;
report "Test14: 2 < -3? => LESS=1";

A <= M4_VAL; B <= M2_VAL; wait for 20 ns;
report "Test15: -4 == -4 => EQUAL=1";

report "===== ";
report "Extended ALU Tests Complete!";
report "===== ";
wait;
end process;
end Behavioral;

```



6.5.2 Instruction_Decoder_14bit

The decoder has the same conceptual structure as the base decoder but handles 12 cases instead of 4. Each opcode maps to a specific combination of:

- ALU operation code (4-bit `ALU_Op`)
- Register write enable / address
- Load selector setting
- Input enable (for IN instruction)
- Jump enable

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;
```

```

use work.Constants.all;

entity Instruction_Decoder_14bit is
    port (
        Instruction          : in  InstructionWord;  -- 14-bit
        Register_Value_For_Jump : in  DataBus;
        Register_Enable       : out RegisterSelect;
        Register_Select_A     : out RegisterSelect;
        Register_Select_B     : out RegisterSelect;
        ALU_Op_Sel           : out ALU_Op_Select;
        Immediate_Value       : out DataBus;
        Jump_Enable           : out STD_LOGIC;
        Jump_Address          : out ProgramCounter;
        Load_Select           : out STD_LOGIC;
        Input_Enable          : out STD_LOGIC
    );
end Instruction_Decoder_14bit;

architecture Behavioral of Instruction_Decoder_14bit is
    signal Opcode : std_logic_vector(3 downto 0);
    signal R1, R2 : std_logic_vector(2 downto 0);
    signal Func_Imm : std_logic_vector(3 downto 0);
    signal Imm4 : std_logic_vector(3 downto 0);
    signal JumpAddr3 : std_logic_vector(2 downto 0);

begin
    -- Extract fields from 14-bit instruction
    Opcode <= Instruction(13 downto 10);
    R1 <= Instruction(9 downto 7);
    R2 <= Instruction(6 downto 4);
    Func_Imm <= Instruction(3 downto 0);
    Imm4 <= Instruction(3 downto 0);
    JumpAddr3 <= Instruction(2 downto 0);

    decode: process(Opcode, R1, R2, Func_Imm, Imm4, JumpAddr3, Register_Value_For_Jump)
    begin
        -- Default values
        Jump_Enable <= '0';
        Immediate_Value <= "0000";
        Load_Select <= LOAD_IMMEDIATE;
        Register_Enable <= "000";
        ALU_Op_Sel <= ALU_ADD;
        Register_Select_A <= REG_ZERO;
        Register_Select_B <= REG_ZERO;
        Jump_Address <= "000";
        Input_Enable <= '0';

        case Opcode is
            when MOVI_OP => -- MOVI R1, d
                Immediate_Value <= Imm4;
                Load_Select <= LOAD_IMMEDIATE;
                Register_Enable <= R1;
        end case;
    end process;
end Behavioral;

```

```

when ADD_OP => -- ADD R1, R2
    Register_Select_A <= R1;
    Register_Select_B <= R2;
    ALU_Op_Sel      <= ALU_ADD;
    Load_Select     <= LOAD_REGISTER;
    Register_Enable  <= R1;

when SUB_OP => -- SUB R1, R2
    Register_Select_A <= R1;
    Register_Select_B <= R2;
    ALU_Op_Sel      <= ALU_SUB;
    Load_Select     <= LOAD_REGISTER;
    Register_Enable  <= R1;

when NEG_OP => -- NEG R1
    Register_Select_A <= REG_ZERO;
    Register_Select_B <= R1;
    ALU_Op_Sel      <= ALU_NEG;
    Load_Select     <= LOAD_REGISTER;
    Register_Enable  <= R1;

when JZR_OP => -- JZR R1, d
    Register_Select_A <= R1;
    Register_Enable   <= "000";
    if Register_Value_For_Jump = "0000" then
        Jump_Enable   <= '1';
        Jump_Address   <= JumpAddr3;
    end if;

when IN_OP => -- IN R1
    Register_Enable <= R1;
    Input_Enable   <= '1';
    Load_Select   <= LOAD_IMMEDIATE;

when MUL_OP => -- MUL R1, R2
    Register_Select_A <= R1;
    Register_Select_B <= R2;
    ALU_Op_Sel      <= ALU_MUL;
    Load_Select     <= LOAD_REGISTER;
    Register_Enable  <= R1;

when AND_OP => -- AND R1, R2
    Register_Select_A <= R1;
    Register_Select_B <= R2;
    ALU_Op_Sel      <= ALU_AND;
    Load_Select     <= LOAD_REGISTER;
    Register_Enable  <= R1;

when OR_OP => -- OR R1, R2
    Register_Select_A <= R1;
    Register_Select_B <= R2;
    ALU_Op_Sel      <= ALU_OR;
    Load_Select     <= LOAD_REGISTER;

```

```

        Register_Enable    <= R1;

    when XOR_OP => -- XOR R1, R2
        Register_Select_A <= R1;
        Register_Select_B <= R2;
        ALU_Op_Sel        <= ALU_XOR;
        Load_Select       <= LOAD_REGISTER;
        Register_Enable    <= R1;

    when NOT_OP => -- NOT R1
        Register_Select_A <= R1;
        Register_Select_B <= REG_ZERO;
        ALU_Op_Sel        <= ALU_NOT;
        Load_Select       <= LOAD_REGISTER;
        Register_Enable    <= R1;

    when CMP_OP => -- CMP R1, R2 (no register write)
        Register_Select_A <= R1;
        Register_Select_B <= R2;
        ALU_Op_Sel        <= ALU_CMP;
        Load_Select       <= LOAD_REGISTER;
        Register_Enable    <= "000"; -- No write for comparison!

    when others =>
        null;
    end case;
end process;

end Behavioral;

```

tb_Instruction_Decoder_14bit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;
use work.Constants.all;

entity tb_Instruction_Decoder_14bit is
end tb_Instruction_Decoder_14bit;

architecture Behavioral of tb_Instruction_Decoder_14bit is
    component Instruction_Decoder_14bit
        port ( Instruction : in InstructionWord;
              Register_Value_For_Jump : in DataBus;
              Register_Enable : out RegisterSelect;
              Register_Select_A : out RegisterSelect;
              Register_Select_B : out RegisterSelect;
              ALU_Op_Sel : out ALU_Op_Select;
              Immediate_Value : out DataBus;

```

```

        Jump_Enable : out STD_LOGIC;
        Jump_Address : out ProgramCounter;
        Load_Select : out STD_LOGIC;
        Input_Enable : out STD_LOGIC);
end component;

signal Instruction : InstructionWord;
signal RegValJump : DataBus;
signal RegEn, RegSelA, RegSelB, JumpAddr : RegisterSelect;
signal ALU_Op : ALU_Op_Select;
signal ImmVal : DataBus;
signal JumpEn, LoadSel, InputEn : STD_LOGIC;

-- Test values from team index last 4 bits
constant TEST_R1 : RegisterSelect := "001"; -- R1
constant TEST_R2 : RegisterSelect := "010"; -- R2
constant TEST_IMM : DataBus := "0010";      -- 2
constant TEST_JUMP : ProgramCounter := "101"; -- 5

begin
    UUT: Instruction_Decoder_14bit port map (
        Instruction => Instruction,
        Register_Value_For_Jump => RegValJump,
        Register_Enable => RegEn,
        Register_Select_A => RegSelA,
        Register_Select_B => RegSelB,
        ALU_Op_Sel => ALU_Op,
        Immediate_Value => ImmVal,
        Jump_Enable => JumpEn,
        Jump_Address => JumpAddr,
        Load_Select => LoadSel,
        Input_Enable => InputEn
    );

    process
    begin
        report "===== ";
        report "Testing 14-bit Instruction Decoder";
        report "===== ";

        -- Test1: MOVI R1, 2 (0000 001 000 0010)
        report "Test1: MOVI R1, 2";
        Instruction <= "0000" & TEST_R1 & "000" & TEST_IMM;
        wait for 20 ns;
    end process;
end;

```

```

-- Test2: ADD R1, R2 (0001 001 010 0000)
report "Test2: ADD R1, R2";
Instruction <= "0001" & TEST_R1 & TEST_R2 & "0000";
wait for 20 ns;

-- Test3: SUB R1, R2 (0010 001 010 0000)
report "Test3: SUB R1, R2";
Instruction <= "0010" & TEST_R1 & TEST_R2 & "0000";
wait for 20 ns;

-- Test4: NEG R1 (0011 001 000 0000)
report "Test4: NEG R1";
Instruction <= "0011" & TEST_R1 & "000" & "0000";
wait for 20 ns;

-- Test5: JZR R0, 5 (0100 000 000 0101) - with R0=0
report "Test5: JZR R0, 5 (Jump enabled)";
RegValJump <= "0000";
Instruction <= "0100" & "000" & "000" & "0" & TEST_JUMP;
wait for 20 ns;

-- Test6: JZR R1, 5 (with R1/=0 - no jump)
report "Test6: JZR R1, 5 (R1/=0, no jump)";
RegValJump <= "0001";
Instruction <= "0100" & TEST_R1 & "000" & "0" & TEST_JUMP;
wait for 20 ns;

-- Test7: IN R1 (0101 001 000 0000)
report "Test7: IN R1";
Instruction <= "0101" & TEST_R1 & "000" & "0000";
wait for 20 ns;

-- Test8: MUL R1, R2 (0110 001 010 0000)
report "Test8: MUL R1, R2";
Instruction <= "0110" & TEST_R1 & TEST_R2 & "0000";
wait for 20 ns;

-- Test9: AND R1, R2 (0111 001 010 0000)
report "Test9: AND R1, R2";
Instruction <= "0111" & TEST_R1 & TEST_R2 & "0000";
wait for 20 ns;

-- Test10: OR R1, R2 (1000 001 010 0000)

```

```

report "Test10: OR R1, R2";
Instruction <= "1000" & TEST_R1 & TEST_R2 & "0000";
wait for 20 ns;

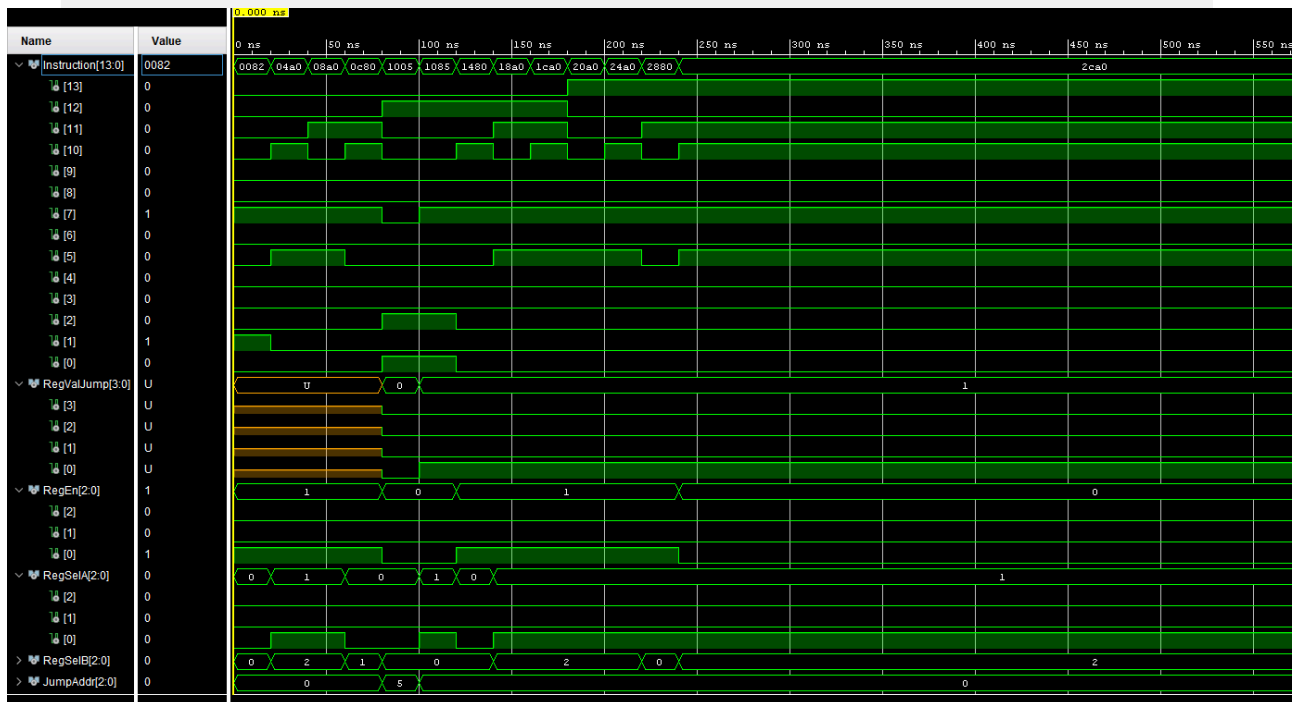
-- Test11: XOR R1, R2 (1001 001 010 0000)
report "Test11: XOR R1, R2";
Instruction <= "1001" & TEST_R1 & TEST_R2 & "0000";
wait for 20 ns;

-- Test12: NOT R1 (1010 001 000 0000)
report "Test12: NOT R1";
Instruction <= "1010" & TEST_R1 & "000" & "0000";
wait for 20 ns;

-- Test13: CMP R1, R2 (1011 001 010 0000)
report "Test13: CMP R1, R2 (No register write)";
Instruction <= "1011" & TEST_R1 & TEST_R2 & "0000";
wait for 20 ns;

report "===== ";
report "Instruction Decoder Tests Complete!";
report "===== ";
wait;
end process;
end Behavioral;

```





6.5.3 Top_NanoProcessor_14bit

The top-level entity is significantly more complex than the base processor. It must:

9. **Instantiate two complete datapaths** (ROM mode and manual mode)
10. **Multiplex their outputs** based on SW15
11. **Manage the slow clock** for ROM mode
12. **Implement edge detection** for Store_Btn and Toggle_Btn
13. **Connect the 16 switches** to both the IN instruction path and the manual mode decoder
14. **Drive all LED and display outputs**

The top-level entity has 15 ports - significantly more than the base processor's 6.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.BusDefinitions.all;
use work.Constants.all;

entity Top_NanoProcessor_14bit is
    port( Clock      : in  STD_LOGIC;
          Reset      : in  STD_LOGIC;      -- BTNC
          Store_Btn  : in  STD_LOGIC;      -- BTNR (Execute)
          Toggle_Btn : in  STD_LOGIC;      -- BTNL (Toggle display)
          Switches   : in  STD_LOGIC_VECTOR(15 downto 0);
          Overflow   : out STD_LOGIC;
          Zero       : out STD_LOGIC;
          CMP_Equal  : out STD_LOGIC;
          CMP_Less   : out STD_LOGIC;
          CMP_Greater : out STD_LOGIC;
          S_7Seg     : out STD_LOGIC_VECTOR(6 downto 0);
          anode      : out STD_LOGIC_VECTOR(3 downto 0);
          Data       : out DataBus);
end Top_NanoProcessor_14bit;

architecture Structural of Top_NanoProcessor_14bit is
    signal s_Mode : STD_LOGIC;
    signal s_R7_Value : DataBus;
    signal s_Zero, s_Overflow : STD_LOGIC;
    signal s_CMP_Equal, s_CMP_Less, s_CMP_Greater : STD_LOGIC;
    signal s_DisplayValue : DataBus;
    signal s_DisplayMode : STD_LOGIC := '0';
    signal s_Toggle_prev : STD_LOGIC := '0';
    signal s_Toggle_pulse : STD_LOGIC := '0';

    -- Store button edge detection signals
    signal store_prev : STD_LOGIC := '0';
    signal store_pulse : STD_LOGIC := '0';

    -- ROM Mode signals
    signal s_SlowClk : STD_LOGIC;
    signal s_Res : STD_LOGIC;
    signal s_PCCurrent : ProgramCounter;
    signal s_PCNext : ProgramCounter;
    signal s_JumpAddr : ProgramCounter;
    signal s_JumpEn : STD_LOGIC;
    signal s_SelectedAddr : ProgramCounter;

```

```

signal s_Instruction : InstructionWord;
signal s_RegEn : RegisterSelect;
signal s_RegSelA : RegisterSelect;
signal s_RegSelB : RegisterSelect;
signal s_ALU_Op_Sel : ALU_Op_Select;
signal s_ImmValue : DataBus;
signal s_LoadSel : STD_LOGIC;
signal s_InputEn : STD_LOGIC;
signal s_RegFileOutputs : RegisterFile;
signal s_OperandA, s_OperandB : DataBus;
signal s_ALUResult : DataBus;
signal s_WriteData : DataBus;
signal s_InputData : DataBus;
signal rom_r7 : DataBus;
signal rom_zero, rom_overflow : STD_LOGIC;
signal rom_cmp_eq, rom_cmp_lt, rom_cmp_gt : STD_LOGIC;

-- Manual Mode signals
signal manual_regs : RegisterFile := (others => (others => '0'));
signal manual_r7 : DataBus;

-- Manual mode decoded signals
signal manual_opcode : std_logic_vector(3 downto 0);
signal manual_R1 : integer range 0 to 7;
signal manual_R2 : integer range 0 to 7;
signal manual_imm : integer range 0 to 15;
signal manual_alu_op : ALU_Op_Select;

-- Manual mode data to write
signal manual_write_data : DataBus;
signal manual_do_write : STD_LOGIC;

-- Manual mode ALU results
signal manual_alu_result : DataBus;
signal manual_alu_zero : STD_LOGIC;
signal manual_alu_overflow : STD_LOGIC;
signal manual_alu_cmp_eq : STD_LOGIC;
signal manual_alu_cmp_lt : STD_LOGIC;
signal manual_alu_cmp_gt : STD_LOGIC;

signal manual_alu_inputA : DataBus;

-- Manual mode output signals
signal manual_zero : STD_LOGIC;

```

```

signal manual_overflow : STD_LOGIC;
signal manual_cmp_eq : STD_LOGIC;
signal manual_cmp_lt : STD_LOGIC;
signal manual_cmp_gt : STD_LOGIC;

-- Manual mode operand B (properly sized)
signal manual_operandB : DataBus;

-- Components
component Slow_Clk
    Generic (SIMULATION_MODE : boolean := false);
    Port ( Clk_in : in STD_LOGIC; Clk_out : out STD_LOGIC);
end component;

component Program_Counter
    Port ( PC_Next : in ProgramCounter; Res : in STD_LOGIC; Clk : in STD_LOGIC;
          PC_Current : out ProgramCounter);
end component;

component PC_Adder
    port ( current_address : in ProgramCounter; next_address : out ProgramCounter);
end component;

component Address_Selector
    port ( Sequential_Address : in ProgramCounter; Jump_Address : in
ProgramCounter;
          Jump_Enable : in std_logic; Selected_Address : out ProgramCounter);
end component;

component Program_ROM_14bit
    port( program_counter : in ProgramCounter; instruction_out : out
InstructionWord);
end component;

component Instruction_Decoder_14bit
    port ( Instruction : in InstructionWord; Register_Value_For_Jump : in DataBus;
          Register_Enable : out RegisterSelect; Register_Select_A : out
RegisterSelect;
          Register_Select_B : out RegisterSelect; ALU_Op_Sel : out ALU_Op_Select;
          Immediate_Value : out DataBus; Jump_Enable : out STD_LOGIC;
          Jump_Address : out ProgramCounter; Load_Select : out STD_LOGIC;
          Input_Enable : out STD_LOGIC);
end component;

```

```

component Load_Selector_Input
    Port ( RegisterValue : in DataBus; ImmediateValue : in DataBus;
          InputValue : in DataBus; LoadSelect : in STD_LOGIC;
          InputEnable : in STD_LOGIC; OutputData : out DataBus);
end component;

component Register_Bank
    Port ( Data : in DataBus; Reset : in STD_LOGIC; Reg_En : in RegisterSelect;
          Clock : in STD_LOGIC; Register_Outputs : out RegisterFile);
end component;

component RegisterData_Multiplexer
    Port ( DataSources : in RegisterFile; SelectAddress : in RegisterSelect;
          OutputData : out DataBus);
end component;

component ALU_Extended_14bit
    Port ( Input_A : in DataBus; Input_B : in DataBus; Op_Sel : in ALU_Op_Select;
          Result : out DataBus; Zero_Flag : out STD_LOGIC; Overflow_Flag : out
STD_LOGIC;
          CMP_Equal : out STD_LOGIC; CMP_Less : out STD_LOGIC; CMP_Greater : out
STD_LOGIC);
end component;

component Input_Selector
    Port ( Switches : in DataBus; Input_Enable : in STD_LOGIC; OutputData : out
DataBus);
end component;

component Display_Controller
    Port ( clk : in STD_LOGIC; Data_in : in DataBus;
          seg : out STD_LOGIC_VECTOR(6 downto 0); an : out STD_LOGIC_VECTOR(3
downto 0));
end component;

begin
    s_Mode <= Switches(15);
    s_Res <= Reset;

    -- Convert switches to integers for safe indexing
    manual_opcode <= Switches(13 downto 10);
    manual_R1 <= to_integer(unsigned(Switches(9 downto 7)));
    manual_R2 <= to_integer(unsigned(Switches(6 downto 4)));
    manual_imm <= to_integer(unsigned(Switches(3 downto 0)));

```

```

-- Properly size manual operand B
process(Switches, manual_regs, manual_R2, manual_imm)
begin
    if Switches(14) = '0' then
        -- Register mode: use value from register R2
        if manual_R2 >= 0 and manual_R2 <= 7 then
            manual_operandB <= manual_regs(manual_R2);
        else
            manual_operandB <= (others => '0');
        end if;
    else
        -- Immediate mode: convert integer to 4-bit DataBus
        manual_operandB <= std_logic_vector(to_unsigned(manual_imm, 4));
    end if;
end process;

-- Check bounds for manual_R1 and assign to manual_alu_inputA
process(manual_R1, manual_regs)
begin
    if manual_R1 >= 0 and manual_R1 <= 7 then
        manual_alu_inputA <= manual_regs(manual_R1);
    else
        manual_alu_inputA <= (others => '0');
    end if;
end process;

-- Store button edge detection (proper style)
process(Clock)
begin
    if rising_edge(Clock) then
        store_prev <= Store_Btn;
        store_pulse <= Store_Btn and not store_prev;
    end if;
end process;

-- Toggle button edge detection
process(Clock)
begin
    if rising_edge(Clock) then
        s_Toggle_prev <= Toggle_Btn;
        s_Toggle_pulse <= Toggle_Btn and not s_Toggle_prev;
        if s_Toggle_pulse = '1' then
            s_DisplayMode <= not s_DisplayMode;
        end if;
    end if;
end process;

```

```

        end if;
    end if;
end process;

-- =====
-- ROM MODE (SW15 = 0)
-- =====

U_SlowClk: Slow_Clk
    generic map (SIMULATION_MODE => false)
    port map (Clk_in => Clock, Clk_out => s_SlowClk);

U_PC: Program_Counter port map (PC_Next => s_SelectedAddr, Res => s_Res, Clk =>
s_SlowClk, PC_Current => s_PCCurrent);
U_PC_Adder: PC_Adder port map (current_address => s_PCCurrent, next_address =>
s_PCNext);
U_AddrSel: Address_Selector port map (Sequential_Address => s_PCNext, Jump_Address
=> s_JumpAddr,
                                     Jump_Enable => s_JumpEn, Selected_Address =>
s_SelectedAddr);

U_ROM: Program_ROM_14bit port map (program_counter => s_PCCurrent, instruction_out
=> s_Instruction);
U_ID: Instruction_Decoder_14bit port map (Instruction => s_Instruction,
Register_Value_For_Jump => s_OperandA,
                                     Register_Enable => s_RegEn,
Register_Select_A => s_RegSelA,
                                     Register_Select_B => s_RegSelB,
ALU_Op_Sel => s_ALU_Op_Sel,
                                     Immediate_Value => s_ImmValue,
Jump_Enable => s_JumpEn,
                                     Jump_Address => s_JumpAddr, Load_Select
=> s_LoadSel,
                                     Input_Enable => s_InputEn);

U_Input: Input_Selector port map (Switches => Switches(3 downto 0), Input_Enable =>
s_InputEn, OutputData => s_InputData);

U_LoadSel: Load_Selector_Input port map (RegisterValue => s_ALUResult,
ImmediateValue => s_ImmValue,
                                     InputValue => s_InputData, LoadSelect =>
s_LoadSel,
                                     InputEnable => s_InputEn, OutputData =>
s_WriteData);

```

```

    U_RegBank: Register_Bank port map (Data => s_WriteData, Reset => s_Res, Reg_En =>
s_RegEn,
                                     Clock => s_SlowClk, Register_Outputs =>
s_RegFileOutputs);

    U_MuxA: RegisterData_Multiplexer port map (DataSources => s_RegFileOutputs,
SelectAddress => s_RegSelA, OutputData => s_OperandA);
    U_MuxB: RegisterData_Multiplexer port map (DataSources => s_RegFileOutputs,
SelectAddress => s_RegSelB, OutputData => s_OperandB);

    U_ALU: ALU_Extended_14bit port map (Input_A => s_OperandA, Input_B => s_OperandB,
Op_Sel => s_ALU_Op_Sel,
                                     Result => s_ALUResult, Zero_Flag => rom_zero,
Overflow_Flag => rom_overflow,
                                     CMP_Equal => rom_cmp_eq, CMP_Less =>
rom_cmp_lt, CMP_Greater => rom_cmp_gt);

    rom_r7 <= s_RegFileOutputs(7);

    -- =====
    -- MANUAL MODE (SW15 = 1)
    -- =====

    -- Determine ALU operation based on manual opcode
    process(manual_opcode)
    begin
        case manual_opcode is
            when "0001" => manual_alu_op <= ALU_ADD;
            when "0010" => manual_alu_op <= ALU_SUB;
            when "0011" => manual_alu_op <= ALU_NEG;
            when "0110" => manual_alu_op <= ALU_MUL;
            when "0111" => manual_alu_op <= ALU_AND;
            when "1000" => manual_alu_op <= ALU_OR;
            when "1001" => manual_alu_op <= ALU_XOR;
            when "1010" => manual_alu_op <= ALU_NOT;
            when "1011" => manual_alu_op <= ALU_CMP;
            when others => manual_alu_op <= ALU_ADD;
        end case;
    end process;

    -- ALU for arithmetic/logic operations
    U_Manual_ALU: ALU_Extended_14bit port map (
        Input_A => manual_alu_inputA,

```

```

    Input_B => manual_operandB,
    Op_Sel => manual_alu_op,
    Result => manual_alu_result,
    Zero_Flag => manual_alu_zero,
    Overflow_Flag => manual_alu_overflow,
    CMP_Equal => manual_alu_cmp_eq,
    CMP_Less => manual_alu_cmp_lt,
    CMP_Greater => manual_alu_cmp_gt);

-- Determine what value to write based on opcode
process(manual_opcode, manual_imm, Switches, manual_alu_result)
begin
    -- Default values
    manual_write_data <= (others => '0');
    manual_do_write <= '0';

    -- MOVI: use immediate value directly (NO ALU!)
    if manual_opcode = "0000" then
        manual_write_data <= std_logic_vector(to_unsigned(manual_imm, 4));
        manual_do_write <= '1';

    -- IN: use switch value directly (NO ALU!)
    elsif manual_opcode = "0101" then
        manual_write_data <= Switches(3 downto 0);
        manual_do_write <= '1';

    -- CMP: do NOT write to register!
    elsif manual_opcode = "1011" then
        manual_write_data <= (others => '0');
        manual_do_write <= '0';

    -- ADD, SUB, MUL, AND, OR, XOR, NOT, etc.
    else
        manual_write_data <= manual_alu_result;
        manual_do_write <= '1';
    end if;
end process;

-- Manual mode flags (from CMP or ALU)
process(manual_opcode, manual_alu_cmp_eq, manual_alu_cmp_lt, manual_alu_cmp_gt,
        manual_alu_zero, manual_alu_overflow)
begin
    -- Default values
    manual_cmp_eq <= '0';

```

```

    manual_cmp_lt <= '0';
    manual_cmp_gt <= '0';
    manual_zero <= '0';
    manual_overflow <= '0';

    if manual_opcode = "1011" then
        -- CMP instruction: use comparison flags from ALU
        manual_cmp_eq <= manual_alu_cmp_eq;
        manual_cmp_lt <= manual_alu_cmp_lt;
        manual_cmp_gt <= manual_alu_cmp_gt;
    else
        -- Normal ALU operations: use zero/overflow flags
        manual_zero <= manual_alu_zero;
        manual_overflow <= manual_alu_overflow;
    end if;
end process;

-- Manual mode register storage (using proper edge detection)
process(Clock, Reset)
begin
    if Reset = '1' then
        for i in 0 to 7 loop
            manual_regs(i) <= (others => '0');
        end loop;
    elsif rising_edge(Clock) then
        if store_pulse = '1' and manual_do_write = '1' and manual_R1 >= 0 and
manual_R1 <= 7 then
            manual_regs(manual_R1) <= manual_write_data;
        end if;
    end if;
end process;

manual_r7 <= manual_regs(7);

-- =====
-- OUTPUTS
-- =====

-- Select R7 value based on mode
s_R7_Value <= rom_r7 when s_Mode = '0' else manual_r7;
s_Zero <= rom_zero when s_Mode = '0' else manual_zero;
s_Overflow <= rom_overflow when s_Mode = '0' else manual_overflow;
s_CMP_Equal <= rom_cmp_eq when s_Mode = '0' else manual_cmp_eq;
s_CMP_Less <= rom_cmp_lt when s_Mode = '0' else manual_cmp_lt;

```

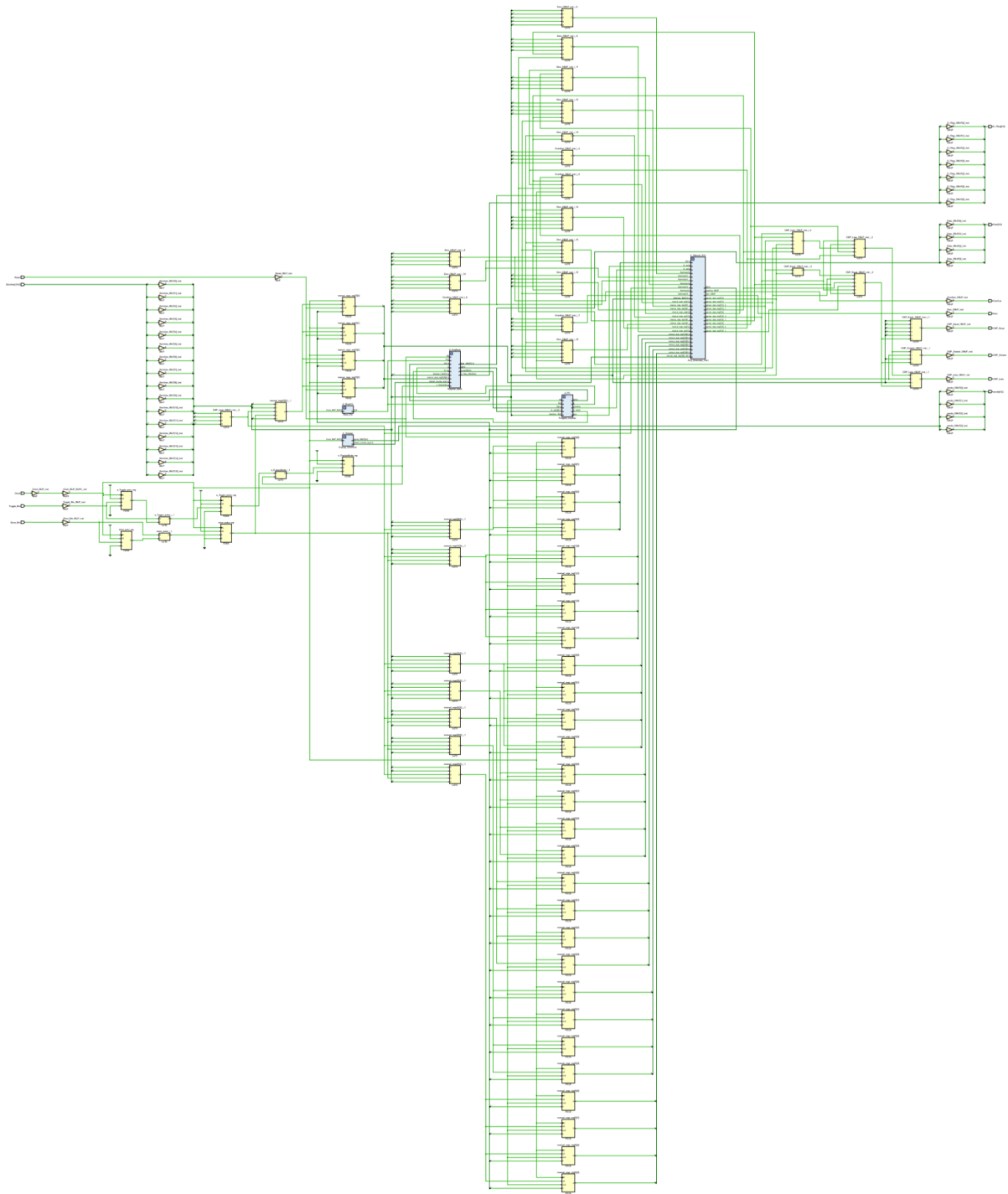
```
s_CMP_Greater <= rom_cmp_gt when s_Mode = '0' else manual_cmp_gt;

Data <= s_R7_Value;
Zero <= s_Zero;
Overflow <= s_Overflow;
CMP_Equal <= s_CMP_Equal;
CMP_Less <= s_CMP_Less;
CMP_Greater <= s_CMP_Greater;

-- Display selection (BTNL toggles)
s_DisplayValue <= Switches(3 downto 0) when s_DisplayMode = '1' else s_R7_Value;

U_Display: Display_Controller port map (clk => Clock, Data_in => s_DisplayValue,
seg => S_7Seg, an => anode);

end Structural;
```



tb_Top_NanoProcessor_14bit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_Top_NanoProcessor_14bit is
```

```

end tb_Top_NanoProcessor_14bit;

architecture Behavioral of tb_Top_NanoProcessor_14bit is
    component Top_NanoProcessor_14bit
        port( Clock : in STD_LOGIC; Reset : in STD_LOGIC;
            Store_Btn : in STD_LOGIC; Toggle_Btn : in STD_LOGIC;
            Switches : in STD_LOGIC_VECTOR(15 downto 0);
            Overflow : out STD_LOGIC; Zero : out STD_LOGIC;
            CMP_Equal : out STD_LOGIC; CMP_Less : out STD_LOGIC; CMP_Greater : out
STD_LOGIC;

            S_7Seg : out STD_LOGIC_VECTOR(6 downto 0);
            anode : out STD_LOGIC_VECTOR(3 downto 0);
            Data : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal Clock : STD_LOGIC := '0';
    signal Reset : STD_LOGIC := '1';
    signal Store_Btn, Toggle_Btn : STD_LOGIC := '0';
    signal Switches : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
    signal Overflow, Zero : STD_LOGIC;
    signal CMP_Equal, CMP_Less, CMP_Greater : STD_LOGIC;
    signal S_7Seg : STD_LOGIC_VECTOR(6 downto 0);
    signal anode : STD_LOGIC_VECTOR(3 downto 0);
    signal Data : STD_LOGIC_VECTOR(3 downto 0);

    constant CLK_PERIOD : time := 10 ns;

    function seg_to_char(seg : std_logic_vector(6 downto 0)) return string is
    begin
        case seg is
            when "1000000" => return "0";
            when "1111001" => return "1";
            when "0100100" => return "2";
            when "0110000" => return "3";
            when "0011001" => return "4";
            when "0010010" => return "5";
            when "0000010" => return "6";
            when "1111000" => return "7";
            when "0111111" => return "-";
            when others => return "?";
        end case;
    end function;

begin

```

```

    UUT: Top_NanoProcessor_14bit port map (
        Clock => Clock, Reset => Reset,
        Store_Btn => Store_Btn, Toggle_Btn => Toggle_Btn,
        Switches => Switches,
        Overflow => Overflow, Zero => Zero,
        CMP_Equal => CMP_Equal, CMP_Less => CMP_Less, CMP_Greater => CMP_Greater,
        S_7Seg => S_7Seg, anode => anode, Data => Data
    );
    Clock <= not Clock after CLK_PERIOD/2;
    process
    begin
        report "=====";
        report "Testing 14-bit Extended Nanoprocessor";
        report "ROM Mode Program: Count from -8 to +7";
        report "=====";

        -- Apply reset
        report "Step 1: Applying RESET...";
        Reset <= '1';
        wait for 100 ns;

        -- Release reset (ROM mode, SW15=0)
        report "Step 2: Releasing RESET - ROM mode starting...";
        Reset <= '0';

        -- Run simulation
        wait for 5000 ns;

        -- Display results
        report "";
        report "=====";
        report "SIMULATION COMPLETE";
        report "=====";
        report "Final R7 value (Data) = " & integer'image(to_integer(unsigned(Data)));
        report "Zero flag = " & std_logic'image(Zero);
        report "Overflow flag = " & std_logic'image(Overflow);
        report "=====";
        wait;
    end process;
end Behavioral;

```

6.5.4 Load_Selector_Input

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;
use work.Constants.all;

entity Load_Selector_Input is
    Port ( RegisterValue : in  DataBus;
          ImmediateValue : in  DataBus;
          InputValue      : in  DataBus;
          LoadSelect      : in  STD_LOGIC;
          InputEnable      : in  STD_LOGIC;
          OutputData       : out DataBus);
end Load_Selector_Input;

architecture Behavioral of Load_Selector_Input is
    signal normal_selection : DataBus;
begin
    -- Normal 2-way mux (immediate vs register)
    normal_selection <= ImmediateValue when LoadSelect = LOAD_IMMEDIATE else
RegisterValue;

    -- Input has highest priority: if InputEnable=1, use InputValue
    OutputData <= InputValue when InputEnable = '1' else normal_selection;

end Behavioral;
```

tb_Load_Selector_Input

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.BusDefinitions.all;
use work.Constants.all;

entity tb_Load_Selector_Input is
end tb_Load_Selector_Input;

architecture Behavioral of tb_Load_Selector_Input is
    component Load_Selector_Input
        Port ( RegisterValue : in DataBus; ImmediateValue : in DataBus;
              InputValue : in DataBus; LoadSelect : in STD_LOGIC;
              InputEnable : in STD_LOGIC; OutputData : out DataBus);
    end component;

    signal RegVal, ImmVal, InputVal, OutputData : DataBus;
```

```

signal LoadSel, InputEn : STD_LOGIC;

constant REG_VAL : DataBus := "0010";  -- 2
constant IMM_VAL : DataBus := "1100";  -- 12
constant INPUT_VAL : DataBus := "0111"; -- 7

begin
  UUT: Load_Selector_Input port map (
    RegisterValue => RegVal, ImmediateValue => ImmVal,
    InputValue => InputVal, LoadSelect => LoadSel,
    InputEnable => InputEn, OutputData => OutputData
  );

  process
  begin
    report "===== ";
    report "Testing Load Selector with Input Priority";
    report "Reg=2(0010), Imm=12(1100), Input=7(0111)";
    report "===== ";

    RegVal <= REG_VAL; ImmVal <= IMM_VAL; InputVal <= INPUT_VAL;

    -- Test1: Input disabled, LoadSelect=0 => Immediate
    report "Test1: InputEn=0, LoadSel=0 => Output=Immediate(1100=12)";
    InputEn <= '0'; LoadSel <= LOAD_IMMEDIATE;
    wait for 20 ns;

    -- Test2: Input disabled, LoadSelect=1 => Register
    report "Test2: InputEn=0, LoadSel=1 => Output=Register(0010=2)";
    LoadSel <= LOAD_REGISTER;
    wait for 20 ns;

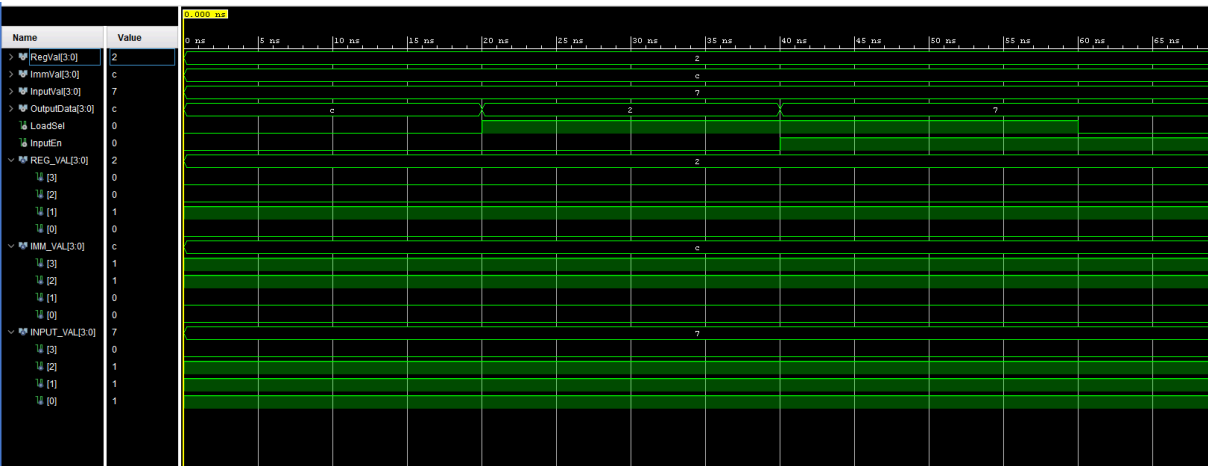
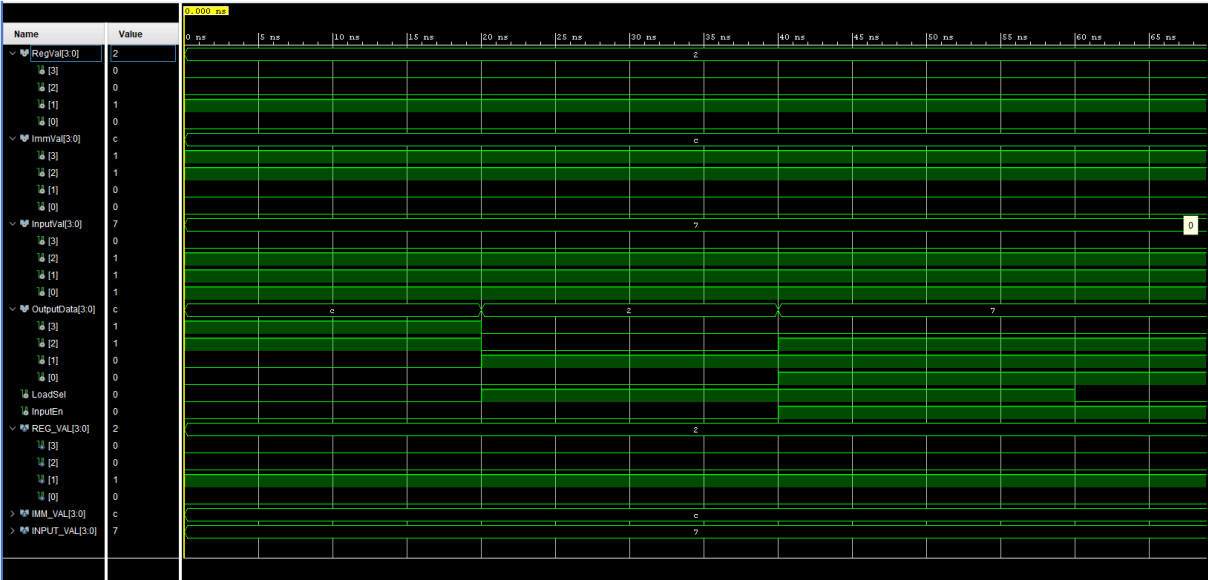
    -- Test3: Input enabled => Overrides both (Input has priority)
    report "Test3: InputEn=1 => Output=Input(0111=7) (priority)";
    InputEn <= '1';
    wait for 20 ns;

    -- Test4: Input enabled, change LoadSel (still Input)
    report "Test4: InputEn=1, LoadSel=0 => Still Input(7)";
    LoadSel <= LOAD_IMMEDIATE;
    wait for 20 ns;

    report "===== ";
    report "Load Selector Tests Complete!";
  end process;
end;

```

```
report "=====";  
wait;  
end process;  
end Behavioral;
```



6.5.5 Program ROM

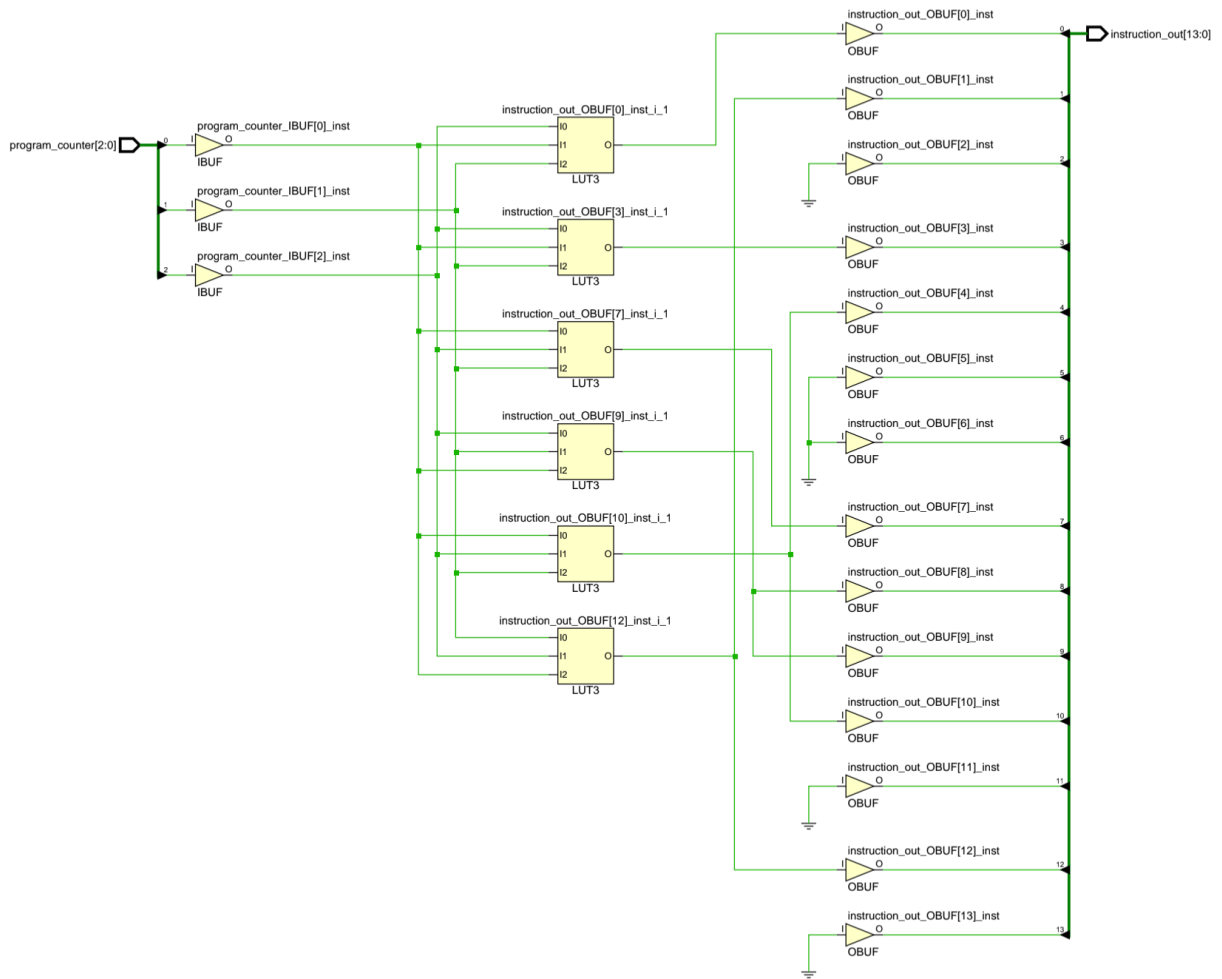
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.BusDefinitions.all;

entity Program_ROM_14bit is
    port( program_counter : in  ProgramCounter;
          instruction_out  : out InstructionWord);
end Program_ROM_14bit;

architecture Behavioral of Program_ROM_14bit is
    type instruction_memory_type is array (0 to 7) of std_logic_vector(13 downto 0);

    -- Format: 0000 RRR RRR FFFF (14 bits total)
    --           4    3    3    4 = 14 bits
    --
    -- MOVI R7, -8: 0000 111 000 1000 = "00001110001000"
    -- MOVI R1, 1: 0000 001 000 0001 = "00000010000001"
    -- ADD R7, R1: 0001 111 001 0000 = "00011110010000"
    -- JZR R0, 2: 0100 000 000 0010 = "01000000000010"
    --
    signal program_instructions : instruction_memory_type := (
        "00001110001000", -- 0: MOVI R7, -8      (R7 = -8 in signed)
        "00000010000001", -- 1: MOVI R1, 1      (R1 = 1)
        "00011110010000", -- 2: ADD R7, R1      (R7 = R7 + 1)
        "00011110010000", -- 3: ADD R7, R1      (R7 = R7 + 1)
        "00011110010000", -- 4: ADD R7, R1      (R7 = R7 + 1)
        "00011110010000", -- 5: ADD R7, R1      (R7 = R7 + 1)
        "00011110010000", -- 6: ADD R7, R1      (R7 = R7 + 1)
        "01000000000010"  -- 7: JZR R0, 2      (Jump to address 2 - loop)
    );

begin
    instruction_out <= program_instructions(to_integer(unsigned(program_counter)));
end Behavioral;
```



tb_Program_ROM_14bit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.BusDefinitions.all;

entity tb_Program_ROM_14bit is
end tb_Program_ROM_14bit;

architecture Behavioral of tb_Program_ROM_14bit is
    component Program_ROM_14bit
        port( program_counter : in ProgramCounter; instruction_out : out
InstructionWord);
    end component;

    signal addr : ProgramCounter;
    signal instr : InstructionWord;

```

```

-- Test addresses from team index last 3 bits
constant ADDR_M1 : ProgramCounter := "010"; -- 2
constant ADDR_M2 : ProgramCounter := "100"; -- 4
constant ADDR_M3 : ProgramCounter := "101"; -- 5

function to_string_14(instr : std_logic_vector(13 downto 0)) return string is
begin
    return std_logic'image(instr(13)) & std_logic'image(instr(12)) &
        std_logic'image(instr(11)) & std_logic'image(instr(10)) &
        std_logic'image(instr(9)) & std_logic'image(instr(8)) &
        std_logic'image(instr(7)) & std_logic'image(instr(6)) &
        std_logic'image(instr(5)) & std_logic'image(instr(4)) &
        std_logic'image(instr(3)) & std_logic'image(instr(2)) &
        std_logic'image(instr(1)) & std_logic'image(instr(0));
end function;

begin
    UUT: Program_ROM_14bit port map (program_counter => addr, instruction_out =>
instr);

    process
    begin
        report "=====";
        report "Testing 14-bit Program ROM";
        report "Program: Count from -8 to +7 using ADD";
        report "=====";

        -- Test all addresses 0-7
        for i in 0 to 7 loop
            addr <= std_logic_vector(to_unsigned(i, 3));
            wait for 20 ns;
            report "Addr=" & integer'image(i) & " Instruction=" & to_string_14(instr);
        end loop;

        -- Test specific addresses from team indexes
        report "";
        report "Testing team index addresses:";

        addr <= ADDR_M1;
        wait for 20 ns;
        report " Addr=M1(010=2): " & to_string_14(instr);

        addr <= ADDR_M2;
        wait for 20 ns;

```

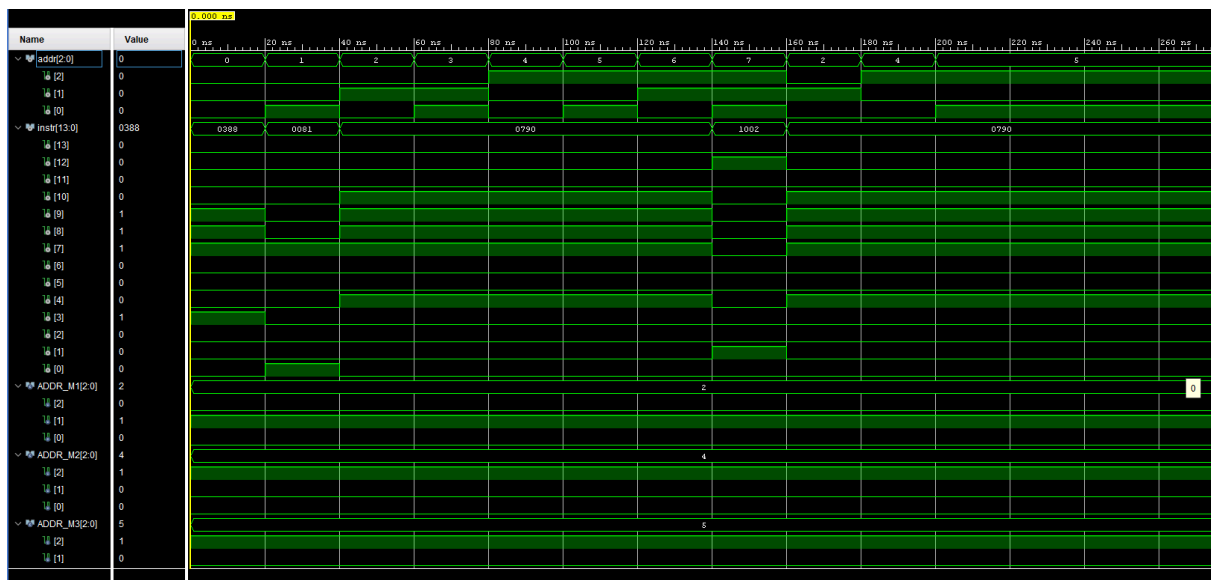
```

report " Addr=M2(100=4): " & to_string_14(instr);

addr <= ADDR_M3;
wait for 20 ns;
report " Addr=M3(101=5): " & to_string_14(instr);

report "===== ";
report "Program_ROM Tests Complete!";
report "===== ";
wait;
end process;
end Behavioral;

```



6.5 BusModifications

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

package BusDefinitions is
    -- 4-bit data bus (4-bit data, 14-bit instructions!)
    -- Your ALU processes 4-bit values - this is CORRECT!
    subtype DataBus is std_logic_vector(3 downto 0);

    -- 3-bit register select (for 8 registers)
    subtype RegisterSelect is std_logic_vector(2 downto 0);

    -- 3-bit program counter (for 8 ROM locations)
    subtype ProgramCounter is std_logic_vector(2 downto 0);

```

```

-- 14-bit instruction word (EXTENDED!)
subtype InstructionWord is std_logic_vector(13 downto 0);

-- 8-bit register enable (one-hot encoded)
subtype RegisterEnable is std_logic_vector(7 downto 0);

-- ALU operation select (4 bits)
subtype ALU_Op_Select is std_logic_vector(3 downto 0);

-- Register file array type
type RegisterFile is array (0 to 7) of DataBus;

end package BusDefinitions;

```

6.6 Constrains - Extended

```

## Clock signal
set_property PACKAGE_PIN W5 [get_ports Clock]
set_property IOSTANDARD LVCMOS33 [get_ports Clock]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports Clock]

## Switches SW0-SW15
set_property PACKAGE_PIN V17 [get_ports {Switches[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[0]}]
set_property PACKAGE_PIN V16 [get_ports {Switches[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[1]}]
set_property PACKAGE_PIN W16 [get_ports {Switches[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[2]}]
set_property PACKAGE_PIN W17 [get_ports {Switches[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[3]}]
set_property PACKAGE_PIN W15 [get_ports {Switches[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[4]}]
set_property PACKAGE_PIN V15 [get_ports {Switches[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[5]}]
set_property PACKAGE_PIN W14 [get_ports {Switches[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[6]}]
set_property PACKAGE_PIN W13 [get_ports {Switches[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[7]}]
set_property PACKAGE_PIN V2 [get_ports {Switches[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[8]}]
set_property PACKAGE_PIN T3 [get_ports {Switches[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[9]}]
set_property PACKAGE_PIN T2 [get_ports {Switches[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[10]}]
set_property PACKAGE_PIN R3 [get_ports {Switches[11]}]

```

```
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[11]}]
set_property PACKAGE_PIN W2 [get_ports {Switches[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[12]}]
set_property PACKAGE_PIN U1 [get_ports {Switches[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[13]}]
set_property PACKAGE_PIN T1 [get_ports {Switches[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[14]}]
set_property PACKAGE_PIN R2 [get_ports {Switches[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Switches[15]}]

## LEDs for Output R7 (Data bus) - LD0 to LD3
set_property PACKAGE_PIN U16 [get_ports {Data[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[0]}]
set_property PACKAGE_PIN E19 [get_ports {Data[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[1]}]
set_property PACKAGE_PIN U19 [get_ports {Data[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[2]}]
set_property PACKAGE_PIN V19 [get_ports {Data[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[3]}]

## Comparison Result Flags - Using available LEDs LD8, LD9, LD10
set_property PACKAGE_PIN V13 [get_ports CMP_Equal]
set_property IOSTANDARD LVCMOS33 [get_ports CMP_Equal]

set_property PACKAGE_PIN V3 [get_ports CMP_Less]
set_property IOSTANDARD LVCMOS33 [get_ports CMP_Less]

set_property PACKAGE_PIN W3 [get_ports CMP_Greater]
set_property IOSTANDARD LVCMOS33 [get_ports CMP_Greater]

## Zero Flag (LD14)
set_property PACKAGE_PIN P1 [get_ports Zero]
set_property IOSTANDARD LVCMOS33 [get_ports Zero]

## Overflow Flag (LD15)
set_property PACKAGE_PIN L1 [get_ports Overflow]
set_property IOSTANDARD LVCMOS33 [get_ports Overflow]

## 7-Segment Display Segments
set_property PACKAGE_PIN W7 [get_ports {S_7Seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {S_7Seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {S_7Seg[2]}]
```

```

set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {S_7Seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {S_7Seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {S_7Seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {S_7Seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {S_7Seg[6]}]

## 7-Segment Display Anodes
set_property PACKAGE_PIN U2 [get_ports {anode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[0]}]
set_property PACKAGE_PIN U4 [get_ports {anode[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[1]}]
set_property PACKAGE_PIN V4 [get_ports {anode[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[2]}]
set_property PACKAGE_PIN W4 [get_ports {anode[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[3]}]

## Buttons
set_property PACKAGE_PIN U18 [get_ports Reset]
set_property IOSTANDARD LVCMOS33 [get_ports Reset]

set_property PACKAGE_PIN T17 [get_ports Store_Btn]
set_property IOSTANDARD LVCMOS33 [get_ports Store_Btn]

set_property PACKAGE_PIN W19 [get_ports Toggle_Btn]
set_property IOSTANDARD LVCMOS33 [get_ports Toggle_Btn]

## Configuration settings
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

```

6.7 What Was Preserved

Component	Used in Extended?	Changes
Full Adder (FA)	✔ Yes	None
D Flip-Flop (D_FF)	✔ Yes	None
4-bit RCA (RCA_4)	✔ Yes	None
Register (Reg)	✔ Yes	None
Decoder_3to8	✔ Yes	None
LUT_16_7	✔ Yes	None

Register_Bank	✓ Yes	None
Program_Counter	✓ Yes	None
PC_Adder	✓ Yes	None
Address_Selector	✓ Yes	None
Slow_Clk	✓ Yes	None
Display_Controller	✓ Yes	None
All Mux components	✓ Yes	None

Reusing all primitive and functional components was possible because the extended processor is architecturally compatible with the base. The data width (4-bit registers, 3-bit PC) did not change - only the control logic grew. This validates the modular design approach: good decomposition enables extension without rework.

6.7 Extended Processor - Feature Summary

Feature	Base	Extended	Notes
Instruction width	12-bit	14-bit	+2 bits for wider opcode
Opcodes	4	12	4-bit opcode space
ALU operations	2 (add, sub)	9	+bitwise, CMP, MUL
Flags	2 (Zero, OVF)	5	+CMP_Eq, CMP_Lt, CMP_Gt
Input	None	SW0-SW3	Via IN instruction
Execution modes	1 (ROM)	2 (ROM + Manual)	SW15 selects
Interactive	No	Yes	BTNR executes instruction
Display toggle	No	Yes	BTNL toggles source
ROM program	1+2+3=6	Count -8 to +7	Loop programs

Part 7 - Conclusion & Reflection

7.1 Summary of Achievements

This lab project set out to design and implement a functioning NanoProcessor from first principles using VHDL. We delivered both phases successfully:

Base NanoProcessor (12-bit):

- Complete fetch-decode-execute processor with 4 instructions
- 8×4 -bit register file with zero register convention
- Ripple carry adder-based ALU with overflow detection
- 3-bit program counter with unconditional and conditional branching
- Seven-segment display output with sign support
- 17 VHDL testbenches, all passing simulation

Extended NanoProcessor (14-bit):

- 12-instruction ISA with 4-bit opcode
- 9-operation ALU including bitwise logic, multiplication, and comparison
- 5-flag system including comparison flags
- Hardware switch input via the IN instruction
- Dual-mode operation: ROM-based automatic execution and interactive manual mode
- Edge-detected button controls with display toggle functionality

7.2 Design Decisions That Worked Well

Hierarchical Component Design

Building everything bottom-up, from a Full Adder to the complete processor, was the single best decision made early in the project. Each component was small, focused, and independently testable. When integration issues arose, they were isolated quickly because the building blocks already had simulation confidence.

The Zero Register ($R0 = 0$)

Hardwiring $R0$ to zero eliminated the need for a dedicated immediate-zero instruction and made unconditional branching (`JZR  $R0$ , addr`) trivially easy to encode. This is the same design decision made in MIPS and RISC-V - it is a real engineering insight, not just a lab trick.

`SIMULATION_MODE` Generic in `Slow_Clk`

Without the `SIMULATION_MODE` generic, running the top-level simulation would require waiting for 100,000,000 simulated clock cycles before observing a single processor instruction. The generic makes simulation practical without any code duplication.

The NEG Implementation Using R0

Implementing NEG as $0 - R2$ (using R0 as the zero operand) reuses the ALU's subtract mode with no additional hardware. This kind of resource sharing - where an architectural convention ($R0 = 0$) enables a hardware optimisation - is exactly the kind of thinking that characterises good processor design.

Structural VHDL for Connectivity

Using structural VHDL for the top-level assembly (port maps, not process statements) meant that the tool could statically verify all connections at elaboration time. Misconnected signals were caught before simulation, not during.

7.3 Challenges Encountered

Clock Domain Management

The `SIMULATION_MODE` generic required careful management. Forgetting to set it to `false` before synthesis would produce a board image where the processor runs far too fast to observe. This was caught during initial hardware testing and became a documented step in the deployment checklist.

Signed Arithmetic in VHDL

VHDL's type system does not automatically infer signed or unsigned arithmetic from `STD_LOGIC_VECTOR`. Mixing `signed()`, `unsigned()`, and `STD_LOGIC_VECTOR` conversions without care leads to subtle bugs - for example, a subtraction that wraps incorrectly, or an overflow check that fires on the wrong cases. Careful use of IEEE 1076.3 numeric packages was essential.

Overflow Detection

The overflow detection logic for signed arithmetic is non-obvious, especially for the subtraction case. Getting the four-case expression right (`two positives → negative`, `two negatives → positive`, `positive minus negative → negative`, `negative minus positive → positive`) required careful derivation and explicit testbench verification.

Display Multiplexing

The seven-segment display's time-division multiplexing is invisible in simulation - when running at simulation speed, the counter cycles too fast for meaningful waveform analysis. This required a separate, dedicated testbench with explicit timing stimulus to verify digit cycling.

Manual Mode Edge Detection

The first version of manual mode executed the instruction continuously while BTNR was held, causing multiple writes per press. This was fixed by adding the two-register edge detector, which is a standard technique but not immediately obvious to someone new to hardware design.

Dual Register Files in Extended Mode

The extended processor requires two independent register files (one for ROM mode, one for manual mode). Initially, a single register file was attempted with mode switching, but ROM mode's automatic PC-driven writes conflicted with manual mode's button-triggered writes. The solution - two completely separate register file instances - is cleaner and eliminates the conflict at the architectural level.

7.4 What We Learned

"Hardware Is Not Software"

The most important lesson is that hardware design requires a completely different mental model from software. In software, code executes sequentially, one line at a time. In hardware, everything happens simultaneously. Every component is always running, every combinational path is always active, and signals propagate in parallel across the entire design. Thinking in terms of concurrent signal assignment (not sequential execution) is a fundamental shift.

Testbenches Are Not Optional

Every component that we skipped writing a testbench for eventually caused problems during integration. Every component with a good testbench was confidently reused without rework. The investment in testbench quality paid back many times over.

The Value of Fixed Instruction Formats

Having all instruction fields in fixed positions dramatically simplified the decoder. A variable-length or variable-position instruction format would have required a multi-stage parser. The trade-off is that some instructions waste bits (e.g., NEG only uses R1, not R2, but the R2 field exists anyway). For a processor this small, the simplicity is worth the wasted bits.

Abstraction Enables Reuse

By defining standard types (`DataBus`, `ProgramCounter`, `RegisterFile`) in a package file, every component speaks the same language. Connecting them requires no type conversions. When extending the processor, all base components dropped straight in - zero modification required because they all speak the same typed interface.

The ISA Is the Design's Core

Early choices about the instruction format constrain every subsequent decision. The 3-bit register addresses (limiting us to 8 registers), the 4-bit immediate (limiting values to 0–15), and the 3-bit jump address (limiting program size to 8 instructions) are all direct consequences of the instruction word width. A wider instruction word gives more flexibility but requires a wider memory and decoder. These trade-offs are real and non-trivial.

7.5 Possible Future Work

Given more time, the following extensions would be valuable:

Extension	Benefit
Shift instructions (SHL, SHR)	Essential for multiplication and division without a dedicated MUL unit
Load/Store to external RAM	A true stored-program computer with data memory, not just instruction memory
16-bit data path	Removes the severe 4-bit value limitation (max 15 or -8)
Deeper program memory	8 instructions is sufficient only for trivial programs; 256 would be practical
UART output	Send processor results over serial to a PC for logging and analysis
Conditional JZR variants	Jump if less-than, jump if equal, jump if overflow - using the CMP flags
Pipeline	Adding instruction pipelining to improve throughput (though complex for this scale)
Assembler	A simple Python assembler that converts human-readable assembly to the ROM constants

Part 8 - Appendix: Team Contributions & Machine Code

8.1 Team Contributions

This project was developed collaboratively across four team members. The component ownership below reflects the primary author for each file, though all members cross-reviewed and tested each other's work.

Member	Primary Components	Estimated Effort
240061C	- Built all primitive building blocks from gates upward - Created the register file foundation - Established bus type definitions for the entire project - Verified basic components through simulation - Created hardware constraints for BASYS3 board	~12 hrs
240066V	- Designed the arithmetic logic unit with overflow detection - Implemented the program counter and sequencing logic - Created clock division circuit for hardware visibility - Developed the 7-segment display LUT - Verified all arithmetic components through simulation	~13 hrs
240092V	- Built the register file subsystem - Implemented all multiplexers for data routing - Designed the seven-segment display controller with negative sign support - Documentation Leading - Verified all data path components through simulation	~12 hrs
240060X	- Integrated all components into complete processor - Extended processor from 12-bit to 14-bit ISA - Added 8 new instructions (SUB, MUL, AND, OR, XOR, NOT, CMP, IN) - Implemented dual execution mode (ROM + Manual)] - Led top-level simulation and verification - Developed manual mode control logic	~14 hrs

8.2 Machine Code Reference - Base Processor Demo Program

The following is the complete binary and hexadecimal encoding of the $1 + 2 + 3 = 6$ demo program loaded into the base processor's ROM. These encodings can be verified directly from the Program ROM simulation waveform in Part 5.

Address	Assembly Instruction	Binary (12-bit)	Hex
0	MOVI R7, 0	1011 1000 0000	0xB80
1	MOVI R1, 1	1000 1000 0001	0x881
2	MOVI R2, 2	1001 0000 0010	0x902
3	MOVI R3, 3	1001 1000 0011	0x983
4	ADD R7, R1	0011 1001 0000	0x390
5	ADD R7, R2	0011 1010 0000	0x3A0
6	ADD R7, R3	0011 1011 0000	0x3B0
7	JZR R0, 4	1100 0000 0100	0xC04

How the Program Executes Step by Step

Reset: PC $\leftarrow$ 000, all registers $\leftarrow$ 0000

```

Cycle 1: PC=0  $\rightarrow$  MOVI R7, 0  $\rightarrow$  R7 = 0000 (R7  $\leftarrow$  immediate 0)
Cycle 2: PC=1  $\rightarrow$  MOVI R1, 1  $\rightarrow$  R1 = 0001 (R1  $\leftarrow$  immediate 1)
Cycle 3: PC=2  $\rightarrow$  MOVI R2, 2  $\rightarrow$  R2 = 0010 (R2  $\leftarrow$  immediate 2)
Cycle 4: PC=3  $\rightarrow$  MOVI R3, 3  $\rightarrow$  R3 = 0011 (R3  $\leftarrow$  immediate 3)
Cycle 5: PC=4  $\rightarrow$  ADD R7, R1  $\rightarrow$  R7 = 0000 + 0001 = 0001
Cycle 6: PC=5  $\rightarrow$  ADD R7, R2  $\rightarrow$  R7 = 0001 + 0010 = 0011 (= 3)
Cycle 7: PC=6  $\rightarrow$  ADD R7, R3  $\rightarrow$  R7 = 0011 + 0011 = 0110 (= 6)  $\leftarrow$  Result!
Cycle 8: PC=7  $\rightarrow$  JZR R0, 4  $\rightarrow$  R0 = 0, so PC  $\leftarrow$  4 (loop)
Cycle 9: PC=4  $\rightarrow$  ADD R7, R1  $\rightarrow$  R7 = 0110 + 0001 = 0111 (= 7)
... continues adding each loop

```

The seven-segment display shows 6 after Cycle 7 before the loop repeats. On hardware at ~ 1 Hz, this gives approximately 7 seconds to observe each instruction step.

8.3 Instruction Encoding Reference - Bit-by-Bit

MOVI R7, 0 $\rightarrow$ `1011 1000 0000`

```

Bit 11-10: 10  $\rightarrow$  opcode MOVI
Bit 9- 7: 111  $\rightarrow$  destination R7
Bit 6- 4: 000  $\rightarrow$  (unused source field)
Bit 3- 0: 0000  $\rightarrow$  immediate value 0

```

ADD R7, R1 $\rightarrow$ `0011 1001 0000`

```

Bit 11-10: 00  $\rightarrow$  opcode ADD
Bit 9- 7: 111  $\rightarrow$  destination R7
Bit 6- 4: 001  $\rightarrow$  source R1
Bit 3- 0: 0000  $\rightarrow$  (unused immediate field)

```

JZR R0, 4 $\rightarrow$ `1100 0000 0100`

```

Bit 11-10: 11  $\rightarrow$  opcode JZR
Bit 9- 7: 000  $\rightarrow$  register to check (R0)
Bit 6- 4: 000  $\rightarrow$  (unused)

```

Bit 3- 0: 0100 → jump target address = 4

8.4 Seven-Segment Display LUT - Full Encoding Table

The LUT_16_7 component maps 4-bit values to 7-bit segment patterns (active-low, common-anode configuration):

Value (Hex)	Decimal	Segments	Binary	Display
0	0	1000000	0x40	0
1	1	1111001	0x79	1
2	2	0100100	0x24	2
3	3	0110000	0x30	3
4	4	0011001	0x19	4
5	5	0010010	0x12	5
6	6	0000010	0x02	6
7	7	1111000	0x78	7
8	8	0000000	0x00	8
9	9	0010000	0x10	9
A	10	0001000	0x08	A
b	11	0000011	0x03	b
C	12	1000110	0x46	C
d	13	0100001	0x21	d
E	14	0000110	0x06	E
F	15	0001110	0x0E	F
–	sign	0111111	0x3F	–
(blank)	none	1111111	0x7F	